

ICPC 算法模板



ICPC 算法模板

ICPC 算法模板	1
一、基础算法.....	6
单调栈.....	6
单调队列.....	7
汉诺塔问题.....	8
计数和基数排序.....	8
二、动态规划.....	10
倍增算法和 ST 表.....	10
换根 dp	12
最长递增子序列 LIS	14
轮廓线 dp	15
三、字符串.....	17
AC 自动机.....	17
KMP 算法.....	19
Manacher 最长回文子串	21
Z 算法(扩展 KMP).....	22
字符串哈希.....	23
四、数学	24
01 分数规划	24
gcd 和 lcm.....	26
MR 质数测试.....	27
Raney 引理.....	28
中国剩余定理.....	28
中国剩余定理扩展.....	30
乘法快速幂.....	32
二元一次不定方程(丢番图方程)	32
二项式反演.....	34
二项式定理.....	36
位运算相关.....	38
卡特兰数(因子计数法)	39

卡特兰数.....	40
向量空间线性基.....	42
哥德巴赫猜想.....	43
四则运算的位运算实现.....	44
埃氏筛质数.....	45
完美洗牌问题.....	46
容斥原理.....	48
常用一般推论.....	50
常见等差等比数列.....	51
康托展开.....	51
异或空间线性基(普通消元法)	53
异或空间线性基(高斯消元法)	54
异或空间线性基的合并.....	56
扩展中国剩余定理模板例题.....	56
扩展欧几里得算法求逆元.....	59
摩尔投票问题.....	60
摩尔投票问题 k 扩展.....	61
矩阵快速幂.....	62
积性函数.....	64
约瑟夫环.....	64
线性欧拉筛质数.....	64
组合数学.....	65
裴蜀定理和扩展欧几里得算法.....	66
试除法质数判断.....	67
质因数拆分.....	67
费马逆元和暴力逆元.....	68
进制转换.....	69
逆元的线性递推.....	70
逆康托展开.....	70
阶乘逆元的线性递推.....	73
随机数函数.....	73
高斯消元法(同余方程组)	74

高斯消元法(异或方程组)	76
高斯消元法(线性方程组)	78
高精度乘法	80
麦乐鸡块定理(笑了)	81
五、图论	82
1、最短路问题	82
01BFS	82
A 星算法	83
BellmanFordSPFA 单源最短路	84
Dijkstra 单源最短路	85
Floyd 多源最短路	89
LGSP 分层图最短路	89
TSP 最短路问题	92
同余最短路(两次绕圈法)	93
同余最短路 Djkt	95
负环和差分约束 Floyd	97
负环和差分约束 SPFA	98
2、树上问题	101
dfs 判断存在负环	101
LcaStjump 最低公共祖先	102
LcaTarjan 最低公共祖先	104
树上差分(点和边差分)	105
树的直径(两次 dfs 求解)	108
树的直径(树型 dp 求解)	110
树的重心(利用子树大小求解)	112
树的重心(利用子树节点数求解)	113
3、生成树	115
Kruskal 最小生成树	115
Prim 最小生成树	118
拓扑排序	123
六、计算几何	125
Pick 定理求格点数	125

三维几何质心.....	126
三维多边形面积.....	127
二维几何质心.....	130
二维多边形面积(鞋带定理).....	130
凸包(Andrew 算法).....	132
判断夹角类型.....	133
向量夹角.....	134
圆相关(二维).....	135
平面法向量.....	137
弧度制角的类型判断及转换.....	137
点乘和叉乘的计算及应用.....	138
点线和点面关系判断.....	142
点线映射和点面映射.....	143
点线距离.....	144
线与线的夹角.....	145
线面夹角和映射.....	147
七、数据结构.....	149
1、前缀树.....	149
普通前缀树.....	149
可持久化前缀树.....	151
2、手写堆.....	151
手写反向索引堆优化.....	151
手写堆.....	153
3、左偏树.....	155
K 短路问题.....	155
可持久化左偏树.....	161
左偏树.....	163
4、并查集.....	166
带权并查集.....	166
并查集.....	167
5、有序表专题.....	168
AVL 平衡树.....	168

FHQ 可持久化.....	174
Splay 树.....	178
TreapFHQ(经典非词频压缩).....	184
TreapFHQ(词频压缩).....	188
Treap 旋转树.....	192
可持久化文艺平衡树.....	197
左偏树.....	201
文艺平衡树(FHQ 懒标记).....	204
替罪羊树(词频压缩).....	206
替罪羊树(非词频压缩).....	211
笛卡尔树.....	215
跳表.....	217
6、树状数组.....	223
单点增加, 范围查询(二维数组).....	223
单点增加, 范围查询.....	223
树状数组.....	224
范围增加, 单点查询.....	229
范围增加, 范围查询(二维数组).....	230
范围增加, 范围查询.....	231
7、线段树.....	232
可持久化线段树(区间 mex 值).....	232
可持久化线段树(区间第 k 小, 单点+区间).....	235
可持久化线段树(懒更新, 范围操作).....	238
可持久化线段树(标记永久化).....	240
可持久化线段树(浮动区间最大上中位数).....	243
线段树区间最值问题.....	247
线段树历史最值问题.....	250
线段树和扫描线(矩阵周长并).....	254
线段树和扫描线(矩阵面积并).....	257

一、基础算法

单调栈

```
// 单调栈的应用很广泛：
// 贪心，dp 优化，很多场景都可以应用

// 模板：
// 单调栈求解左侧和右侧第一个严格小于的位置
stack<int>sta;
const int MAXN = 2e5;
int right[MAXN + 1];
int left[MAXN + 1];
int a[MAXN + 1];
void solve(int n) {
    // -1 标记不存在
    // 栈必须维护栈顶是最大值
    for (int i = 1; i <= n; i++) {
        // 右侧在弹出时进行维护
        while (sta.size() && a[i] < a[sta.top()]) {
            int t = sta.top();
            ::right[t] = i;
            sta.pop();
        }

        // 左侧在压入时进行维护
        // 若存在相同元素，和左侧第一个相同位置对齐
        if (sta.size()) {
            if (a[i] == a[sta.top()]) {
                ::left[i] = ::left[sta.top()];
            }
            else {
                ::left[i] = sta.top();
            }
        }
        else {
            ::left[i] = -1;
        }
        sta.push(i);
    }
    while (sta.size()) {
        int t = sta.top();
        ::right[t] = -1;
        sta.pop();
    }
}
```

```
}  
}
```

单调队列

```
#include<bits/stdc++.h>  
using namespace std;  
using ll = long long;  
  
int a[500000];  
  
void prove() {  
    int n = 1000;  
    // randomArray(n, a);  
    deque<int>q;//单调队列维护区间最大值  
    int len = 2;  
    //注意：务必在循环内维护真实的[l,r]区间  
    //队列内的左右端点为最大值服务,未必是真实左右端点  
    //若题目涉及真实[l,r]的相关信息,务必维护  
    for (int l = 1, r = 1;r <= n;r++) {  
        if (r - l + 1 > len) {  
            l++;  
        }  
        //维护右侧最大候选值  
        while (q.size() && a[q.back()] < a[r]) {  
            q.pop_back();  
        }  
        q.push_back(r);//新节点入队  
  
        //维护左侧合法下标  
        while (q.size() && q.front() < l) {  
            q.pop_front();  
        }  
  
        //输出区间最大值  
        //加上如下限定,区间长度严格=len,否则<=len  
        if (r - l + 1 == len) {  
            cout << l << "----" << r << ' ' << a[q.front()] << '\n';  
        }  
    }  
}
```

汉诺塔问题

```
#include<bits/stdc++.h>
using namespace std;
class TOH {
public:
    //经典汉诺塔问题:
    //把 A 堆的所有盘子移动到 C,B 作为工具堆
    //要求每个盘子都必须严格大于他下面的盘子
    using ll = long long;
    ll f(ll n, char cur, char tool, char goal) {
        //当前 cur 移动到 goal,tool 是工具盘
        if (n == 0) {
            return 0;
        }
        //递归
        ll ans = 0;
        ans = ans + f(n - 1, cur, goal, tool); //[1, n - 1] 移动到 tool 堆,
        goal 堆作为实际工具堆

        ans = ans + 1; //+1 表示最后第 n 个移动到 goal 堆
        cout << cur << "->" << goal << '\n'; //输出第 n 个盘移动方式

        ans = ans + f(n - 1, tool, cur, goal); //最后 tool 作为新的 cur 堆,把上
        面的 n-1 个移动到 goal

        return ans;
    }
};
```

计数和基数排序

```
#include<bits/stdc++.h>
using namespace std;
//计数排序,时间复杂度和空间复杂度 O(n)
//对数据的范围有严格要求
class CountSort {
private:
    static const int MAXN = 1e7;
    int cnt[MAXN + 1] = { 0 };
    //unordered_map<int,int>cnt
public:

    void Sort(int* a, int n, int maxval) {
        fill(cnt, cnt + maxval + 1, 0);
```

```

        for (int i = 1; i <= n; i++) {
            cnt[a[i]]++;
        }
        int idx = 0;
        for (int i = 0; i <= maxval; i++) {
            for (int j = 1; j <= cnt[i]; j++) {
                a[++idx] = i;
            }
        }
    }
};

//基数排序 RadixSort
//可以用于不同进制下的非负整数排序
//时间复杂度 O(n)
class RadixSort {
private:
    static const int MAXN = 1e7;
    int help[MAXN + 1]; //辅助数组
    int bit = 0;
    int GetHeightBit(int* a, int n, int base) {

        //获取最高位
        int maxval = 0;
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            maxval = max(maxval, a[i]);
        }
        while (maxval) {
            maxval /= base;
            ans++;
        }
        return ans;
    }

    void copy(int n, int* a, int* b) {
        //a->b
        for (int i = 1; i <= n; i++) {
            b[i] = a[i];
        }
    }

public:
    void Sort(int* a, int n, int base) {
        //合法下标 1~n
        bit = GetHeightBit(a, n, base); //获取在 BASE 下的最高有效位
    }
};

```

```

        for (int i = 1, offset = 1; i <= bit; i++, offset *= base) {

            //依次处理每一位
            vector<int>cnt(base, 0); //0~base-1
            for (int i = 1; i <= n; i++) {
                cnt[(a[i] / offset) % base]++;
            }

            //base 进制意义下的前缀和
            for (int i = 1; i < base; i++) {
                cnt[i] += cnt[i - 1];
            }

            //排序
            for (int i = n; i >= 1; i--) {
                help[cnt[(a[i] / offset) % base] - 1] = a[i];
            }
            copy(n, help, a);
        }
    }
};

```

二、动态规划

倍增算法和 ST 表

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//倍增表和倍增算法的理解核心:
//1、二进制的可拆分性
//2、二进制数学上天然的必然可组合性
//3、维护信息的可合并性
//4、动态规划的思想

//倍增表维护环状最短路问题
//https://www.luogu.com.cn/problem/P4155
const int MAXN = 2e5;
struct node {
    int idx;
    int st, ed;
};

```

```

}line[2 * MAXN + 1];
int stjump[2 * MAXN + 1][32];
int power = 32;
int ans[MAXN + 1];

//建立倍增表,核心代码
void buildStjump(int n) {
    power = ceil(log2(n));
    int e = n << 1;//line 的数量
    for (int i = 1, arrive = 1;i <= e;i++) { //arrive 可抵达的 line 编号
        while (arrive + 1 <= e && line[arrive + 1].st <= line[i].ed) {
            arrive++; //若可达
        }
        stjump[i][0] = arrive;
    }
    for (int p = 1;p <= power;p++) {
        for (int i = 1;i <= e;i++) {
            stjump[i][p] = stjump[stjump[i][p - 1]][p - 1];
        }
    }
}

int jump(int i, int m) {
    int aim = line[i].st + m;
    int cur = i, nxt, ans = 1;
    for (int p = power;p >= 0;p--) {
        nxt = stjump[cur][p];
        if (nxt != 0 && line[nxt].ed < aim) {
            ans += (1 << p);
            cur = nxt;
        }
    }
    return ans + 1; //stjump[cur][0],最后一步
}

bool cmp(node a, node b) {
    return a.st < b.st;
}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1;i <= n;i++) {
        cin >> line[i].st >> line[i].ed;
        line[i].idx = i;
        if (line[i].st > line[i].ed) {

```

```

        line[i].ed += m;
    }
}
sort(line + 1, line + n + 1, cmp);
for (int i = 1; i <= n; i++) {
    line[i + n] = line[i];
    line[i + n].st += m;
    line[i + n].ed += m;
}
buildStjump(n);
for (int i = 1; i <= n; i++) {
    ans[line[i].idx] = jump(i, m);
}
for (int i = 1; i <= n; i++) {
    cout << ans[i] << ' ';
}
}

```

换根 dp

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e6;

// 换根 dp: 解决快速查询以每个节点作为根的某个信息
// 需要维护每个节点的若干信息
// 一般需要两次 dfs
// 第一次递归回溯时自底向上获取所有节点信息，并获取 root 节点的答案
// 第二次通过自顶向下严格位置依赖的转移方程，获得所有节点的答案
// 核心：在于分析父节点和子节点之间通过若干信息的状态转移，最终得到关键信息

ll dp[MAXN + 1]; // 以 i 为根，所有节点抵达 i 的距离之和
int siz[MAXN + 1]; // 子树 i 的大小
ll sum[MAXN + 1]; // i 的所有儿子走向 i 的距离之和
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];
int idx = 0;
void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}
void dfsDp(int u, int fa, int n) {

```

```

    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            // !!! 核心代码如下: !!!
            // 节点 1 的儿子 2
            // 2 的所有子树节点走向 1, 再退一步(-siz[v])就抵达 2
            // 与 2 反向的 1 的所有子树节点走向 1, 再进一步(n-siz[v])就抵达 2
            dp[v] = dp[u] - siz[v] + (n - siz[v]);
            dfsDp(v, u, n);
        }
    }
}

void dfs(int u, int fa) {
    siz[u] = 1;
    sum[u] = 0;
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfs(v, u);
            siz[u] += siz[v];
            sum[u] = sum[v] + siz[v];
        }
    }
}

void solve() {
    // https://www.luogu.com.cn/problem/P3478
    int n;
    cin >> n;
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(1, 0);
    dp[1] = sum[1];
    dfsDp(1, 0, n);
    int best = 0;
    ll dist = -1e18;
    for (int i = 1; i <= n; i++) {
        if (dp[i] > dist) {
            dist = dp[i];
            best = i;
        }
    }
}

```

```
    cout << best << '\n';  
}
```

最长递增子序列 LIS

```
#include<bits/stdc++.h>  
using std::cin, std::cout;  
const int MAXN = 1e6;  
using ll = long long;  
//LIS:最长递增子序列(严格递增)  
  
int dp[MAXN + 1]; //dp[i]以 i 结尾的最长递增子序列  
int end[MAXN + 1]; //长度为 i 的最大结尾元素递增子序列  
  
int binary_search(int x, int n, int* arr) {  
    //查找首个>=x 的位置  
    //对于最长非严格递增子序列(即可以相等),求首个>x 的位置即可  
    int l = 1, r = n;  
    while (l <= r) {  
        int mid = (l + r) >> 1;  
        if (arr[mid] >= x) {  
            //对于最长非严格递增子序列,>x 时 r=mid-1  
            r = mid - 1;  
        }  
        else {  
            l = mid + 1;  
        }  
    }  
    return l;  
}  
  
//获取最长递增子序列的长度  
int LIS(int n, int* arr) {  
    int R = 1;  
    dp[1] = 1;  
    end[1] = arr[1];  
    for (int i = 2; i <= n; i++) {  
        int idx = binary_search(arr[i], R, end);  
        if (idx > R) {  
            end[++R] = arr[i];  
        }  
        else {  
            end[idx] = arr[i];  
        }  
        dp[i] = idx;  
    }  
}
```

```

    }
    return R;
}

```

轮廓线 dp

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 轮廓线 dp
// 时间复杂度:  $O(n * m * 2^m)$ 
// 普通状压:  $O(n * 2^m * 2^m)$ 
// 轮廓线 dp 核心是把前后两行的信息压缩到一个位信息里
// 通常的, 对于一个位状态  $s$ , 当处理第  $i$  个位置时,  $s[0 \dots i - 1]$  表示当前行新做出的决策,  $s[i \dots m]$  表示上一行的决策
// 处理完成后, 进入  $i + 1$ , 第  $i$  个位置就是当前行的决策状态
// 普通状压: 整行的位信息分开进行更新
// 轮廓线 dp: 依赖单一格子逐步递推
// https://www.luogu.com.cn/problem/P1879

const int mod = 1e8;

ll getMsk(ll x, int i) { return (x >> i) & 1; } // 获取第  $i$  位状态
ll revMsk(ll x, int i) { return x ^ (1LL << i); } // 第  $i$  位取反
ll setMsk(ll x, int i, int v) { return v == 0 ? (x & ~(1LL << i)) : (x | (1LL << i)); } // 设置第  $i$  位状态

int a[12][12];
int f[12][12][1 << 12]; // 空间压缩
int dp[13][1 << 12]; // 空间压缩
int prepare[1 << 12]; // 辅助数组

int f1(int i, int j, int s, int n, int m) {
    if (i == n) {
        return 1;
    }
    if (j == m) {
        return f1(i + 1, 0, s, n, m);
    }
    if (f[i][j][s] != -1) {
        return f[i][j][s];
    }
}

```

```

    int ans = f1(i, j + 1, setMsk(s, j, 0), n, m) % mod;
    // (矩阵位置可以放) 且 (第一列 或 左边没有放) 且 (上边没有放)
    if (a[i][j] == 1 && (j == 0 || getMsk(s, j - 1) == 0) && getMsk(s, j) == 0) {
        ans = (ans + f1(i, j + 1, setMsk(s, j, 1), n, m)) % mod;
    }
    f[i][j][s] = ans;
    return ans;
}

int dp1(int n, int m) {
    int maxs = 1 << m;
    //prepare 初始化为第 n 行的值, 全是 1
    fill(prepare, prepare + maxs, 1);

    for (int i = n - 1; i >= 0; i--) {
        //j==m 时, 直接复用下一行 j=0(i+1 行, j=0 时)的值
        for (int s = 0; s < maxs; s++) {
            dp[m][s] = prepare[s];
        }

        //j[m-1,0]
        for (int j = m - 1; j >= 0; j--) {
            for (int s = 0; s < maxs; s++) {
                dp[j][s] = dp[j + 1][setMsk(s, j, 0)];
                if (a[i][j] == 1 && (j == 0 || getMsk(s, j - 1) == 0) && getMsk(s, j) == 0) {
                    dp[j][s] = (dp[j][s] + dp[j + 1][setMsk(s, j, 1)]) % mod;
                }
            }
        }

        //设置 prepare, 供上一行(i-1 行)依赖
        for (int s = 0; s < maxs; s++) {
            prepare[s] = dp[0][s];
        }
    }

    return dp[0][0];
}

void solve() {
    int n, m;
    cin >> n >> m;

```

```

    int maxs = 1 << m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            fill(f[i][j], f[i][j] + maxs, -1);
            cin >> a[i][j];
        }
    }
    // cout << f1(0, 0, 0, n, m) << '\n';
    cout << dp1(n, m) << '\n';
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    solve();
    return 0;
}

```

三、字符串

AC 自动机

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// AC 自动机(优化版): 统计目标串在查询串中的出现次数
// tree(直通表): Trie 转移表, 直接存节点遇到字符后应跳到的节点, 已融合 fail, 避免回跳
// fail: 指向当前路径最长合法后缀对应的前缀节点, 类似 KMP 的 next
// (维护拥有和当前截止字符路径的合法后缀相同的前缀节点(且长度最长), 类似 kmp 的 next 数组)
// 统计: 遍历查询串, 对到达节点累加计数
// 汇总: 按 fail 反向建图, 自底向上累加计数到根节点

// AC 自动机可以实现除收集词频以外的功能, 可根据题目定制
// 例: alert(bool 报警发现目标信息, 该信息可延 fail 传递
// 其核心是根据目标串建立 tree 直通表, 然后再构建 fail 表

// 调用顺序: insert -> setFail -> buildTree/查询
// 复杂度: O(S+T+L)
// S=查询串长, T=目标串总长, L=tree 节点数

```

```

// 模板例题: https://www.luogu.com.cn/problem/P5357

const int MAXN = 2e5;
const int MAXS = 2e6;

int ed[MAXN + 1]; // 每个目标字符的结尾位置
int fail[MAXS + 1]; // fail 指针
int times[MAXS + 1]; // 词频
int tree[MAXS + 1][26]; // tree 直通表
int head[MAXS + 1]; // 建图
int nxt[MAXS + 1];
int to[MAXS + 1];
int idx = 0;
int cnt = 0;

void addEdge(int u, int v) { // 建图
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void insert(int i, string s) { // 前缀树 tree 插入字符串
    int cur = 0;
    for (char ch : s) {
        int path = ch - 'a';
        if (tree[cur][path] == 0) {
            tree[cur][path] = ++cnt;
        }
        cur = tree[cur][path];
    }
    ed[i] = cur;
}

void setFail() { // 设置 fail 指针和 tree 直通表
    queue<int> q;
    fail[0] = 0;
    for (int path = 0; path <= 25; path++) {
        if (tree[0][path] > 0) {
            fail[tree[0][path]] = 0;
            q.push(tree[0][path]);
        }
    }
    while (q.size()) {
        int cur = q.front();
        q.pop();
        for (int path = 0; path <= 25; path++) {
            if (tree[cur][path] == 0) {
                tree[cur][path] = tree[fail[cur]][path];
            }
        }
    }
}

```

```

        }
        else {
            fail[tree[cur][path]] = tree[fail[cur]][path];
            q.push(tree[cur][path]);
        }
    }
}

void buildTree(int n) { // 建立 fail 反向图
    for (int i = 1; i <= n; i++) {
        addEdge(fail[i], i);
    }
}

void Get(string s) { // 读入大文章, 获取初始单点词频
    int cur = 0;
    for (char ch : s) {
        int path = ch - 'a';
        cur = tree[cur][path];
        times[cur]++;
    }
}

void f(int cur) { // 递归统计真实词频
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        f(to[edge]);
        times[cur] += times[to[edge]];
    }
}

```

KMP 算法

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
class KMP {
private:
    /*
        整体时间复杂度  $O(n + m)$ 
        kmp :  $O(n)$ 
        nextArr :  $O(m)$ 

        kmp 返回字符串 t 作为 s 的子串的开头位置
        如若是返回第一个符合要求的子串的开头位置, 修改如下
        while (i < n && j < m) {
            .....
            (不需要 if: j == m 的特判)
        }
    */

```

```

        return j == m ? i - j : -1;
    }

```

`nxt` 数组中 `nxt[0]` 作为哨兵位置, `nxt[m]` 需要用来表示字符 `t` 整体
`nxt[i]` 表示(不包含当前位置 `i`)最长前后缀相同字符长度

例如 `aaavfvdfaaad`, `nxt[12] = 3`, `"aaa" = "aaa"`

生成目标字符 `t` 的 `nxt` 数组

*/

```

static const int MAXN = 1e6;
int nxt[MAXN + 2];

```

```

vector<int> kmp(string s, string t) {
    vector<int> res;
    int x = 0, y = 0; // x 是 s 的指针, y 是 t 的指针
    int n = s.size(), m = t.size();
    while (x < n) {
        if (s[x] == t[y]) {
            x++, y++; // 配对成功
        }
        else if (y == 0) {
            x++; // y 指向 t 开头
        }
        else { // j 跳跃
            y = nxt[y];
        }
        if (y == m) { // 满足
            res.push_back(x - y);
            y = nxt[y];
        }
    }
    return res;
}

```

```

void nextArr(string s) {
    int n = s.size();
    nxt[0] = -1;
    nxt[1] = 0;
    if (n == 1) {
        return;
    }
    int i = 2;
    int cn = 0; // 左侧字符匹配位置
    while (i <= n) {
        // 配对时不包含当前位置
        if (s[i - 1] == s[cn]) {

```

```

        nxt[i++] = ++cn;
    }
    else if (cn > 0) {
        cn = nxt[cn]; // 跳跃
    }
    else { // cn == 0
        nxt[i++] = 0;
    }
    }
}
};

```

Manacher 最长回文子串

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//manacher 求最长回文子串
//p[i]:表示以 i 为中心扩展的回文子串的半径
//r:所有已确定点的回文串向右延伸的截止位置的最大位置
//c:回文串截止位置=r 的最早中心点
//情况 1: 当前位置 i 没有被 r 包裹,以 i 为中心扩展
//情况 2: 当前 i 被 r 包住,对称点 i2 在大回文区域内,p[i]=p[i2]
//情况 3: 当前 i 被 r 包住,对称点 i2 在大回文区域外,p[i]=r-i
//情况 4: 当前 i 被 r 包住,对称点 i2 的回文半径撞到边界,从 r 的位置扩展
class Man {
private:
    static const int MAXN = 2e5;
    static int p[2 * MAXN + 2];
public:
    int manacher(string t) {
        string s = "#";
        for (char ch : t) {
            s += ch; //不能写为 s+=ch+'#'
            s += '#'; //这样写会把 ch 和#的 ASCII 码相加,得到一个字符
        }
        int maxlen = 0;
        //实际回文长度=回文半径-1
        for (int i = 0, r = 0, c = 0, len; i < s.size(); i++) {
            len = r > i ? min(p[2 * c - i], r - i) : 1; //若被 r 包住,获取答案,否则至少 len=1
            while (i + len < s.size() && i - len >= 0 && s[i + len] == s[i - len]) {
                //情况 2,3 不可能执行该 while 循环
            }
        }
    }
};

```

```

        //情况 1,4 可能执行该 while 循环
        len++;
    }
    if (i + len > r) {
        //判断最长回文半径
        r = i + len;
        c = i;
    }
    maxlen = max(maxlen, len);
    p[i] = len;
    int st = (i - len + 1) / 2; //该子串实际映射的开头位置
    int ed = (i + len - 3) / 2; //该子串实际映射的结尾位置
}
return maxlen - 1;
}
};

```

Z 算法(扩展 KMP)

```

#include<bits/stdc++.h>
using namespace std;
//Z 算法
//z[i]表示 s[i...n]和 s[0...n]的最长匹配前缀长度
//r 表示已知的最远配对距离
//c 表示达到最远配对距离的最早位置
//情况 1: i 不被 r 包住,直接从 i 扩展
//情况 2: i 被 r 包住,z[i-c]在扩展区域内,z[i]=z[i-c]
//情况 3: i 被 r 包住,z[i-c]在扩展区域外,z[i]=r-i
//情况 4: i 被 r 包住,z[i-c]在扩展区域边界,从 r 开始扩展

//e[i]表示 a[i...n]和 b[0...n]的最长匹配前缀长度
//e 的所有情况类比 z 的四种情况
class Z {
private:
    static const int MAXN = 2e5;
    static int z[MAXN + 2];
    static int e[MAXN + 2];
public:
    void zArray(string s) {
        z[0] = s.size();
        for (int i = 1, c = 1, r = 1, len; i < s.size(); i++) {
            len = r > i ? min(r - i, z[i - c]) : 0; //获取情况 1,3 否者赋值 0
            while (i + len < s.size() && s[i + len] == s[len]) {
                //while 循环只有情况 1,4 会正常执行
            }

```

```

        len++;
    }
    if (i + len > r) {
        r = i + len;
        c = i;
    }
    z[i] = len;
}
}
void eArray(string a, string b) {
    int n = a.size();
    int m = b.size();
    zArray(b); //生成 a 对于 b 的 e 数组, 依赖于 b 的 z 数组
    for (int i = 0, c = 0, r = 0, len; i < n; i++) {
        len = r > i ? min(r - i, z[i - c]) : 0;
        while (i + len < n && len < m && a[i + len] == b[len]) {
            len++;
        }
        if (i + len > r) {
            r = i + len;
            c = i;
        }
        e[i] = len;
    }
}
};

```

字符串哈希

```

#include<bits/stdc++.h>
using namespace std;
class HashString {
    //字符串哈希
    //入参: 字符串
    //返回结果: 特定数值
    //字符串哈希需要选定 base 进制
    //如: 433,499,599,1e9+7,31,131,1313,13131,131313
    //字符的赋值 ch->正整数(不要取!0!),可定制
private:
    using ll = long long;
    using ull = unsigned long long;
    static const ull base = 599;
    //数字+大写+小写
    static const int MAXN = 2e6;
    static ull pow[MAXN + 1];

```

```

static ull hash[MAXN + 1];
ull value(const char& ch) {
    if (ch >= 'a' && ch <= 'z') {
        return ch - 'a' + 37;
    }
    else if (ch >= 'A' && ch <= 'Z') {
        return ch - 'A' + 11;
    }
    else if (ch >= '0' && ch <= '9') {
        return ch - '0' + 1;
    }
    return 0;//非法返回结果
}
public:
    ull HashAll(const string& s) {
        ull ans = value(s[0]);
        for (ull i = 1; i < s.size(); i++) {
            ans = ans * base + value(s[i]);
        }
        return ans;
    }

    void build(const string& s, int n) {
        pow[0] = 1;
        for (int i = 1; i < n; i++) {
            pow[i] = pow[i - 1] * base;
        }
        hash[0] = value(s[0]);
        for (int i = 1; i < n; i++) {
            hash[i] = hash[i - 1] * base + value(s[i]);
        }
    }
    ull HashRange(int l, int r) {
        //计算区间[l,r]的哈希值
        //调用 HashRange 前先手动调用 build 函数
        if (l == 0) return hash[r];
        return hash[r] - hash[l - 1] * pow[r - l + 1];
    }
};

```

四、数学

01 分数规划

```
#include<bits/stdc++.h>
using namespace std;

// 01 分数规划, 性价比问题
// 给定数组 a, b
// 选择部分下标, 使得 sum(a) / sum(b) 最大化
// 公式转化:
//  $\text{sum}(a) / \text{sum}(b) = x \rightarrow \text{sum}(a) - x * \text{sum}(b) = 0 \rightarrow \text{sum}(a[i] - b[i] * x) = 0$ 
// 使用二分答案求求解

// https://www.luogu.com.cn/problem/P10505
const int MAXN = 1e3;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;
struct stc {
    ld a, b;
    ld val;
}a[MAXN + 1];
bool cmp(stc s1, stc s2) {
    return s1.val > s2.val;
}
bool check(ld x, int n, int k) {
    for (int i = 1; i <= n; i++) {
        a[i].val = a[i].a - a[i].b * x;
    }
    sort(a + 1, a + n + 1, cmp);
    ld sum = 0;
    for (int i = 1; i <= k; i++) {
        sum += a[i].val;
    }
    return sum >= eps;
}

bool solve() {
    int n, k;
    cin >> n >> k;
    cout << fixed << setprecision(16);
    if (n == 0 && k == 0) {
        return false;
    }
}
```

```

    ld l = 0, r = 0;
    k = n - k;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].a;
        r += a[i].a;
    }
    for (int i = 1; i <= n; i++) {
        cin >> a[i].b;
    }
    while (l < r && r - l > eps) {
        ld mid = (l + r) / 2.0;
        if (check(mid, n, k)) {
            l = mid + eps;
        }
        else {
            r = mid - eps;
        }
    }
    cout << (ll)round(l * 100.0) << '\n';
    return true;
}

```

gcd 和 lcm

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//最大公约数
//辗转相除法
//gcd(a,b)=gcd(b,a%b)
ll gcd(ll a, ll b) {
    while (b) {
        ll tmp = a;
        a = b;
        b = tmp % b;
    }
    return a;
}

//最小公倍数
ll lcm(ll a, ll b) {
    if (a == 0 || b == 0) {
        return 0;
    }
}

```

```

    }
    return a / gcd(a, b) * b;
}

```

MR 质数测试

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//快速幂
ll pw(ll a, ll b, ll mod) {//...
}
//Miller_Rabin 算法测质数
//辅助验证函数
bool witness(ll a, ll n) {
    ll u = n - 1;
    int t = 0;
    while ((u & 1) == 0) {
        t++;
        u >>= 1;
    }
    ll x1 = pw(a, u, n), x2;
    for (int i = 1; i <= t; i++) {
        x2 = pw(x1, 2, n);
        if (x2 == 1 && x1 != 1 && x1 != n - 1) {
            return true;
        }
        x1 = x2;
    }
    if (x1 != 1) {
        return true;
    }
    return false;
}
//MR_p 测试数组
ll MR_p[] = { 2,3,5,7,11,13,17,19,23,29,31,37 };
bool Miller_Rabin(ll n) {
    //模板例题: https://www.luogu.com.cn/problem/U148828
    //判断 x 是否为质数
    if (n <= 2) {
        return n == 2;
    }
    if ((n & 1) == 0) {
        return false;
    }
}

```

```

    for (ll p : MR_p) {
        if (p >= n) break;
        if (witness(p, n)) {
            return false;
        }
    }
    return true;
}

```

Raney 引理

```

#include<bits/stdc++.h>
using namespace std;
//Raney 引理:若整数序列(下标 1~m)每一项不超过 1( $a[i] \leq 1$ ),总和为正整数 n,
//则它的 m 个循环移位中恰有 n 个移位的所有非空前缀和为正
void Raney() {
    cout << "Raney:" << '\n';
    const int m = 22;
    int arr[m + 1] = { 0, 1, 1, 1, -2, 1, -1, 1, 1, -3, 1, 1,
        -2, 1, 1, 1, 1, 1, 1, -1, -2, 1, 1 };
    int n = 0;
    for (int i = 1; i <= m; i++) {
        n += arr[i];
    }
    cout << n << ' ' << m << '\n';
    int cnt = 0;
    for (int i = 1; i <= m; i++) {
        int sum = 0;
        int flg = 1;
        for (int j = i; j <= i + m - 1; j++) {
            sum += arr[(j - 1) % m + 1];
            if (sum <= 0) flg = 0;
        }
        if (flg) cnt++;
    }
    cout << "n: " << n << '\n';
    cout << "cnt: " << cnt << '\n';
}

```

中国剩余定理

```

#include<bits/stdc++.h>
using namespace std;

```

```

using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
// 中国剩余定理
// 一般形式: 要求 m1, m2, ...mn 互质
// x% m1 = r1, x% m2 = r2, ..., x% mn = rn
// lcm = m1 * m2 * m3*...*mn(互质)
// ai = lcm / mi
// ci = (ri * ai * inv(ai, mi)) % lcm
// 每一个 ci 满足 ci% mi = ri, ci% mj = 0(j != i)
// x = c1 + c2 + c3 + ... + cn
// 最小整数解 minx = x % lcm
// 通解 X = k * lcm + minx(k >= 0)

const int MAXN = 2e5;
ll r[MAXN + 1];
ll m[MAXN + 1];
ll c[MAXN + 1];
ll a[MAXN + 1];

ll add(ll a, ll b, ll mod) {
    return b == 0 ? a : add(a ^ b, ((a & b) << 1) % mod, mod);
}

ll multiply(ll a, ll b, ll ans, ll mod) {
    return b == 0 ? ans : multiply((a << 1) % mod, (ull)b >> 1, (b & 1) ?
add(ans, a, mod) : ans, mod);
}

ll mul(ll a, ll b, ll mod) { return multiply(a, b, 0, mod); }

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

ll inv(ll a, ll mod) {

```

```

    ll x, y;
    exgcd(a, mod, x, y);
    x = (x % mod + mod) % mod;
    return x;
}

// 模板: https://www.luogu.com.cn/problem/P1495
// 注意:
// 使用__int128_t 的数据类型, 避免中间数据爆 ll 类型
// 或自行使用位运算实现乘法, 每一步进行取模运算
// 两者都可 AC, C++建议使用__int128_t 类型, 更高效和安全

void solve() {
    int n;
    cin >> n;
    ll lcm = 1;
    for (int i = 1; i <= n; i++) {
        cin >> m[i] >> r[i];
        lcm *= m[i];
    }
    ll x = 0;

    for (int i = 1; i <= n; i++) {
        a[i] = lcm / m[i];
        // c[i] = mul(r[i], mul(a[i], inv(a[i], m[i]), lcm), lcm);
        c[i] = (ill)r[i] * (ill)a[i] % lcm * (ill)inv(a[i], m[i]) % lcm;
        x = (x + c[i]) % lcm;
    }
    x %= lcm;
    cout << x << '\n';
}

```

中国剩余定理扩展

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ill = __int128_t;

// 中国剩余定理
// 扩展:  $m_1, m_2, m_3, \dots, m_n$  之间不需要严格互质
//  $x \% m_1 = r_1, x \% m_2 = r_2, \dots, x \% m_n = r_n$ 
// 补充初始模数  $m_0 = 1, lcm = 1, tail = 0$ , 得到  $ans1 = x * lcm + tail$ 
// 对于  $m_i, r_i$ , 得到  $ans2 = y * m_i + r_i$ 
// 显然  $ans1 = ans2$ , 相消得到  $lcm * x + m_i * y = r_i - tail$ , 即  $ax + by = c$ 

```

```

// exgcd 求解，若不存在解说明无解

// 通解  $x = x_0 + (b / d) * n$ ，带入  $ans = lcm * x + tail$ ,  $ans = lcm * (b / d) * n + (lcm * x_0 + tail)$ 
// 即进行如下运算：
// 得到  $lcm1 = lcm * (b / d)$ ,  $tail1 = (lcm * x_0 + tail) \% lcm1$ 
// 得到  $ans = lcm1 * x + tail1$ 
// 持续迭代直到结束，得到最后的  $tail$  就是答案(最小整数解)

const int MAXN = 2e5;
ll r[MAXN + 1];
ll m[MAXN + 1];

ll add(ll a, ll b, ll mod) {
    return b == 0 ? a : add(a ^ b, ((a & b) << 1) % mod, mod);
}
ll multiply(ll a, ll b, ll ans, ll mod) {
    return b == 0 ? ans : multiply((a << 1) % mod, (ull)b >> 1, (b & 1) ?
add(ans, a, mod) : ans, mod);
}
ll mul(ll a, ll b, ll mod) { return multiply(a, b, 0, mod); }

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

// 模板: https://www.luogu.com.cn/problem/P4777
// C++建议使用__int128_t 类型，比位运算实现更为安全和高效
ll excrt(int n) {
    ll tail = 0, lcm = 1, tmp, a, b, c, x0;
    for (int i = 1; i <= n; i++) {
        // 此处 a, b, c 可能因题意而定制
        a = lcm;
    }
}

```

```

    b = m[i];
    c = ((r[i] - tail) % b + b) % b;

    // 以下代码是通用部分
    ll x, y;
    ll d = exgcd(a, b, x, y);
    if (c % d != 0) {
        return -1; // 无解
    }

    // x0 保证非负特解
    x0 = ((ll)x * (ll)(c / d) % (b / d) + ll(b / d)) % (b / d);
    tmp = lcm * (b / d); // 此时的 tmp 就是新的 lcm1
    tail = ((ll)tail + ll(x0) * ll(lcm) % tmp) % tmp;
    lcm = tmp;
}
// tail 是返回最小非负解
return tail;
}

```

乘法快速幂

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
#define ill __int128_t
//快速幂
ll pw(ll a, ll b, ll mod) {
    ll res = 1;
    while (b) {
        if (b & 1) {
            res = ill(res) * ill(a) % mod;
        }
        a = ill(a) * ill(a) % mod;
        b >>= 1;
    }
    return res;
}

```

二元一次不定方程(丢番图方程)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
// 给定 a, b 不全为 0
// d = gcd(a, b)

```

```

// ax + by = d
// 存在特解(xx, yy)
// 存在无穷解: x = xx + (b / d) * n
//             y = yy - (a / d) * n
// n 为任意整数

// ax + by = c(c = d * k, k 为任意整数)
// 存在特解(xx, yy)
// x1 = xx * (c / d), y1 = yy * (c / d)
// 存在无穷解: x = x1 + (b / d) * n
//             y = y1 - (a / d) * n
// n 为任意整数

ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

ll a, b, c, d, x, y;

// 若存在正整数解
// x 取 >= 1 的最小时, y 取最大(>= 1 的最大)
// y 取 >= 1 的最小时, x 取最大(>= 1 的最大)
// 若不存在
// x 取 >= 1 最小时, y < 0
// y 取 >= 1 最小时, x < 0

//模板: https://www.luogu.com.cn/problem/P5656
void solve() {
    cin >> a >> b >> c;
    d = gcd(a, b);
    if (c % d != 0) {
        cout << -1 << '\n';
        return;
    }
}

```

```

exgcd(a, b, x, y);
// 解
x = x * (c / d);
y = y * (c / d);
// 增幅
ll xd = b / d;
ll yd = a / d;
if (x <= 0) {
    ll n = (1 - x + xd - 1) / xd;
    x += n * xd;
    y -= n * yd;
}
else {
    ll n = (x - 1) / xd;
    x -= n * xd;
    y += n * yd;
}
if (y <= 0) {
    cout << x << ' ';
    ll n = (1 - y + yd - 1) / yd;
    y += n * yd;
    cout << y << '\n';
}
else {
    cout << ((y - 1) / yd + 1) << ' ';
    cout << x << ' ';
    cout << y - ((y - 1) / yd) * yd << ' ';
    cout << x + ((y - 1) / yd) * xd << ' ';
    cout << y << ' ';
    cout << '\n';
}
}
}

```

二项式反演

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 二项式反演:
// 以下 f 和 g 函数都是对 i 范围上的求和
// 0 <= i <= n
//  $g(n) = (-1)^i * C(n, i) * f(i)$  等价于  $f(n) = (-1)^i * C(n, i) * g(i)$ 
//  $g(n) = C(n, i) * f(i)$  等价于  $f(n) = (-1)^{(n-i)} * C(n, i) * g(i)$ 

// n <= i <= N

```

```

//  $g(n) = (-1)^i * C(i, n) * f(i)$  等价于  $f(n) = (-1)^i * C(i, n) * g(i)$ 
//  $g(n) = C(i, n) * f(i)$  等价于  $f(n) = (-1)^{(i - n)} * C(i, n) * g(i)$ 

//  $g(i)$ : 钦定必须选择某  $i$  个元素, 且形成的交集元素至少包含这  $i$  个
//  $2^{(n - i)}$ : 选择了必选的  $i$  个元素的所有集合( $i$  个必选, 决定集合不同的项是其他  $n - i$  个,  $n - i$  个任选)
//  $2^{(2^{(n - i)})} - 1$ : 所有选择了必选元素的所有集合, 选或不选的方案数( $-1$  即不能为空)
//  $g(i) = C(n, i) * (2^{(2^{(n - i)})} - 1)$ 

//  $f(i)$ : 挑选集合后, 集合形成的交集刚好是  $i$  个元素
// 特征式:  $g(k) = f(i) * C(i, k)$  求和( $k \leq i \leq n$ )
// 表示: 从  $f(i)$  刚好的  $i$  个元素中, 选择出  $k$  个作为  $g(k)$  钦定的至少  $k$  个元素
// 命中了第四个反演式
//  $f(k) = (-1)^{(i - k)} * C(i, k) * g(i)$  求和( $k \leq i \leq n$ )

// https://www.luogu.com.cn/problem/P10596
// 核心: 该题运用了钦定至少  $k$  个  $\rightarrow$  恰好  $k$  个的转化
// 此类转化往往是二项式反演的核心

const int mod = 1e9 + 7;
const int MAXN = 1e6;
ll fac[MAXN + 1]; // fac[i] 即  $(i!)$  对 mod 取模的结果
ll inv[MAXN + 1]; // inv[i] 即  $(i!)$  模 mod 意义下的逆元
ll g[MAXN + 1];

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {
    return pw(a, mod - 2);
}

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }
}

```

```

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }
}

ll C(ll n, ll m) {
    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

void prepare(int n) {
    init(n);
    // g(i) = C(n, i) * (2 ^ (2 ^ (n - i)) - 1)
    ll tmp = 2;
    for (int i = n; i >= 0; i--) {
        g[i] = tmp;
        tmp = tmp * tmp % mod;
    }
    for (int i = 0; i <= n; i++) {
        g[i] = (g[i] + mod - 1) % mod * C(n, i) % mod;
    }
}

ll f(int k, int n) {
    ll ans = 0;
    // f(k) = (-1)^(i - k) * C(i, k) * g(i) 求和(k <= i <= n)
    for (int i = k; i <= n; i++) {
        if ((i - k) & 1) {
            ans = (ans + C(i, k) * (mod - 1) % mod * g[i] % mod) % mod;
        }
        else {
            ans = (ans + C(i, k) * g[i] % mod) % mod;
        }
    }
    return ans;
}

void solve() {
    int n, k;
    cin >> n >> k;
    prepare(n);
    cout << f(k, n);
}

```

二项式定理

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 组合恒等式:  $C(n, m) = C(n - 1, m) + C(n - 1, m - 1)$ 
//  $C(n, m) = n! / (m! * (n - m)!)$ 
//  $0! = 1$ , 0 的 0 次幂 = 1

// 二项式系数的组合分解公式:
//  $C(j, i) * C(i, n) = C(j, n) * C(j - n, j - i)$ 

const int mod = 1e9 + 7;
const int MAXN = 2e5;
ll fac[MAXN + 1]; // fac[i] 即 (i!) 对 mod 取模的结果
ll inv[MAXN + 1]; // inv[i] 即 (i!) 模 mod 意义下的逆元

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {
    return pw(a, mod - 2);
}

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }
}

ll C(ll n, ll m) {
```

```

    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

```

位运算相关

```

#include<bits/stdc++.h>
using namespace std;

//2 的幂快速判断
void isPwTwo(int n) {
    //若  $x > 0, x \& (\sim x + 1) == x$  则  $x$  是 2 的幂
    //  $\sim x + 1 = -x$ 
    if (n == 0) {
        cout << 'F' << '\n';
        return;
    }
    if ((n & (-n)) == n) {
        cout << 'T' << '\n';
    }
    else {
        cout << 'F' << '\n';
    }
}

int GetLowBitOne(int a){}
//找到最小的非负整数 k,使得 a+k 的最低位 1  $\geq$  minV
int calCost(int a, int minV) {
    int v = GetLowBitOne(a);
    if (v >= minV) return 0;
    int pow2 = 1 << minV;
    int m = (a + pow2 - 1) >> minV;
    int b = m << minV;
    return b - a;
}

// 用给定位状态 status 的 1 位组成的集合(即 status 位状态的所有子集)
// 循环遍历,该方法的遍历保证严格下降不重复
void s(){
    for (int i = status; i > 0; i = (i - 1) & status) {
        cout << i << ' ';
    }
}

// 加法运算的位本质
// 对于任意整数 a, b
// 都满足  $a + b = a \wedge b + 2(a \& b)$ 

```

```
// 快速提取 x 最右侧的 1(x!=0)
// 1l lowBitOne=x&(-x)
```

卡特兰数(因子计数法)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 卡特兰数:
//  $f(n) = C(2n, n) / (n + 1)$ 
// 公式 2,  $n \geq 0$ 

// 当数据量达到  $1e6$  规模, 模数 mod 未必是质数
// 需要用到公式 2 + 因子计数法求解
// 维护最小质因子表, 质数不需要维护(默认标记 0, 否则会出错!)

// 时间复杂度: $O(n)$ 
// https://www.luogu.com.cn/problem/P3200
// 核心代码如下:

ll f2(int n, int mod) {
    // 2 ~ n : -1
    fill(cnt + 2, cnt + n + 1, -1);
    // n + 2 ~ 2n : 1
    fill(cnt + n + 2, cnt + 2 * n + 1, 1);
    for (int i = 2 * n; i >= 2; i--) {
        if (minFactor[i] != 0) {
            cnt[minFactor[i]] += cnt[i];
            cnt[i / minFactor[i]] += cnt[i];
            cnt[i] = 0;
        }
    }
    ll ans = 1;
    for (int i = 2; i <= 2 * n; i++) {
        if (cnt[i] != 0) {
            ans = ans * pw(i, cnt[i], mod) % mod;
        }
    }
    return ans;
}

const int MAXN = 1e6;
// !!! 开二倍空间!!!
```

```

ll minFactor[2 * MAXN + 1]; // minFactor[i]表示 i 的最小质因子(线性欧拉筛维护最小质因子表)
bitset<2 * MAXN + 1>isPrime; // 标记质数
ll Primes[2 * MAXN + 1];
int cnt[2 * MAXN + 1];
void getPrimes_Linear(ll n) {
    isPrime.set();
    isPrime[0] = isPrime[1] = 0;
    Primes[0] = 0;
    for (ll i = 2; i <= n; i++) {
        // 注意：卡特兰数和因子计数法，质数不需要获取
        if (isPrime[i]) Primes[++Primes[0]] = i;
        for (ll j = 1, v; j <= Primes[0]; j++) {
            v = Primes[j];
            if (v * i > n) break;
            isPrime[i * v] = 0;
            minFactor[i * v] = v;
            if (i % v == 0) break;
        }
    }
}

ll pw(ll a, ll b, int mod) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

```

卡特兰数

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 卡特兰数:
//  $f(n) = C(2n, n) - C(2n, n - 1)$ 
//  $f(n) = C(2n, n) / (n + 1)$ 
//  $f(n) = f(n - 1) * ((4n - 2) / (n + 1))$ 
//  $f(n) = [\text{求和 } 0 \leq i \leq n - 1: f(i) * f(n - 1 - i)]$ 
// 其中，公式 1, 3, 4 要求  $n \geq 1$ 
// 公式 2,  $n \geq 0$ 

```

```

// 卡特兰数应用核心：
// 1、进出栈、括号匹配问题
// 2、非交叉配对问题
// 3、二叉树形态计数问题
// 4、矩阵走线问题
// 5、凸多边形三角剖分
// 6、前缀不越界问题(即不违反特定条件)
// 以上问题的核心特征：可拆分性，可递归性，不交叉性

// 核心代码如下：
ll f1(int n) {
    f[n] = (C(2 * n, n) - C(2 * n, n - 1) + mod) % mod;
    return f[n];
}

ll f2(int n) {
    f[n] = C(2 * n, n) * invVal[n + 1] % mod;
    return f[n];
}

ll f3(int n) {
    f[0] = f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = f[i - 1] * (4 * i - 2) % mod * invVal[i + 1] % mod;
    }
    return f[n];
}

ll f4(int n) {
    f[0] = f[1] = 1;
    for (int i = 2; i <= n; i++) {
        f[i] = 0;
        for (int j = 0; j <= i - 1; j++) {
            f[i] = (f[i] + f[j] * f[i - 1 - j] % mod) % mod;
        }
    }
    return f[n];
}

const int mod = 1e9 + 7;
const int MAXN = 2e5;
ll invVal[MAXN + 10]; // 连续常数的逆元
ll fac[MAXN + 10]; // fac[i]即(i!)对mod取模的结果
ll inv[MAXN + 10]; // inv[i]即(i!)模mod意义下的逆元

```

```

ll f[MAXN + 10];

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

ll inv_(ll a) {
    return pw(a, mod - 2);
}

void init(int n) {
    fac[0] = 1;
    for (ll i = 1; i <= n; i++) {
        fac[i] = fac[i - 1] * i % mod;
    }

    inv[0] = 1;
    inv[n] = inv_(fac[n]);
    for (ll i = n - 1; i >= 1; i--) {
        inv[i] = (i + 1) * inv[i + 1] % mod;
    }

    invVal[1] = 1;
    for (ll i = 2; i <= n; i++) {
        invVal[i] = mod - invVal[mod % i] * (mod / i) % mod;
    }
}

ll C(ll n, ll m) {
    return (((fac[n] * inv[m]) % mod) * inv[n - m]) % mod;
}

```

向量空间线性基

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-7;
array<int, 2> cost[501];

```

```

ld a[501][501];
int basis[501];

bool insert(int i, int m) {
    for (int j = 1; j <= m; j++) {
        if (fabsl(a[i][j]) >= eps) {
            if (basis[j] == 0) {
                basis[j] = i;
                return true;
            }
            ld rate = a[i][j] / a[basis[j]][j];
            for (int k = j; k <= m; k++) {
                a[i][k] -= rate * a[basis[j]][k];
            }
        }
    }
    return false;
}

bool cmp(array<int, 2>a, array<int, 2>b) {
    return a[0] < b[0];
}

void solve() {
    cout << fixed << setprecision(12);
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            cin >> a[i][j];
        }
    }
    for (int i = 1; i <= n; i++) {
        cin >> cost[i][0];
        cost[i][1] = i;
    }
    sort(cost + 1, cost + n + 1, cmp);
    int cnt = 0;
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (insert(cost[i][1], m)) {
            cnt++;
            ans += cost[i][0];
        }
    }
    cout << cnt << ' ' << ans << '\n';
}

```

哥德巴赫猜想

```
// 强哥德巴赫猜想:任意大于 2 的偶数都可以表示成两个质数之和,该猜想对于[1,1e9]内的  
整数已证明满足
```

```
// 弱哥德巴赫猜想(此猜想已被彻底证明成立):任一大于 5 的奇数都可以表示成三个质数之  
和
```

```
// 其中若  $n \geq 1$  且  $n-2$  是质数则最少可以拆为 2 个
```

四则运算的位运算实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
// 无算数运算实现加减乘除
// 纯粹的位运算

// 溢出问题需要自行取模运算解决

// 加法: 异或运算 + 与运算
// 异或运算: 无进位相加
// 与运算: 维护进位信息
ll add(ll a, ll b) { // 无进位相加 进位信息
    return b == 0 ? a : add(a ^ b, (a & b) << 1);
}

// 取相反数
//  $-b = \sim b + 1$ 
ll neg(ll b) {
    return add(~b, 1LL);
}

// 减法
//  $a - b = a + (-b) = a + (\sim b + 1)$ 
ll minus(ll a, ll b) {
    return add(a, neg(b));
}

// 乘法
// b 一定要转为无符号右移
ll multiply(ll a, ll b, ll ans) {
    return b == 0 ? ans : multiply(a << 1, (ull)b >> 1, (b & 1) ? add(ans, a) : ans);
}
ll mul(ll a, ll b) { return multiply(a, b, 0); }
```

```

// int:0~30
// long long:0~62
// 31 和 63 是符号位
// 注意:a, b 不能是 LLONG_MIN(数据类型最小值)
ll divide(ll a, ll b) {
    ll x = a < 0 ? neg(a) : a;
    ll y = b < 0 ? neg(b) : b;
    ll ans = 0; // 此处::是全局命名空间,避免和 std 冲突
    for (int i = 62; i >= 0; i = ::minus(i, 1)) {
        if ((x >> i) >= y) {
            ans |= (1LL << i);
            x = ::minus(x, y << i);
        }
    }
    return (a < 0) ^ (b < 0) ? neg(ans) : ans;
}

// 出现数据类型最小值的除法
ll div2(ll a, ll b) {
    if (a == LLONG_MIN && b == LLONG_MIN) {
        return 1;
    }
    if (a != LLONG_MIN && b != LLONG_MIN) {
        return divide(a, b);
    }
    if (b == LLONG_MIN) {
        return 0;
    }
    // a 是最小, b 不是最小
    a = add(a, b > 0 ? b : neg(b));
    ll ans = divide(a, b);
    ll offset = b > 0 ? neg(1) : 1;
    return add(ans, offset);
}

```

埃氏筛质数

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e7;
bitset<MAXN + 1>isPrime;//标记质数
ll Primes[MAXN + 1];
ll minFactor[MAXN + 1];
void getPrimes_Eratosthenes(ll n) {

```

```

//埃氏筛
//复杂度  $O(n \cdot \log_2(\log_2 n))$ 
//获取 1~n 的所有质数
Primes[0] = 0;
isPrime.set();
isPrime[0] = isPrime[1] = 0;
for (ll i = 2; i <= n; i++) {
    if (isPrime[i]) {
        minFactor[i] = i;
        Primes[++Primes[0]] = i;
        for (ll j = i * i; j <= n; j += i) {
            isPrime[j] = 0;
            if (minFactor[j] == 0) {
                minFactor[j] = i;
            }
        }
    }
}
}
}

```

完美洗牌问题

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 完美洗牌问题:
// 给定区间[1...r], 长度偶数
// mid = (1 + r) >> 1, 记[1, mid]为数组{ l1,l2,l3,... }
// 记[mid+1, r]为数组{ r1,r2,r3,... }
// 把[1, r]交换为{ r1,l1,r2,l2,r3,l3,... }的形式

// 核心:
// 若 i <= mid, go(i) = i + (i - 1 + r)
// 若 i > mid, go(i) = i - (r - i + 1)
// 存在若干个下标的循环交换
// 当长度  $k = 3^k - 1$  时
// 所有循环起点是{  $3^0, 3^1, 3^2, \dots, 3^{(k-1)}$  }
// 通过三进制拆分的方法, 解决该问题

// 使用辅助数组 help, 可以在时间复杂度  $O(n)$ , 空间  $O(n)$  解决
// 使用三进制拆分, 可以在时间  $O(n)$ , 空间  $O(1)$  解决

const int MAXN = 2e5;
int a[MAXN + 1];

```

```

int b[MAXN + 1];
int start[21];
ll split[21]; // 三进制拆分数组
// 3 的 20 次幂是 3486784401, 足够大了

int prepare(int n) {
    int siz = 0;
    for (ll s = 1, p = 2; p <= n; s *= 3, p = s * 3 - 1) {
        start[++siz] = s; // 起点是 3 的幂
        split[siz] = p; // 分割点是 3 的幂-1
    }
    return siz;
}

int go(int l, int r, int i) {
    int mid = (l + r) >> 1;
    if (i <= mid) return i + (i - l + 1);
    return i - (r - i + 1);
}

// 翻转区间[l, r]
void rev(int l, int r, int* arr) {
    while (l < r) {
        swap(arr[l], arr[r]);
        l++, r--;
    }
}

//[l, k], [k + 1, r]交换
void rotate(int l, int k, int r, int* arr) {
    rev(l, k, arr);
    rev(k + 1, r, arr);
    rev(l, r, arr);
}

void circle(int l, int r, int i, int* arr) {
    for (int j = i; j >= 1; j--) {
        int cur = l + start[j] - 1, next;
        next = go(l, r, cur);
        int curv = arr[cur];
        while (next != l + start[j] - 1) {
            int tmp = arr[next];
            arr[next] = curv;
            curv = tmp;
            cur = next, next = go(l, r, next);
        }
    }
}

```

```

        arr[next] = curv;
    }
}

void shuffle(int l, int r, int* arr) {
    int n = r - l + 1;
    int siz = prepare(n);
    for (int i = siz, mid; n > 0;) {
        if (split[i] <= n) {
            mid = (l + r) >> 1;
            // 处理长度为 split[i] 的段
            // 分别由左侧和右侧的部分拼接而成
            rotate(l + split[i] / 2, mid, mid + split[i] / 2, arr);
            circle(l, l + split[i] - 1, i, arr);
            l += split[i];
            n -= split[i];
        }
        else {
            i--;
        }
    }
}

void shuffleHelp(int l, int r, int* arr) {
    int n = r - l + 1;
    vector<int>help(n + 1, 0);
    int mid = (l + r) >> 1;
    for (int i = 1, j = mid + 1; j <= r; i += 2, j++) {
        help[i] = arr[j];
    }
    for (int i = 2, j = 1; j <= mid; i += 2, j++) {
        help[i] = arr[j];
    }
    for (int i = 1, j = 1; j <= r; j++, i++) {
        arr[j] = help[i];
    }
}

```

容斥原理

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
//容斥原理

//核心:
//+C(k,1)的和-C(k,2)的交集+C(k,3)的和-C(k,4)的交集+.....

```

```

//奇数个+,偶数个-
//例题:https://codeforces.com/problemset/problem/803/F
//例题:https://www.luogu.com.cn/problem/P1450

class Inclusion_Exclusion {//容斥原理
private:
    const static int mod = 1e9 + 7;
    int cnt[100001];
    ll dp[100001];
    //dp[i]表示以 i 为最小公约数的子序列个数
    ll pow2[100001];
    void initial() {
        pow2[0] = 1;
        for (ll i = 1; i <= 100000; i++) {
            cnt[i] = 0;
            pow2[i] = pow2[i - 1] * 2LL % mod;
        }
    }

    ll dp02[100001];

public:
    void solve01() {
        initial();
        int n;
        cin >> n;
        ll max_val = 1;
        for (int i = 1; i <= n; i++) {
            int x;
            cin >> x;
            cnt[x]++;
            max_val = max(max_val, (ll)x);
        }
        //数组中,包含 i 本身的 gcd=i 的子序列,需要满足所有其他元素都是 i 的倍数
        for (int i = max_val; i >= 1; i--) {
            ll count = 0;
            for (int j = i; j <= max_val; j += i) {
                count += cnt[j];
            }
            dp[i] = (pow2[count] - 1 + mod) % mod;
            for (int j = i + i; j <= max_val; j += i) {
                dp[i] = (dp[i] - dp[j] + mod) % mod;
            }
        }
    }
}

```

```

        cout << dp[1] << '\n';
    }

    void solve02() {
        //假定无个数限制,dp[s]完全背包表示 1~i 凑到 s 元的所有方案数
        //dp[s-(di+1)*ci]表示第 i 枚硬币花费超出 di 个的方案数
        //根据容斥原理求 1~4 都花费超出的方案数
        ll value[4];
        ll cnt[4];
        int n;
        cin >> value[0] >> value[1] >> value[2] >> value[3] >> n;
        dp[0] = 1;
        //完全背包问题
        for (int i = 0; i <= 3; i++) {
            for (int j = value[i]; j <= 100000; j++) {
                dp[j] += dp[j - value[i]];
            }
        }
        for (int i = 0; i < n; i++) {
            ll s;
            for (int i = 0; i < 4; i++) {
                cin >> cnt[i];
            }
            cin >> s;
            //0001->1111(1->15)
            //用位状态表示第 1~i 枚硬币的违规状态的方案数
            ll illegal = 0; //非法
            for (int i = 1; i <= ((1 << 4) - 1); i++) {
                ll tmp = s; //总值
                int flg = 0; //标记加/减
                for (int j = 0; j <= 3; j++) {
                    //查看各位的状态
                    if ((i >> j) & 1) {
                        tmp -= value[j] * (cnt[j] + 1);
                        flg ^= 1;
                    }
                }
                if (tmp >= 0) {
                    illegal += flg ? dp[tmp] : -dp[tmp];
                }
            }
            cout << dp[s] - illegal << '\n';
        }
    }
};

```

常用一般推论

```
//1、
//正整数 a,b
//若存在  $a \% c = b$ 
//则一定满足  $a = b$ , 或  $a > 2 * b$ 
//正整数 a 可以得到的所有余数  $[0 \dots (a-1)/2] \&\& [a]$ 
//向下取整

//2、
//最大公约数相关
//任意正整数  $a < b$ 
//都有  $\gcd(a, b) = \gcd(a, b - a)$ 

//3、
//对于任意正整数  $a < b$ 
//若  $a \% m = b \% m$ , 则  $(b - a) \% m = 0$ 

//4、
// 若  $(a + b) \% \text{mod} = 0$ 
// 给定 a, 则  $b \% \text{mod} = (\text{mod} - a \% \text{mod}) \% \text{mod}$ 

//5、
//  $(a + b) \% \text{mod} == j$ 
// 给定 a,  $b \% \text{mod} = ((j - a) \% \text{mod} + \text{mod}) \% \text{mod}$ 

//6、
//  $(a \% m * b \% m) \% m = (a * b) \% \text{mod}$ 
//  $-a \% \text{mod} = (-a \% \text{mod} + \text{mod}) \% \text{mod} = (\text{mod} - a) \% \text{mod}$ 
```

常见等差等比数列

```
等比数列:
i >= n
a[i] = i * i
b[i] = i * i - 1
前 n 项和:
Sa[i] = n(n+1)(2n+1)/6
Sb[i] = n(n-1)(2n+5)/6
```

康托展开

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```

// 康托展开:
// 给定数字 1~n, 给定 1~n 的排列 P, 求该排列 P 的名次(字典序升序)
// rank(P) = [范围求和 1 <= i <= n]: rightSmall(P[i]) * (n - i)!

// https://www.luogu.com.cn/problem/P5367

const int MAXN = 1e6;
const int mod = 998244353;
int tree[MAXN + 1];
int fac[MAXN + 1];
int p[MAXN + 1];

int lowbit(int x) {
    return x & -x;
}

void add(int i, int n, int v) {
    while (i <= n) {
        tree[i] += v;
        i += lowbit(i);
    }
}

int sum(int r) {
    int ans = 0;
    while (r > 0) {
        ans = 1ll * ((1ll)ans + (1ll)tree[r]) % mod;
        r -= lowbit(r);
    }
    return ans;
}

int rank(int n) {
    int ans = 0;
    for (int i = 1; i <= n; i++) {
        ans = 1ll * ((1ll)ans + (1ll)sum(p[i] - 1) * (1ll)fac[n - i] % mod) %
mod;
        add(p[i], n, -1);
    }
    return ans;
}

void prepare(int n) {
    fac[0] = 1;
    for (int i = 1; i <= n; i++) {

```

```

        fac[i] = ll((ll)fac[i - 1] * (ll)i) % mod;
    }
}

void solve() {
    int n;
    cin >> n;
    prepare(n);
    for (int i = 1; i <= n; i++) {
        add(i, n, 1);
        cin >> p[i];
    }
    cout << (::rank(n) + 1) % mod;
}

```

异或空间线性基(普通消元法)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 60;
ll basis[bit + 1];
ll a[MAXN + 1];
bool zero = false;

// 基：可以表示原空间所有元素的一组极小线性无关组(不唯一)
// 普通消元法或高斯消元法

// 普通消元法求解异或线性基
// 时间复杂度  $O(n \log V)$ ,  $V$  是最大值
// 普通消元法

// 若线性基大小 =  $m$ , 则原始异或和共有  $2^m - 1$  种
// 最大异或和：从高位开始贪心, 异或和可以变大则异或, 否则不异或
// 第  $k$  小异或和, 需要用高斯消元法

// 若插入失败, 意味着被其他线性基消除, 原始异或和存在 0
bool insert(ll x) {
    for (int i = bit; i >= 0; i--) {
        if ((x >> i) & 1) {
            if (basis[i] == 0) {
                basis[i] = x;
                return true;
            }
        }
        x ^= basis[i];
    }
}

```

```

    }
}
return false;
}

void buildBasis(ll* arr, int n) {
    zero = false;
    fill(basis, basis + bit + 1, 0);
    for (int i = 1; i <= n; i++) {
        if (!insert(arr[i])) {
            zero = true;
        }
    }
}

// 最大异或和
ll GetXorMax() {
    ll res = 0;
    for (int i = bit; i >= 0; i--) {
        res = max(res, res ^ basis[i]);
    }
    return res;
}

```

异或空间线性基(高斯消元法)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 60;

// !! 注意!!
// 由于高斯消元需要拷贝原始数组, 需要设置较大内存
ll basis[MAXN + 1];
ll a[MAXN + 1];
bool zero = false;
int len; // 线性基的大小

// 基: 可以表示原空间所有元素的一组极小线性无关组(不唯一)
// 普通消元法或高斯消元法

// 高斯消元求解线性基是高斯消元求解方程组的阉割版本
// 1、不需要给自由元留位置 2、不需要主元和自由元的依赖关系
// 时间复杂度  $O(n \log V)$ ,  $V$  是最大值

```

```
// 若线性基大小 = m，则原始异或和共有  $2^m - 1$  种
// 最大异或和：从高位开始贪心，异或和可以变大则异或，否则不异或
// 第 k 小异或和，拆解 k 的二进制位，求异或和
```

```
void gauss(int n) {
    len = 1;
    for (int i = bit; i >= 0; i--) {
        for (int j = len; j <= n; j++) {
            if (basis[j] & (1LL << i)) {
                swap(basis[j], basis[len]);
                break;
            }
        }
        if (basis[len] & (1LL << i)) {
            for (int j = 1; j <= n; j++) {
                if (j != len && (basis[j] & (1LL << i))) {
                    basis[j] ^= basis[len];
                }
            }
            len++;
        }
    }
    len--;
    zero = len != n;
}
```

```
void buildBasis(ll* arr, int n) {
    zero = false;
    for (int i = 1; i <= n; i++) {
        basis[i] = arr[i];
    }
    gauss(n);
}
```

```
// 第 k 小非空异或和
ll GetXorKth(ll k) {
    if (k == 1 && zero) {
        return 0;
    }
    if (zero) {
        k = k - 1;
    }
    if (k >= (1LL << len)) {
        return -1;
    }
}
```

```

    ll ans = 0;
    for (int i = len, j = 0; i >= 1; i--, j++) {
        if ((k & (1LL << j))) {
            ans ^= basis[i];
        }
    }
    return ans;
}

// 最大异或和
ll GetXorMax() {
    ll res = 0;
    for (int i = 1; i <= len; i++) {
        res = max(res, res ^ basis[i]);
    }
    return res;
}

```

异或空间线性基的合并

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const int bit = 30;

// 把 x 插入到 b 里去
void insert(ll x, ll* b) {
    for (int i = bit; i >= 0; i--) {
        if ((x >> i) & 1) {
            if (b[i] == 0) {
                b[i] = x;
                return;
            }
            x ^= b[i];
        }
    }
}

// 把 b 的基合并到 t 里
void merge(ll* b, ll* t) {
    for (int i = bit; i >= 0; i--) {
        if (b[i] != 0) {
            insert(b[i], t);
        }
    }
}

```

扩展中国剩余定理模板例题

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;
const int MAXN = 1e5;

// https://www.luogu.com.cn/problem/P4774
// 该题核心在于对中国剩余定理过程中 a, b, c 求解的定制

ll hp[MAXN + 1]; // 生命
ll recovery[MAXN + 1]; // 恢复
ll attack[MAXN + 1]; // 用于攻击第 i 条龙的剑的伤害
ll reward[MAXN + 1]; // 奖励
ll maxCnt = 0;

ll exgcd(ll hp, ll attack, ll& x, ll& y) {
    if (attack == 0) {
        x = 1;
        y = 0;
        return hp;
    }
    else {
        ll g = exgcd(attack, hp % attack, x, y);
        ll tmp = y;
        y = x - y * (hp / attack);
        x = tmp;
        return g;
    }
}

ll excrt(int n) {
    // attack[i] * ans ≡ hp[i] (% recovery[i])
    // ans = lcm * x + tail
    // attack[i] * ans = lcm * x * attack[i] + tail * attack[i]
    // attack[i] * ans = recovery[i] * y + hp[i]
    // lcm * attack[i] * x + recovery[i] * y = hp[i] - tail * attack[i]
    ll tail = 0, lcm = 1, tmp, a, b, c, x0;
    for (int i = 1; i <= n; i++) {
        a = ill(lcm * attack[i]) % recovery[i];
        b = recovery[i];
        c = ((hp[i] - tail * attack[i]) % b + b) % b;
        ll x, y;
        ll d = exgcd(a, b, x, y);
        if (c % d != 0) {
```

```

        return -1;//无解
    }
    x0 = ((i11)x * (i11)(c / d) % i11(b / d) + i11(b / d)) % i11(b /
d);
    tmp = i11(lcm) * i11(b / d);// tmp 此时就是新的 lcm1
    tail = i11((i11)tail + i11(x0) * i11(lcm) % i11(tmp)) % i11(tmp);
    lcm = tmp;
}
// ans = ? * lcm + tail
if (tail >= maxCnt) {
    return tail;
}
return (maxCnt - tail + lcm - 1) / lcm * lcm + tail;
}

void solve() {
    int n, m;
    cin >> n >> m;
    maxCnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> hp[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> recovery[i];
    }
    for (int i = 1; i <= n; i++) {
        cin >> reward[i];
    }
    multiset<ll>st;
    for (int i = 1; i <= m; i++) {
        ll x;
        cin >> x;
        st.insert(x);
    }
    for (int i = 1; i <= n; i++) {
        if (!st.size()) {
            cout << -1 << '\n';
            return;
        }
        auto it = st.lower_bound(hp[i]);
        if (it == st.end()) { // 全都小于 a[i]
            attack[i] = *prev(st.end());
            st.erase(prev(st.end()));
        }
        else {

```

```

        if (it == st.begin()) {
            // 全部>=hp[i]
            attack[i] = *it;
            st.erase(it);
        }
        else {
            if (*it > hp[i]) {
                it--;
            }
            attack[i] = *it;
            st.erase(it);
        }
    }
    st.insert(reward[i]);
    maxCnt = max(maxCnt, ll(hp[i] + attack[i] - 1) / attack[i]);
    hp[i] %= recovery[i];
}
cout << exCRT(n) << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}

```

扩展欧几里得算法求逆元

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);
        ll tmp = y;
        y = x - y * (a / b);
    }
}

```

```

        x = tmp;
        return g;
    }
}

// 求解 a 在模 mod 意义下的逆元
// 要求 a,mod 必须互质
// 时间复杂度  $O((\log\min(a, \text{mod}))^3)$ 
ll inv(ll a, ll mod) {
    ll x, y;
    exgcd(a, mod, x, y);
    x = (x % mod + mod) % mod;
    return x;
}

```

摩尔投票问题

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 2e5;
int a[MAXN + 1];

// 摩尔投票问题：求解词频严格大于 n/2 的元素(n/2 向下取整)

bool check(int n, int x) {
    for (int i = 1, j = 0; i <= n; i++) {
        j += (a[i] == x);
        if (j > n / 2) {
            return true;
        }
    }
    return false;
}

int waterNumber(int n) {
    int hp = 0;
    int cand = -1;
    for (int i = 1; i <= n; i++) {
        if (cand == -1) {
            cand = a[i];
            hp = 1;
        }
        else {
            if (a[i] == cand) {
                hp++;
            }
            else {
                if (--hp == 0) {

```

```

        cand = -1;
    }
}
}
}
if (hp == 0) return -1;
return check(n, cand) ? cand : -1;
}

```

摩尔投票问题 k 扩展

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 2e5;
const int MAXK = 10;
//摩尔投票问题扩展,寻找数组中出现次数>n/k 的元素
//维护大小为 k-1 的候选表
//一次删除 k 个元素
//k 不能太大,时间复杂度会成为瓶颈,或问题本身变得无意义
int a[MAXN + 1];
int cands[MAXK + 1][2]; //候选表
void update(int k, int val) {
    //已在候选表
    for (int i = 0; i < k; i++) {
        if (cands[i][0] == val && cands[i][1] > 0) {
            cands[i][1]++;
            return;
        }
    }
    //存在空位
    for (int i = 0; i < k; i++) {
        if (cands[i][1] == 0) {
            cands[i][0] = val;
            cands[i][1] = 1;
            return;
        }
    }
    //进行攻击
    for (int i = 0; i < k; i++) {
        if (cands[i][1] > 0) {
            cands[i][1]--;
        }
    }
}
void collect(int k, int n, vector<int>& ans) {
    for (int i = 0, cur, real; i < k; i++) {

```

```

        if (cands[i][1] > 0) {
            cur = cands[i][0];
            real = 0;
            for (int j = 1; j <= n; j++) {
                if (cur == a[j]) {
                    real++;
                }
            }
            if (real > n / (k + 1)) {
                ans.push_back(cur);
            }
        }
    }
}

vector<int> waterNumber(int n, int k) {
    k = k - 1; // 维护的候选表大小是 k-1, 此步必须有!!!
    for (int i = 1; i <= n; i++) {
        update(k, a[i]);
    }
    vector<int> ans;
    collect(k, n, ans);
    return ans;
}

```

矩阵快速幂

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int mod = 998244353;

// 矩阵快速幂、矩阵乘法
// 往往同时用于动态规划的优化
// 矩阵快速幂时间复杂度:
// 1 维 k 阶, k 维 1 阶:  $O(\log n * k^3)$ 

// 矩阵快速幂解决 dp
// 核心在于把 i - 1 和 i 的严格位置依赖关系
// 转移到矩阵中去, 关系矩阵 A
//  $dp[i] = A * dp[i - 1]$ 

// 矩阵乘法: 前列必须等于后行, (使用引用符号, 省去拷贝的耗费)
vector<vector<ll>> multiply(const vector<vector<ll>>& a, const
vector<vector<ll>>& b) {
    int n = a.size();
    int m = b[0].size();

```

```

    int k = a[0].size();
    vector<vector<ll>>ans(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            for (int c = 0; c < k; c++) {
                ans[i][j] += a[i][c] * b[c][j];
            }
        }
    }
    return ans;
}

// 矩阵快速幂：严格用于方阵，即  $n * n$  的矩阵 ( $n \geq 1$ )
// 此处不要使用引用符号，直接拷贝过来
vector<vector<ll>> MatPow(vector<vector<ll>> a, ll b) {
    int n = a.size();
    vector<vector<ll>>ans(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) { // 主对角线必须为 1，其他为 0
        ans[i][i] = 1;
    }
    while (b) {
        if (b & 1) {
            ans = multiply(ans, a);
        }
        a = multiply(a, a);
        b >>= 1;
    }
    return ans;
}

// 例题: https://ac.nowcoder.com/acm/contest/139659/F
// 注意!：该题存在取模运算

// 关系映射，行表示第  $i$  个位置，列表示第  $i - 1$  个位置
// 表示在第  $i - 1$  个位置处于某种状态时，第  $i$  个位置得到某个状态的方案数
vector<vector<ll>>A = { {1,1,0,1},
                      {1,1,1,0},
                      {0,1,1,1},
                      {1,0,1,1} };

// 初始单位状态
vector<vector<ll>>v0 = { {1},
                       {0},
                       {0},

```

```

        {0} };

void solve() {
    ll n, r;
    cin >> n >> r;
    vector<vector<ll>>ans = multiply(MatPow(A, n), v0);
    cout << ans[r % 4][0] % mod << '\n';
}

```

积性函数

```

// 函数 f(n), n ≥ 1
// 积性函数: f(1)=1 且对于 gcd(m,n)=1, 存在 f(m*n)=f(n)*f(m)
// 完全积性函数: f(1)=1 且对于 n ≥ 1, m ≥ 1, 存在 f(m*n)=f(n)*f(m)
// 完全积性函数
// 除了 f(n*m)=f(n)*f(m)
// 也可以是 f(n*m*...*k)=f(n)*f(m)*...f(k) 多个

```

约瑟夫环

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 记某次操作的环大小 n, 被杀节点编号 s, 循环 k 次
// 操作前节点编号记 OddPos, 操作后 NewPos
// 则存在: OddPos = (NewPos + k - 1) % n + 1
// 最后结束的操作, NewPos = 1

// https://www.luogu.com.cn/problem/P8671

void Cycle() {
    int n, k;
    cin >> n >> k;
    int ans = 1;
    // 最后一个的 NewPos = 1
    // 枚举上次操作的环大小 c
    for (int c = 2; c <= n; c++) {
        ans = (ans + k - 1) % c + 1;
    }
    cout << ans << '\n';
}

```

线性欧拉筛质数

```

#include<bits/stdc++.h>
using namespace std;

```

```

using ll = long long;
const int MAXN = 1e7;
ll minFactor[MAXN + 10]; // minFactor[i]表示 i 的最小质因子(线性欧拉筛维护最小质因子表)
bitset<MAXN + 1>isPrime; // 标记质数
ll Primes[MAXN + 1];
void getPrimes_Linear(ll n) {
    isPrime.set();
    isPrime[0] = isPrime[1] = 0;
    Primes[0] = 0;
    for (ll i = 2; i <= n; i++) {
        if (isPrime[i]) {
            minFactor[i] = i; // 质数的最小质因子是他本身
            Primes[++Primes[0]] = i;
        }
        for (ll j = 1, v; j <= Primes[0]; j++) {
            v = Primes[j];
            if (v * i > n) break;
            isPrime[i * v] = 0;
            minFactor[i * v] = v; // 获取最小质因子, 必须放在 break 前
            if (i % v == 0) break;
        }
    }
}

```

组合数学

```

#include<bits/stdc++.h>
using namespace std;

// m=[0,n]
// 求和  $C(n,m) \cdot 2^m = 3^n$ 
//  $C(n,0) \cdot 2^0 + C(n,1) \cdot 2^1 + \dots + C(n,m) \cdot 2^m + \dots + C(n,n) \cdot 2^n = 3^n$ 

// 排列组合
ll C(ll n, ll m) {
    if (m == 0 || m == n) return 1;
    else if (m < 0 || m > n) return 0;
    ll ans = N[n] % mod * inv_(N[m] * N[n - m], mod) % mod;
    return ans;
}

ll f(ll n, ll l, ll k) {
    // n 个元素(数量不限), 要求用 n 个元素凑出长度=l 的数组

```

```

// 两个相同元素之间的间隔要>=k
// 0<=k<n<=1
// 可能的方案数=n!*(n-k)^(1-k)/(n-k)!
// n-k 的 1-k 次幂
return N[n] * pw(n - k, 1 - k) % mod * inv_(N[n - k], mod);
}

```

裴蜀定理和扩展欧几里得算法

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 裴蜀定理:
// 如果 a, b 是不全为 0 的整数, 则一定存在整数 x, y, 使得 ax + by = gcd(a, b)
// 注意: 其中的 x, y 只要满足整数即可
// a 和 b 的最大公约数 = ax 和 by, 随意给定的整数 x, y, 能取得的最小正数差值
// 推论:
// 1、若 a, b 是不全为 0 的整数, a, b 互质, 当且仅当存在整数 x, y, 使得 ax + by = 1
// 2、若 a, b 是不全为 0 的整数, 并且 ax + by = c 有解, 那么 c 一定是 gcd(a, b) 的整数倍(0 倍也算)
// 3、a 和 b 两项的裴蜀定理, 可以推广到多项的情况
// 例, ax + by + cz = g * k, 其中 g = gcd(a, b, c), k >= 0, 一定存在 x, y, z 的解

// 扩展欧几里得算法:
// 给定 ax + by = c, c 是 gcd(a, b) 的倍数
// g = gcd(a, b)
// exgcd 求解 x, y 的一组特解 x0, y0
// 该特解 x0, y0 是 ax + by = gcd(a, b) 时的解
// 当前公式 ax + by 的解: (xx, yy) = (x0 * (c / d), y0 * (c / d))
// 通解: x = x0 + (b / d) * n, y = y0 - (a / d) * n
// 其中 x 的最小非负解 minx = (x0 * (c / d) % (b / d) + (b / d)) % (b / d)
// 时间复杂度 O((logmin(a, b)) ^ 3)
// 注意: 传入的 a, b 不能出现负数
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    else {
        ll g = exgcd(b, a % b, x, y);

```

```

        ll tmp = y;
        y = x - y * (a / b);
        x = tmp;
        return g;
    }
}

```

试除法质数判断

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
bool is_Prime(ll x) {
    //判断 x 是否为质数
    //复杂度 O(sqrt(n))
    if (x <= 1) {
        return false;
    }
    else if (x == 2) {
        return true;
    }
    else if (x % 2 == 0) {
        return false;
    }
    else {
        for (ll r = 3; r * r <= x; r += 2) {
            if (x % r == 0) {
                return false;
            }
        }
    }
    return true;
}

```

质因数拆分

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e7;
bitset<2 * MAXN + 1>isPrime;//标记质数
vector<ll>Primes;//存储质数
vector<ll>factors[2 * MAXN + 1];//factors[i]存储 i 的质因数

void getFactors(ll n) {
    //复杂度 O(sqrt(n)/log2(n))
    //分解 n 的质因数
}

```

```

    ll val = n;
    if (isPrime[n] == 1) {
        factors[n].push_back(n);
        return;
    }
    for (ll p : Primes) {
        if (val == 1) break;
        if ((ll)p * p > (ll)val) break;
        if (val % p != 0) continue;
        factors[n].push_back(p);
        while (val % p == 0) val /= p;
    }
    if (val > 1) factors[n].push_back(val);
}

//获取 1~n 范围的所有数的质因数
void GetFactors(ll n) {
    for (int p : Primes) {
        for (int i = p; i <= n; i += p) {
            factors[i].push_back(p);
        }
    }
}

```

费马逆元和暴力逆元

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;
ll pw(ll a, ll b){}

//费马小定理,求逆元
ll inv_(ll a, ll p) {
    return pw(a, p - 2);
}

//暴力求逆元
//只需满足 gcd(a,mod)=1 即可
//时间复杂度 O(mod),建议使用扩展欧几里得算法求解
ll inv02_(ll a, ll mod) {
    a %= mod;
    for (ll t = 1; t < mod; t++) {
        if ((a * t) % mod == 1) return t;
    }
}

```

```
    return 1;
}
```

进制转换

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const string DIGIT =
"0123456789"
"abcdefghijklmnopqrstuvwxyz"
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"
"!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~";

//该 DIGIT 字符最多支持 94 进制的转换
//如果仅限数字和字母最多支持 62 进制的转换

//把 x 转换为 base 进制
string toBase(ll x, int base) {
    if (x == 0) {
        return "0";
    }
    string ans;
    while (x) {
        ans += DIGIT[x % base];
        x /= base;
    }
    reverse(ans.begin(), ans.end());
    return ans;
}

ll pw(ll a, ll b) {
    ll res = 1;
    while (b) {
        if (b & 1) {
            res = res * a;
        }
        a = a * a;
        b >>= 1;
    }
    return res;
}

//获取 x 在 base 进制下的第 i 位
ll getDigit(ll x, int base, int i) {
    return x / pw(base, i) % base;
}
```

```

}

//basej 进制下的第 i 位设置为 v
ll setDigit(ll s, int base, int i, int v) {
    ll p = pw(base, i);
    ll old = s / p % base;
    return s + (v - old) * p;
}

```

逆元的线性递推

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//逆元的线性递推
//例题: https://www.luogu.com.cn/problem/P3811
ll inv[3000001];
void initial_inv(ll n, ll p) {
    inv[1] = 1;
    for (ll i = 2; i <= n; i++) {
        inv[i] = p - inv[p % i] * (p / i) % p;
    }
}

```

逆康托展开

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 逆康托展开:
// 给定数字 1~n, 给定 1~n 的排列 P
// P 排列的名次 x, 求第 x + m 的排列
// 核心: 阶乘进制, 第 i 位使用 i! 作为基准(最低位 0)
// 第 i 位的数位就是 rightSmall[P[i]](康托展开内容)

// https://www.luogu.com.cn/problem/U72177

const int MAXN = 1e5;
int sum[MAXN << 2 | 1];
ll a[MAXN + 1];
void add(int jobi, int jobv, int l, int r, int i);
int range(int jobl, int jobr, int l, int r, int i);
void build(int l, int r, int i);

```

```

int queryAndDelete(int jobk, int l, int r, int i);
// 核心代码:
void compute(int n, ll m) {
    build(1, n, 1);

    // 把排列的每一位,转化为对应的阶乘进制位
    // 方法: 小于当前位值的未使用元素个数
    for (int i = 1; i <= n; i++) {
        int tmp = a[i];
        a[i] = range(1, tmp - 1, 1, n, 1);
        add(tmp, -1, 1, n, 1);
    }

    // 阶乘进位:
    // 和下一位相比,多乘的值 val,进行除法取模进位
    a[n] += m;
    for (int i = n; i >= 1; i--) {
        a[i - 1] += a[i] / ll(n - i + 1);
        a[i] %= ll(n - i + 1);
    }
    build(1, n, 1);
    for (int i = 1; i <= n; i++) {
        a[i] = queryAndDelete(a[i] + 1, 1, n, 1);
    }
}

void build(int l, int r, int i) {
    if (l == r) {
        sum[i] = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1);
        build(mid + 1, r, i << 1 | 1);
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
    }
}

int range(int jobl, int jobr, int l, int r, int i) {
    if (jobl > jobr) return 0;
    if (jobl <= l && r <= jobr) {
        return sum[i];
    }
    else {
        int mid = (l + r) >> 1;
        int ans = 0;

```

```

        if (jobl <= mid) {
            ans += range(jobl, jobr, l, mid, i << 1);
        }
        if (jobr > mid) {
            ans += range(jobl, jobr, mid + 1, r, i << 1 | 1);
        }
        return ans;
    }
}

void add(int jobi, int jobv, int l, int r, int i) {
    if (l == r) {
        sum[i] += jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            add(jobi, jobv, l, mid, i << 1);
        }
        else {
            add(jobi, jobv, mid + 1, r, i << 1 | 1);
        }
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
    }
}

int queryAndDelete(int jobk, int l, int r, int i) {
    if (l == r) {
        sum[i]--;
        return l;
    }
    else {
        int lsum = sum[i << 1];
        int rsum = sum[i << 1 | 1];
        int mid = (l + r) >> 1;
        int ans;
        if (lsum >= jobk) {
            ans = queryAndDelete(jobk, l, mid, i << 1);
        }
        else {
            ans = queryAndDelete(jobk - lsum, mid + 1, r, i << 1 | 1);
        }
        sum[i] = sum[i << 1] + sum[i << 1 | 1];
        return ans;
    }
}

```

```

void solve() {
    ll n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    compute(n, m);
    for (int i = 1; i <= n; i++) {
        cout << a[i] << ' ';
    }
}

```

阶乘逆元的线性递推

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;

//连续阶乘逆元的线性递推
//即阶乘的逆元
ll invN[200001];
void initial_invN(int n) {
    invN[0] = 1;
    invN[n] = inv_(N[n], mod);
    for (ll i = n - 1; i >= 1; i--) {
        invN[i] = (i + 1) * invN[i + 1] % mod;
    }
}

```

随机数函数

```

#include<bits/stdc++.h>
using namespace std;
int main()
{
    random_device rd; //获取真随机数种子

    //性能即生成速度(cpu 时间)和内存占用
    //质量即接近真随机数的程度,越高其生成数据周期越长,规律性越弱

    //直接使用 rd()种子生成的数据是真随机,质量最好,但是性能相对一般
    uniform_int_distribution<int> dist(1, 100);
    cout << dist(rd) << '\n';

    //使用伪随机数引擎接受来自 rd()的种子,从而生成伪随机种子,伪随机但性能提升显著
}

```

```

//根据真随机种子,生成伪随机种子
mt19937 gen(rd());//质量高,性能优秀,周期律约 1e6001

//mt19937_64 生成 64 位数值的优化选择,周期律约 1e6001
//minstd_rand 性能速度极佳,质量差,周期律约 2^31-2
//ranlux48 质量极高,性能低,周期律约 1e576

uniform_int_distribution<int> int_dist(1, 100);//整数均匀分布
uniform_real_distribution<double> real_dist(0.0, 100.0);
cout << int_dist(gen) << '\n';//随机数来源是 gen(种子)
cout << real_dist(gen) << '\n';
}

```

高斯消元法(同余方程组)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 100;
const int MOD = 7;;
// 逆元线性递推, 获取逆元表
ll inv[MOD + 1];
ll mat[MAXN + 1][MAXN + 1];
void inv_(int mod) {
    inv[1] = 1;
    for (int i = 2; i < mod; i++) {
        inv[i] = (int)(mod - (ll)inv[mod % i] * ll(mod / i) % mod);
    }
}
ll gcd(ll a, ll b) {
    return b == 0 ? a : gcd(b, a % b);
}

// 高斯消元求解同余方程组
// 时间复杂度  $O(n^3)$ 
// 主元依赖自由元, 自由元不依赖任何其他元
// 唯一解: 存在  $n$  个主元 (即所有列都是主元列)
// 多解: 存在自由元, 且所有自由元所在行均为  $0 = 0$ 
// 矛盾: 存在自由元所在行为  $0 = b(b \neq 0)$ 

//  $n$  个变量,  $n$  个表达式
// 第  $n + 1$  列是值
// 若方程数不足  $n$ , 需补全零元; 若方程冗余 (线性相关), 消元后自然产生自由元

```

```

// 检查解时，只需要检查实际变量个数 n 的行[1, n]

// 处理 i 行 i 列主元时
// 注意!：若要严格区分唯一解、矛盾、多解，j < i 的非主元行也要参与计算
// 若只需判断是否存在唯一解，从第 i 行开始即可

// !!! 注意!!!
// 所有系数若读入负数，使用以下公式保证 gauss 消元初始系数都是正数
// -a % mod = (-a % mod + mod) % mod

void gauss(int n, int mod) {
    for (int i = 1; i <= n; i++) {
        // 处理 i 行 i 列主元
        // 获取位置 i 等于 1 的行
        // ! 注意!：若要严格区分唯一解、矛盾、多解，j < i 的非主元行也要参与计
        // 算
        // 若只需判断是否存在唯一解，从第 i 行开始即可
        for (int j = 1; j <= n; j++) {
            if (j < i && mat[j][j] != 0) { // 主元行，跳过
                continue;
            }
            if (mat[j][i] != 0) { // 对于同余方程组，只要发现非 0 值直接交换
                swap(mat[i], mat[j]);
                break;
            }
        }
        if (mat[i][i] != 0) {
            for (int j = 1; j <= n; j++) {
                if (i != j && mat[j][i] != 0) {
                    ll g = gcd(mat[j][i], mat[i][i]);
                    ll a = mat[i][i] / g;
                    ll b = mat[j][i] / g;

                    if (j < i && mat[j][j] != 0) {
                        // !!! 注意!!!
                        // 若 j 行存在主元，从 j 到 i - 1 列所有系数 * a
                        // 维护正确的主元和自由元的关系
                        for (int k = j; k < i; k++) {
                            mat[j][k] = (mat[j][k] * a) % mod;
                        }
                    }
                    // 正常消元
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] = ((mat[j][k] * a - mat[i][k] * b) % mod

```

```

+ mod) % mod;
    }
    }
}

bool sign = false;
// 判断解
for (int i = 1; i <= n; i++) {
    if (mat[i][i] != 0) {
        // 主元行, 判断是否被自由元影响
        bool flg = false;
        for (int j = i + 1; j <= n; j++) {
            if (mat[i][j] != 0) {
                flg = true;
                break;
            }
        }
        if (!flg) {
            // 不被自由元影响, 确定唯一解的值
            mat[i][n + 1] = (mat[i][n + 1] * inv[mat[i][i]]) % mod;
            mat[i][i] = 1;
        }
    }
    else {
        if (mat[i][n + 1] != 0) {
            cout << "矛盾, 无解" << '\n';
            return;
        }
        sign = true;
    }
}

if (sign) {
    cout << "多解" << '\n';
}
else {
    cout << "存在唯一解" << '\n';
}
}

```

高斯消元法(异或方程组)

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 100;
int mat[MAXN + 1][MAXN + 1];

```

```

// 高斯消元求解异或方程组
// 时间复杂度  $O(n^3)$ 
// 主元依赖自由元，自由元不依赖任何其他元
// 唯一解：存在  $n$  个主元（即所有列都是主元列）
// 多解：存在自由元，且所有自由元所在行均为  $0 = 0$ 
// 矛盾：存在自由元所在行为  $0 = b(b \neq 0)$ 

// 高斯消元法应用的核心在于：把关系转化为 01 的异或关系
// 例如，奇偶性的改变

//  $n$  个变量， $n$  个表达式
// 第  $n + 1$  列是值
// 若方程数不足  $n$ ，需补全零元；若方程冗余（线性相关），消元后自然产生自由元
// 检查解时，只需要检查实际变量个数  $n$  的行[1, n]

// 处理  $i$  行  $i$  列主元时
// 注意！：若要严格区分唯一解、矛盾、多解， $j < i$  的非主元行也要参与计算
// 若只需判断是否存在唯一解，从第  $i$  行开始即可

void gauss(int n) {
    for (int i = 1; i <= n; i++) {
        // 处理  $i$  行  $i$  列主元
        // 获取位置  $i$  等于 1 的行
        // 注意！：若要严格区分唯一解、矛盾、多解， $j < i$  的非主元行也要参与计算
        // 若只需判断是否存在唯一解，从第  $i$  行开始即可
        for (int j = 1; j <= n; j++) {
            if (j < i && mat[j][j] == 1) { // 主元行，跳过
                continue;
            }
            if (mat[j][i] == 1) { // 对于异或方程组，只要发现 1 直接交换
                swap(mat[i], mat[j]);
                break;
            }
        }

        if (mat[i][i] == 1) {
            for (int j = 1; j <= n; j++) {
                if (i != j && mat[j][i] == 1) {
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] ^= mat[i][k];
                    }
                }
            }
        }
    }
}

```

```

    }
}

// 判断解的类型
bool sign = false;
for (int i = 1; i <= n; i++) {
    if (mat[i][i] == 0) { // 自由元
        if (mat[i][n + 1] != 0) {
            cout << "矛盾, 无解" << '\n';
            return;
        }
        else {
            sign = true;
        }
    }
}
if (sign) {
    cout << "多解" << '\n';
    return;
}
else {
    cout << "存在唯一解" << '\n';
}
for (int i = 1; i <= n; i++) {
    cout << mat[i][n + 1] << ' ';
}
cout << '\n';
}

```

高斯消元法(线性方程组)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const int MAXN = 100;
const ld eps = 1e-12;
ld mat[MAXN + 1][MAXN + 1];

// 高斯消元求解线性方程组
// 时间复杂度  $O(n^3)$ 
// 主元依赖自由元, 自由元不依赖任何其他元
// 唯一解: 存在  $n$  个主元 (即所有列都是主元列)
// 多解: 存在自由元, 且所有自由元所在行均为  $0 = 0$ 
// 矛盾: 存在自由元所在行为  $0 = b (b \neq 0)$ 

```

```

// n 个变量, n 个表达式
// 第 n + 1 列是值
// 若方程数不足 n, 需补全零元; 若方程冗余 (线性相关), 消元后自然产生自由元
// 检查解时, 只需要检查实际变量个数 n 的行[1, n]

// 处理 i 行 i 列主元时
// 注意!: 若要严格区分唯一解、矛盾、多解, j < i 的非主元行也要参与计算
// 若只需判断是否存在唯一解, 从第 i 行开始即可

void gauss(int n) {
    cout << fixed << setprecision(16);
    for (int i = 1; i <= n; i++) {
        // 处理 i 行 i 列主元
        int maxpick = i;
        // 获取位置 i 最大的行
        // 注意!: 若要严格区分唯一解、矛盾、多解, j < i 的非主元行也要参与计算
        // 若只需判断是否存在唯一解, 从第 i 行开始即可
        for (int j = 1; j <= n; j++) {
            if (j < i && fabs1(mat[j][j]) >= eps) { // 主元行, 跳过
                continue;
            }
            if (fabs1(mat[j][i]) > fabs1(mat[maxpick][i])) {
                maxpick = j;
            }
        }
        // 交换最大行
        swap(mat[i], mat[maxpick]);
        // 存在非 0 最大值, 开始消元
        if (fabs1(mat[i][i]) >= eps) {
            ld tmp = mat[i][i];
            for (int j = i; j <= n + 1; j++) {
                mat[i][j] /= tmp;
            }
            for (int j = 1; j <= n; j++) {
                if (i != j) {
                    ld rate = mat[j][i] / mat[i][i];
                    for (int k = i; k <= n + 1; k++) {
                        mat[j][k] -= mat[i][k] * rate;
                    }
                }
            }
        }
    }
}

```

```

// 判断解的类型
bool sign = false;
for (int i = 1; i <= n; i++) {
    if (fabsl(mat[i][i]) < eps) { // 自由元
        if (fabsl(mat[i][n + 1]) >= eps) {
            cout << "矛盾, 无解" << '\n';
            return;
        }
        else {
            sign = true;
        }
    }
}
if (sign) {
    cout << "多解" << '\n';
    return;
}
else {
    cout << "存在唯一解" << '\n';
}
for (int i = 1; i <= n; i++) {
    cout << mat[i][n + 1] << ' ';
}
cout << '\n';
}

```

高精度乘法

```

#include<bits/stdc++.h>
using namespace std;

// a * b
// 数组逆序存储数位
// 从 1 ~ n 分别是低位到高位

// 高精度乘法, 用数组存储数位, 手动模拟乘法运算
// 对于最终结果, 数位长度不超过 a, b 数位之和
const int MAXN = 300;
int a[MAXN + 1];
int b[MAXN + 1];
int res[2 * MAXN + 1];

void multiply(string s1, string s2) {

```

```

int n = s1.size();
int m = s2.size();
for (int i = 1; i <= n; i++) {
    a[i] = s1[n - i] - '0';
}
for (int i = 1; i <= m; i++) {
    b[i] = s2[m - i] - '0';
}
fill(res, res + n + m + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        res[j + i - 1] += a[i] * b[j];
        if (res[j + i - 1] >= 10) {
            res[j + i] += res[j + i - 1] / 10;
            res[j + i - 1] %= 10;
        }
    }
}
bool isZero = true;
int idx = n + m;
for (int i = idx; i >= 0; i--) {
    if (res[i] != 0) {
        idx = i;
        isZero = false;
        break;
    }
}
if (isZero) {
    cout << 0 << '\n';
    return;
}
for (int i = idx; i >= 1; i--) {
    cout << res[i];
}
cout << '\n';
}

```

麦乐鸡块定理(笑了)

```

#include<bits/stdc++.h>
using namespace std;

// 给定两个互质的正整数 a, b
// 无法由  $ax + by = c$  得到的最大整数是  $ab - a - b$ 

void MLJK(int a, int b) {
    if (gcd(a, b) != 1) {

```

```

        cout << "No delicious!!!" << '\n';
    }
    else {
        cout << a * b - a - b << '\n';
        cout << "Perfect!!!" << '\n';
    }
}

```

五、图论

1、最短路问题

01BFS

```

#include<bits/stdc++.h>
using namespace std;

//01bfs 处理边权(耗费)全是 0 或 1 的图
//求原点到任意点的最短距离
//bfs+双端队列求解

//流程(核心): 入队,遍历儿子
//若 dist[son]>dist[father]+w,更新
//dist[son]=dist[father]+w
//w==0,入队头
//w==1,入队尾

class Tmp {
private:

    static const int INF = 1e9;

    int mat[501][501]; //二维矩阵
    int dist[501][501]; //最短路径

    pair<int, int> dir[4] = { {1,0},{-1,0},{0,1},{0,-1} };

    void bfs(int rx, int ry, int n, int m) {
        dist[rx][ry] = 0; //起点路径=0
        deque<array<int, 2>> q;
        q.push_front({ rx, ry });
        while (q.size()) {
            //从队头弹出
            auto [x, y] = q.front();
            q.pop_front();

```

```

        for (auto [di, dj] : dir) {
            int nx = x + di, ny = y + dj;
            if (nx<1 || nx>n || ny<1 || ny>m) {
                continue;
            }
            //根据题目定义获取边权
            int w = mat[x][y] == mat[nx][ny] ? 0 : 1;
            if (dist[nx][ny] > dist[x][y] + w) {
                dist[nx][ny] = dist[x][y] + w;

                //核心代码: 0 入队头, 1 入队尾
                if (w == 0) {
                    q.push_front({ nx,ny });
                }
                else {
                    q.push_back({ nx,ny });
                }
            }
        }
    }

    void init(int n, int m) {
        //初始化....略.....
    }

public:
    //模板例题: https://www.luogu.com.cn/problem/P4554
    void solve() {
        //输入,略.....
    }
};

```

A 星算法

```

//A*算法是 Djk 算法的扩展
//不同之处:
//A*算法针对于指定起点和终点的最短路
//入堆时, 不加入真实的距离, 而是真是距离+预估距离
//A*算法依赖强大的启发式预估函数实现路径更优选择
//使得搜索空间更贴近最短路附近(类似拟合)

```

```
//进而使得时间复杂度得到常数优化
//预估函数计算出的距离<=真实距离
```

BellmanFordSPFA 单源最短路

```
#include<bits/stdc++.h>
using namespace std;

//BellmanFord 和 SPFA
//单源最短路算法
//队列优化
//出队->松弛操作->入队(不在队内)->出队
//要求不能有负环

//松弛操作(核心):
//通过边和点去检查是否更新更小的 dist
//dist[son]=min(dist[father]+wight)

const int MAXN = 2e3;
const int MAXM = 3e3;
const int INF = 1e9;

int idx = 0;
int head[MAXN + 1];
int nxt[2 * MAXM + 1];
int to[2 * MAXM + 1];
int weight[2 * MAXM + 1];

void addEdge(int u, int v, int w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

int roundCnt[MAXN + 1];
int dist[MAXN + 1];
bool vis[MAXN + 1];
//start 是源头
int SPFA(int start, int n) {
    queue<int>q;
    q.push(start);
    vis[start] = true;
    roundCnt[start] = 1;//轮次
    dist[start] = 0;
    while (q.size()) {
```

```

    int cur = q.front();
    vis[cur] = false;
    q.pop();

    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        int son = to[edge];
        int w = weight[edge];
        if (dist[cur] + w < dist[son]) {
            dist[son] = dist[cur] + w;
            if (!vis[son]) {

                //只有当不在队列里时,视为一轮松弛操作
                if (++roundCnt[son] >= n) {
                    //判断负环,陷入循环
                    return -1;
                }
                vis[son] = true;
                q.push(son);
            }
        }
    }
}

return 1; //成功得到结果
}

```

Dijkstra 单源最短路

```

#include<bits/stdc++.h>
using namespace std;

//Djt 算法解决边权非负图的单源最短路径问题
//有向/无向图,有/无环,边权非负
//以上类型的图都可以解决

//反向索引堆优化
//时间复杂度: 点 n, 边 m
//O((n+m)logn)
//未优化时间复杂度:O((n+m)logm)

//Djk 和 Bfs 的差别
//Djk 在弹出时标记
//普通 bfs 在入队时标记
//原因在于 bfs 的所有边权相同,逐层遍历更早抵达的一定更优
//而复杂的图需要用 Djk 在弹出时标记,获取最优更新

class Tmp {

```

private:

```
// 核心代码如下:

// 此处使用了反向索引堆优化
// 普通 Djk, 维护小根堆
// 每次需要判断未出过队且 dist[v] 大于 dist[u]+weight
// 若满足, 则更新 dist[v] 并入队
// 出队时标记
void Djk(int s) {
    HeapInsert(0, s);
    dist[s] = 0;
    vis[s] = true;
    while (siz > 0) {
        pop();
        // 弹出, 意味着找到该点的最优解
        dist[heapPoint] = heapVal;
        if(vis[heapPoint])continue;
        vis[heapPoint] = true;
        for (int edge = head[heapPoint]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            ll val = weight[edge];
            // 已弹出不受理
            if (vis[son]) {
                continue;
            }
            // HeapInsert 函数自带增添、更新选择
            HeapInsert(heapVal + val, son);
        }
    }
}

using ll = long long;
static const int MAXN = 1e5;
static const int MAXM = 2e5;
static const ll INF = 1e18;

int head[MAXN + 1];
int nxt[MAXM + 1];
int to[MAXM + 1];
int idx = 0;
ll weight[MAXM + 1];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
```

```

        head[u] = idx;
        to[idx] = v;
        weight[idx] = w;
    }

    int heapPoint;
    ll heapVal;
    int siz = 0;
    int f[MAXN + 1]; //点->堆中位置
    struct {
        ll val;
        int point;
    } heap[MAXN + 1];

    void HeapSwap(int i, int j) {
        swap(f[heap[i].point], f[heap[j].point]);
        swap(heap[i], heap[j]);
    }

    void siftUp(int i) {
        while (i > 0 && heap[i].val < heap[(i - 1) >> 1].val) {
            HeapSwap(i, (i - 1) >> 1);
            i = (i - 1) >> 1;
        }
    }

    void siftDown(int i) {
        int l = 2 * i + 1, r = l + 1;
        while (l < siz) {
            int best = r < siz && heap[r].val < heap[l].val ? r : l;
            if (heap[best].val >= heap[i].val) {
                return;
            }
            HeapSwap(i, best);
            i = best;
            l = 2 * i + 1, r = l + 1;
        }
    }

    void update(ll val, int point) {
        int idx = f[point];
        //更优,更新
        if (heap[idx].val > val) {
            heap[idx].val = val;
            siftUp(idx);
        }
    }

```

```

}

void HeapInsert(ll val, int point) {
    if (f[point] != -1) {
        //已插入,更新
        update(val, point);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].point = point;
    heap[cur].val = val;
    f[point] = cur;
    siftUp(cur);
}

```

```

void pop() {
    if (siz < 1) {
        heapPoint = -1;
        return;
    }
    HeapSwap(0, --siz);
    heapVal = heap[siz].val;
    heapPoint = heap[siz].point;
    f[heapPoint] = -1; //删除映射
    siftDown(0);
}

```

```

ll dist[MAXN + 1];
bool vis[MAXN + 1];

```

```

void init(int n) {
    idx = siz = 0;
    for (int i = 1; i <= n; i++) {
        dist[i] = INF;
        head[i] = 0;
        f[i] = -1;
        vis[i] = false;
    }
}

```

public:

```

//模板例题: https://www.luogu.com.cn/problem/P4779
void solve() {
    //输入,略.....
}

```

```

    }
};

```

Floyd 多源最短路

```

//多源最短路算法
//时间复杂度  $O(n^3)$ ,  $n$  是节点个数
//枚举 bridge 跳板一定要在最外侧!!!
//适用于除了有负环的任何图(即不能存在和为负数的环)
const int MAXN=100;
const int INF=1e9;
int dist[MAXN+1][MAXN+1];
for(int bridge=1;bridge<=n;bridge++){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            //不存在路,非法
            if(dist[i][bridge]==INF||dist[bridge][j]==INF)continue;
            dist[i][j]=min(dist[i][j],dist[i][bridge]+dist[bridge][j]);
        }
    }
}

```

LGSP 分层图最短路

```

#include<bits/stdc++.h>
using namespace std;

//分层图最短路(状态空间图)
//用于图中点的状态不明确,因路径选择不同存在不同的实际状态
//把点抽象为{坐标, 状态}的形式
//具体因题而异
//例如: {坐标, 位信息}视为一个点
//基于此进行 Djk 算法

class Tmp {

private:

    static const int MAXN = 1e4;
    static const int MAXM = 5e4;
    static const int INF = 1e9;

    //建图
    int head[MAXN + 1] = { 0 };
    int nxt[2 * MAXM + 1] = { 0 };
    int to[2 * MAXM + 1] = { 0 };
    int idx = 0;

```

```

int weight[2 * MAXM + 1];

void addEdge(int u, int v, int w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

//手动堆,反向索引堆优化
int heapCnt;
int heapPoint;
int heapVal;
int siz = 0;
int f[MAXN + 1][11]; //{点,状态}->堆中位置
struct {
    int val; //耗费
    int cnt; //免费次数使用状态
    int point; //点
} heap[MAXN + 1];

void HeapSwap(int i, int j) {
    //交换映射->交换数据
    swap(f[heap[i].point][heap[i].cnt],
f[heap[j].point][heap[j].cnt]);
    swap(heap[i], heap[j]);
}

void siftUp(int i) {
    while (i > 0 && heap[i].val < heap[(i - 1) >> 1].val) {
        HeapSwap(i, (i - 1) >> 1);
        i = (i - 1) >> 1;
    }
}

void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].val < heap[l].val ? r : l;
        if (heap[best].val >= heap[i].val) {
            return;
        }
        HeapSwap(i, best);
        i = best;
        l = 2 * i + 1, r = l + 1;
    }
}

```

```

}

void update(int val, int point, int cnt) {
    int idx = f[point][cnt];
    //更优,更新
    if (heap[idx].val > val) {
        heap[idx].val = val;
        siftUp(idx);
    }
}

void HeapInsert(int val, int point, int cnt) {
    if (f[point][cnt] != -1) {
        //已插入,更新
        update(val, point, cnt);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].point = point;
    heap[cur].val = val;
    heap[cur].cnt = cnt;
    f[point][cnt] = cur;
    siftUp(cur);
}

void pop() {
    if (siz < 1) {
        heapPoint = -1;
        return;
    }
    HeapSwap(0, --siz);
    heapVal = heap[siz].val;
    heapPoint = heap[siz].point;
    heapCnt = heap[siz].cnt;
    f[heapPoint][heapCnt] = -1; //删除映射
    siftDown(0);
}

bool vis[MAXN + 1][11] = { false };

int Djk(int st, int ed, int k) {
    vis[st][0] = true;
    HeapInsert(0, st, 0);
    while (siz > 0) {
        pop();
        vis[heapPoint][heapCnt] = true;
    }
}

```

```

        if (heapPoint == ed) {
            return heapVal;
        }
        for (int edge = head[heapPoint]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            int cost = weight[edge];
            if (vis[son][heapCnt]) continue;
            //不使用免费次数
            //HeapInsert 函数自带增添、更新、不做修改的检查
            HeapInsert(cost + heapVal, son, heapCnt);
            if (heapCnt < k) {
                if (vis[son][heapCnt + 1]) continue;
                HeapInsert(heapVal, son, heapCnt + 1);
            }
        }
    }
}

public:

    //模板例题: https://www.luogu.com.cn/problem/P4568
    void solve() {
        //略.....
    }
};

```

TSP 最短路问题

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 20;

// 核心: 状态压缩
// f(status,i)表示从处于 status 状态且村庄 i 最后抵达的最短路
// 枚举下一个抵达的村庄 j
// dp[s][i]=min(dp[s|(1<<j)][j]+dist[i][j])

ll dist[MAXN + 1][MAXN + 1];
ll dp[1 << MAXN][MAXN + 1];
ll solve() {
    //模板例题:https://www.luogu.com.cn/problem/P1171
    int n;
    cin >> n;
    memset(dp, 0x7f7f7f7f7f7f7f7f, sizeof(dp));
}

```

```

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> dist[i][j];
        }
    }
    //n=4, status=111(位状态仅存储非起点)
    int status = (1 << (n - 1)) - 1;
    for (int j = 2; j <= n; j++) {
        dp[status][j] = dist[j][1];
    }
    ll ans = 1e18;
    //s 的位状态表示 2~n
    for (int s = status; s >= 0; s--) {
        for (int i = 2; i <= n; i++) {
            if ((s >> (i - 2)) & 1) {
                //位置 i 最后抵达
                for (int j = 2; j <= n; j++) {
                    if (!((s >> (j - 2)) & 1)) {
                        //下一个位置抵达 j
                        dp[s][i] = min(dp[s][i], dp[s | (1 << (j - 2))][j]
+ dist[i][j]));
                    }
                }
            }
        }
    }
    //枚举第一个抵达的位置
    for (int i = 2; i <= n; i++) {
        ans = min(ans, dp[1 << (i - 2)][i] + dist[1][i]);
    }
    return ans;
}

```

同余最短路(两次绕圈法)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int inf = 1e9;
const ll INF = 1e18;

const int MAXN = 12;
const int MAXV = 5e5;
int a[MAXN + 1];
ll dist[MAXV + 1];

int gcd(int a, int b) {

```

```

    return b == 0 ? a : gcd(b, a % b);
}

void Cycle(int n) {
    int x = a[1];
    fill(dist, dist + x, INF);
    dist[0] = 0;
    for (int i = 2; i <= n; i++) {
        int d = gcd(a[i], x);
        for (int st = 0; st < d; st++) {
            for (int cur = st, next, round = 0; round < 2; round += (cur ==
st ? 1 : 0)) {
                next = (cur + a[i]) % x;
                if (dist[cur] != INF) {
                    dist[next] = min(dist[next], dist[cur] + a[i]);
                }
                cur = next;
            }
        }
    }
}

ll Cal(ll r, int x) {
    if (r < 0) return 0;
    ll ans = 0;
    for (int i = 0; i < x; i++) {
        if (r >= dist[i]) {
            ans += max(0LL, (r - dist[i]) / x + 1);
        }
    }
    return ans;
}

void solve() {
    ll n, l, r;
    cin >> n >> l >> r;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    int siz = 0;
    sort(a + 1, a + n + 1);
    for (int i = 1; i <= n; i++) {
        if (a[i] != 0) {
            a[++siz] = a[i];
        }
    }
    if (siz == 0) {

```

```

        cout << 0 << '\n';
    }
    else {
        Cycle(siz);
        cout << Cal(r, a[1]) - Cal(l - 1, a[1]) << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int t = 1;
    // cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}

```

同余最短路 Djkt

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// https://www.luogu.com.cn/problem/P3403

// 解决数据量大的完全背包问题
// 使用特定数值的余数分组
// 注意：按照余数分组[0, x - 1]，所以是以 0 为基准而非 1
// 根据题目不同可能需要对背包容量进行定制，例如 n = n - 1
// 最终计算的 dist[i] 即 % base = i 可达到的最低值

const int MAXN = 1e5;
const ll INF = LLONG_MAX;
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
ll dist[MAXN + 1];
bool vis[MAXN + 1];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
}

```

```

    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

void Djk() {
    dist[0] = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>>q;
    q.push({ 0,0 });
    while (q.size()) {
        auto [d, u] = q.top();
        q.pop();
        if (vis[u])continue;
        dist[u] = d;
        vis[u] = true;
        for (int edge = head[u];edge > 0;edge = nxt[edge]) {
            int v = to[edge];
            if (!vis[v] && dist[v] > dist[u] + (ll)weight[edge]) {
                dist[v] = dist[u] + weight[edge];
                q.push({ dist[v],v });
            }
        }
    }
}

void solve() {
    ll h, x, y, z;
    cin >> h >> x >> y >> z;
    h = h - 1;
    // 让最小值成为 x, 减少开销
    if (y > z)swap(y, z);
    if (x > y)swap(x, y);
    fill(dist, dist + x + 1, INF);
    for (int i = 0;i < x;i++) {
        addEdge(i, (i + y) % x, y);
        addEdge(i, (i + z) % x, z);
    }
    Djk();
    ll ans = 0;
    for (int i = 0;i < x;i++) {
        if (dist[i] <= h) {
            ans += (h - dist[i]) / x + 1;
        }
    }
    cout << ans << '\n';
}

```

负环和差分约束 Floyd

```
#include<bits/stdc++.h>
using namespace std;

// 模板: https://www.luogu.com.cn/problem/P2474
// Floyd 算法核心在于 u, v 关系通过 bridge 桥接
// 该题只要求判断 a,b 和 u,v 这两队的关系
// 且给定了任意两个的某种关系, 因此可以使用 Floyd 算法把该关系严格化

int dmin[51][51]; // i 和 j 的最小可能差值
int dmax[51][51]; // i 和 j 的最大可能差值

void solve() {
    int n, a, b;
    cin >> n >> a >> b;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            char r;
            cin >> r;
            if (r == '=') {
                dmin[i][j] = dmax[i][j] = 0;
            }
            else if (r == '-') {
                // i < j
                dmin[i][j] = -2;
                dmax[i][j] = -1;
            }
            else if (r == '+') {
                // i > j
                dmin[i][j] = 1;
                dmax[i][j] = 2;
            }
            else {
                dmin[i][j] = -2;
                dmax[i][j] = 2;
            }
        }
    }
    for (int i = 1; i <= n; i++) {
        dmin[i][i] = dmax[i][i] = 0;
    }
    // 桥接, 把两两的关系更严格化
    for (int bridge = 1; bridge <= n; bridge++) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
```

```

        dmin[i][j] = max(dmin[i][j], dmin[i][bridge] +
dmin[bridge][j]);
        dmax[i][j] = min(dmax[i][j], dmax[i][bridge] +
dmax[bridge][j]);
    }
}
}
int ans1 = 0, ans2 = 0, ans3 = 0;
for (int i = 1; i <= n; i++) {
    for (int j = i + 1; j <= n; j++) {
        if (i != a && j != a && i != b && j != b) {
            if (dmin[a][i] > dmax[j][b] || dmin[a][j] > dmax[i][b]) {
                ans1++;
            }
            if (dmax[a][i] < dmin[j][b] || dmax[a][j] < dmin[i][b]) {
                ans3++;
            }
            if (dmin[a][i] == dmax[a][i] && dmin[j][b] == dmax[j][b]
&& dmin[a][i] == dmin[j][b]) {
                ans2++;
            }
            else if (dmin[b][i] == dmax[b][i] && dmin[j][a] ==
dmax[j][a] && dmin[j][a] == dmin[b][i]) {
                ans2++;
            }
        }
    }
}
cout << ans1 << ' ' << ans2 << ' ' << ans3 << '\n';
}
int main() {
    solve();
    return 0;
}

```

负环和差分约束 SPFA

```

#include<bits/stdc++.h>
using namespace std;

// 差分约束两种形式
// 1、 $x_i - y_i \leq c_i$  等价于  $x_i \leq y_i + c_i$ 
//  $c_i$  是常量
// 建边  $y_i \rightarrow x_i$ ,  $c_i$  是边权, 初始所有点权是(+INF)
// 通过 SPFA, 不断松弛操作, 若出现负环, 无解

// 2、 $x_i - y_i \geq c_i$  等价于  $x_i \geq y_i + c_i$ 

```

```

// ci 是常量
// 建边 yi->xi, ci 是边权, 初始所有点权是(-INF)
// 通过 SPFA, 不断扩张操作, 若出现正环, 无解

// 连通超级源点:
// 建图时需要一个超级源点 root 作为初始点连接所有点
// 二者可以相互转化, 区别在于松弛和扩张, 负环和正环
// ! 注意! 因为超级源点 root 的原因, 判断条件不是 >= n, 而是 > n

// 限制超级源点:
// 若出现若干变量的值已确定
// 建立一个限制超级源点 limitRoot, 分别和已确定值的变量连接正负两条边
// 超级源点 root 和限制超级源点一定要分离

// SPFA
// 时间复杂度:O(n* m)
// n:点的个数 m:边的个数

// 模板: https://www.luogu.com.cn/problem/P5960

const int MAXN = 5e3;
using ll = long long;
const ll INF = 1e18;

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
int roundCnt[MAXN + 1];
ll dist[MAXN + 1];
bool vis[MAXN + 1];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

void SPFA(int n) {
    int root = 0;
    queue<int>q;
    vis[root] = true;
    dist[root] = 0;

```

```

roundCnt[root] = 1;
q.push(root);
while (q.size()) {
    int u = q.front();
    vis[u] = false;
    q.pop();
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        int v = to[edge];
        if (dist[v] > dist[u] + weight[edge]) {
            dist[v] = dist[u] + weight[edge];
            if (!vis[v]) {
                // !!! 注意!!!
                // 此处因为超级源点 root 的原因
                // 判断条件不是 >= n, 而是 > n
                if (++roundCnt[v] > n) {
                    cout << "NO" << '\n';
                    return;
                }
                q.push(v);
                vis[v] = true;
            }
        }
    }
}

for (int i = 1; i <= n; i++) {
    cout << dist[i] << ' ';
}

}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        addEdge(0, i, 0);
        head[i] = 0;
        vis[i] = false;
        dist[i] = INF;
        roundCnt[i] = 0;
    }
    for (int i = 1; i <= m; i++) {
        ll x, y, c;
        cin >> x >> y >> c;
        // x - y <= c
        // x <= y + c
        // y->x, c
        addEdge(y, x, c);
    }
}

```

```

    }
    SPFA(n);
}

```

2、树上问题

dfs 判断存在负环

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 有向有权图
// dfs 判断是否存在负环

const int MAXN = 3e3;
const int MAXM = 1e4;
int head[MAXN + 1];
int nxt[MAXM + 1];
int to[MAXM + 1];
ll weight[MAXM + 1];
ll value[MAXN + 1];
bool vis[MAXN + 1];
int idx = 0;

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

bool dfs(int u, int n) {
    vis[u] = true;
    if (u == 0) {
        for (int i = 1; i <= n; i++) {
            if (dfs(i, n)) {
                return true;
            }
        }
    }
    else {
        for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
            int v = to[edge];
            ll w = weight[edge];
            if (value[v] > value[u] + w) {
                value[v] = value[u] + w;
            }
        }
    }
}

```

```

        if (vis[v] || dfs(v, n)) {
            return true;
        }
    }
}
}
vis[u] = false;
return false;
}

bool check(int n) {
    // 判断是否存在负环，所有点的路径累加权值设置为 0，保证只有出现负数才会更新
    fill(value, value + n + 1, 0);
    fill(vis, vis + n + 1, false);
    // 从超级源点 0 出发抵达每个点
    return dfs(0, n);
}

```

LcaStjump 最低公共祖先

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 5e5;
//树上倍增求 LCA
//预处理 O(nlogn)
//单次在线查询 O(logn)
//整体时间复杂度: O(nlogn)

//https://www.luogu.com.cn/problem/P3379

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int power = 21;
int deep[MAXN + 1];
int stjump[MAXN + 1][21];

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void dfs(int cur, int fa) {
    deep[cur] = deep[fa] + 1;
}

```

```

    stjump[cur][0] = fa;
    for (int p = 1; p <= power; p++) {
        stjump[cur][p] = stjump[stjump[cur][p - 1]][p - 1];
    }
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(to[edge], cur);
        }
    }
}

void buildStjump(int n, int root) {
    power = ceil(log2(n));
    dfs(root, 0);
}

int lca(int a, int b) {
    if (deep[a] < deep[b]) {
        swap(a, b);
    }
    for (int p = power; p >= 0; p--) {
        if (deep[stjump[a][p]] >= deep[b]) {
            a = stjump[a][p];
        }
    }
    if (a == b) {
        return a;
    }
    for (int p = power; p >= 0; p--) {
        if (stjump[a][p] != stjump[b][p]) {
            a = stjump[a][p];
            b = stjump[b][p];
        }
    }
    return stjump[a][0];
}

void solve() {
    int n, q, root;
    cin >> n >> q >> root;
    idx = 0;
    fill(head, head + n + 1, 0);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
}

```

```

    }
    buildStjump(n, root);
    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << lca(a, b) << '\n';
    }
}

```

LcaTarjan 最低公共祖先

```

#include<bits/stdc++.h>
using namespace std;
const int MAXN = 5e5;

//离线处理操作
//离线处理: O(n)
//单次离线查询: O(1)
//整体复杂度: O(n)
//核心代码: dfs+并查集
//模板例题: https://www.luogu.com.cn/problem/P3379

int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int ans[MAXN + 1];
vector<array<int, 2>>query[MAXN + 1];
int fa[MAXN + 1];
bool vis[MAXN + 1];
int find(int u) {
    return fa[u] == u ? u : fa[u] = find(fa[u]);
}

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void tarjan(int cur, int father) {
    // !!! 一旦进入, 立刻标记已抵达
    vis[cur] = true;
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != father) {
            tarjan(to[edge], cur);
        }
    }
}

```

```

        /// !!! 注意 !!!
        /// !!! 设置父节点必须写在子节点递归回溯后 !!!
        fa[to[edge]] = cur;
    }
}
// !!! 子节点遍历完，再查询答案
int a = cur;
for (auto& [b, i] : query[a]) {
    if (vis[b]) {
        ans[i] = find(b);
    }
}
}

void solve() {
    int n, q, root;
    cin >> n >> q >> root;
    idx = 0;
    fill(head, head + n + 1, 0);
    fill(vis, vis + n + 1, 0);
    for (int i = 1; i <= n; i++) {
        fa[i] = i;
    }
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    for (int i = 1; i <= q; i++) {
        int a, b;
        cin >> a >> b;
        query[a].push_back({ b, i });
        query[b].push_back({ a, i });
    }
    tarjan(root, 0);
    for (int i = 1; i <= q; i++) {
        cout << ans[i] << '\n';
    }
}
}

```

树上差分(点和边差分)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

// 树上点和边差分，离线算法

// 根节点需要补充虚拟节点作为父亲
// 给定一棵树和树上节点 u, v(u != v)
// 为 u->v 的路径上点的权值 + val
// 数组 a 是真实点权
// lc = lca(u, v)
// a[u] += val, a[v] += val, a[lc] -= val, a[fa[lc]] -= val

// 为 u->v 的路径上边的权值 + val
// 数组 b 是辅助修改边权的临时点权数组，非真实点权
// lc = lca(u, v)
// b[u] += val, b[v] += val, b[lc] -= 2 * val
// 最后 dfs 遍历一遍树即可

const int MAXN = 5e4;
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
ll a[MAXN + 1];
ll weight[2 * MAXN + 1];
ll b[MAXN + 1];
int idx = 0;
int power = 32;
int deep[MAXN + 1];
int stjump[MAXN + 1][32];

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}

void dfsStjump(int cur, int fa) {
    deep[cur] = deep[fa] + 1;
    stjump[cur][0] = fa;
    for (int p = 1; p <= power; p++) {
        stjump[cur][p] = stjump[stjump[cur][p - 1]][p - 1];
    }
    for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfsStjump(to[edge], cur);
        }
    }
}

```

```

void buildStjump(int n, int root) {
    power = ceil(log2(n));
    deep[0] = 0;
    dfsStjump(root, 0);
}

int lca(int a, int b) {
    if (deep[a] < deep[b]) {
        swap(a, b);
    }
    for (int p = power; p >= 0; p--) {
        if (deep[stjump[a][p]] >= deep[b]) {
            a = stjump[a][p];
        }
    }
    if (a == b) {
        return a;
    }
    for (int p = power; p >= 0; p--) {
        if (stjump[a][p] != stjump[b][p]) {
            a = stjump[a][p];
            b = stjump[b][p];
        }
    }
    return stjump[a][0];
}

//点差分核心代码
void dfsPoint(int u, int fa) {
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfsPoint(v, u);
            a[u] += a[v];
        }
    }
}

void addPoint(int u, int v, ll val) {
    a[u] += val;
    a[v] += val;
    int lc = lca(u, v);
    a[lc] -= val;
    a[stjump[lc][0]] -= val;
}

```

```

//边差分核心代码
void addRoad(int u, int v, ll val) {
    b[u] += val;
    b[v] += val;
    int lc = lca(u, v);
    b[lc] -= 2 * val;
}

void dfsRoad(int u, int fa) {
    for (int v, edge = head[u]; edge > 0; edge = nxt[edge]) {
        v = to[edge];
        if (v != fa) {
            dfsRoad(v, u);
            b[u] += b[v];
            weight[edge] += b[v];
        }
    }
}

```

树的直径(两次 dfs 求解)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 树的直径：树上距离最远的两个点，形成的路径
// 性质：基于边权都  $\geq 0$  (非负)
// 1、若有多条直径，则他们一定存在共有的中间部分(一个点或一段路径)
// 2、对于树上任意点 p，与点 p 相隔最远的点的集合，直径的两端至少有一个在里面

// 树合并,直径边权最短问题:
// 条件：要求所有边权非负且完全相等
// 两颗树，直径上边的数量分别是 A,B,则合并后最长直径=A+B+1,最短
// =max(A,B,A/2+B/2+1),除法分别向上取整

// 解法:
// 1、两次 dfs，可以得到直径、沿途点....等信息
// 条件：边权非负
// 2、树型 dp，只能得到直径
// 条件：适用任何树

const int MAXN = 5e5;
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];

```

```

ll weight[2 * MAXN + 1];
int lst[MAXN + 1]; // 路径的父节点
ll dist[MAXN + 1];
int idx = 0;
int st, ed; ll diameter;
void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
    weight[idx] = w;
}
void initial(int n) {
    idx = 0;
    fill(head, head + n + 1, 0);
}
void dfs(int u, int fa) {
    lst[u] = fa;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dist[to[edge]] = dist[u] + weight[edge];
            dfs(to[edge], u);
        }
    }
}
void GetMaxRoad(int n) {
    dist[1] = 0;
    dfs(1, 0);
    st = 1;
    for (int i = 2; i <= n; i++) {
        if (dist[i] > dist[st]) {
            st = i;
        }
    }
    fill(lst, lst + n + 1, 0);
    dist[st] = 0;
    dfs(st, 0);
    ed = 1;
    for (int i = 2; i <= n; i++) {
        if (dist[i] > dist[ed]) {
            ed = i;
        }
    }
    diameter = dist[ed];
}
void solve() {
    int n;
    cin >> n;

```

```

    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v; ll w;
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    GetMaxRoad(n);
    cout << st << "->" << ed << ' ' << diameter << '\n';
}

```

树的直径(树型 dp 求解)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 树的直径：树上距离最远的两个点，形成的路径
// 性质：基于边权都  $\geq 0$  (非负)
// 1、若有多条直径，则他们一定存在共有的中间部分(一个点或一段路径)
// 2、对于树上任意点  $p$ ，与点  $p$  相隔最远的点的集合，直径的两端至少有一个在里面

// 树合并, 直径边权最短问题:
// 条件：要求所有边权非负且完全相等
// 两颗树，直径上边的数量分别是  $A, B$ ，则合并后最长直径= $A+B+1$ ，最短
//  $=\max(A, B, A/2+B/2+1)$ ，除法分别向上取整

// 解法:
// 1、两次 dfs，可以得到直径、沿途点.... 等信息
// 条件：边权非负
// 2、树型 dp，只能得到直径
// 条件：适用任何树

const ll INF = 1e18;
const int MAXN = 5e5;
int head[MAXN + 1];
int to[2 * MAXN + 1];
int nxt[2 * MAXN + 1];
ll weight[2 * MAXN + 1];
int idx = 0;
ll dist[MAXN + 1]; // 以 u 为头，必须向下走得到的最长路径(单路程)
ll ans[MAXN + 1]; // 必须包含 u(u 为头)得到的最长路径
ll diameter = -INF;

void addEdge(int u, int v, ll w) {
    nxt[++idx] = head[u];
    head[u] = idx;

```

```

        to[idx] = v;
        weight[idx] = w;
    }
    void initial(int n) {
        idx = 0;
        fill(head, head + n + 1, 0);
        fill(ans, ans + n + 1, 0);
        fill(dist, dist + n + 1, 0);
    }
    void dfs(int u, int fa) {
        for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
            int v = to[edge];
            if (v != fa) {
                dfs(v, u);
            }
        }
        for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
            int v = to[edge];
            if (v != fa) {
                // 历史最长    vs    当前新的路径长度(历史单程最长+新路径长度)
                // diameter = max(diameter, dist[u] + dist[v] + weight[edge]);
                此处为简化代码
                ans[u] = max(ans[u], dist[u] + dist[v] + weight[edge]);
                //                获取最长单程路径(向下扎)
                dist[u] = max(dist[u], dist[v] + weight[edge]);
            }
        }
    }
}
void GetMaxRoad(int n) {
    dfs(1, 0);
    for (int i = 1; i <= n; i++) {
        diameter = max(diameter, ans[i]);
    }
}
void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v; ll w;
        cin >> u >> v >> w;
        addEdge(u, v, w);
        addEdge(v, u, w);
    }
    GetMaxRoad(n);
}

```

```

        cout << diameter << '\n';
    }

```

树的重心(利用子树大小求解)

```

#include<bits/stdc++.h>
using namespace std;

// 树的重心: 节点 p
// 1、以节点 p 为根, 所有子树中的最大节点数最小化
// 2、以节点 p 为根, 每个子树的节点数都小于等于总结点数的一半(向下取整)
// 3、以节点 p 为根, 所有节点走向 p 的总边数最少
// 性质:
// 1、重心最多两个, 一定相邻
// 2、若树上增加或删除一个叶节点, 转移后的重心最多移动一条边
// 3、若把两棵树连接, 那么新的重心一定在原先重心的路径上
// 4、若树上边权都 >= 0, 一定存在所有节点走向重心的总距离和最小

//http://poj.org/problem?id=1655
const int MAXN = 2e4;
int best, G;
int siz[MAXN + 1];
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
void initial(int n) {
    best = INT_MAX;
    G = INT_MAX;
    idx = 0;
    fill(siz, siz + n + 1, 0);
    fill(head, head + n + 1, 0);
}
void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}
void dfs(int n, int u, int fa) {
    siz[u] = 1;
    int maxsub = 0;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(n, to[edge], u);
            siz[u] += siz[to[edge]];
            maxsub = max(maxsub, siz[to[edge]]);
        }
    }
}

```

```

    }
}
maxsub = max(maxsub, n - siz[u]);
if (maxsub < best || (maxsub == best && u < G)) {
    G = u;
    best = maxsub;
}
}
void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(n, 1, 0);
    cout << G << ' ' << best << '\n';
}

```

树的重心(利用子树节点数求解)

```

#include<bits/stdc++.h>
using namespace std;

// 树的重心: 节点 p
// 1、以节点 p 为根, 所有子树中的最大节点数最小化
// 2、以节点 p 为根, 每个子树的节点数都小于等于总结点数的一半(向下取整)
// 3、以节点 p 为根, 所有节点走向 p 的总边数最少
// 性质:
// 1、重心最多两个, 一定相邻
// 2、若树上增加或删除一个叶节点, 转移后的重心最多移动一条边
// 3、若把两棵树连接, 那么新的重心一定在原先重心的路径上
// 4、若树上边权都  $\geq 0$ , 一定存在所有节点走向重心的总距离和最小

//http://poj.org/problem?id=3107
const int MAXN = 5e4;
int siz[MAXN + 1];
int head[MAXN + 1];
int nxt[2 * MAXN + 1];
int to[2 * MAXN + 1];
int idx = 0;
int G[3];
int maxsub[MAXN + 1];

```

```

void initial(int n) {
    idx = 0;
    fill(G, G + 3, 0);
    fill(maxsub, maxsub + n + 1, 0);
    fill(siz, siz + n + 1, 0);
    fill(head, head + n + 1, 0);
}

void addEdge(int u, int v) {
    nxt[++idx] = head[u];
    head[u] = idx;
    to[idx] = v;
}

void dfs(int n, int u, int fa) {
    siz[u] = 1;
    for (int edge = head[u]; edge > 0; edge = nxt[edge]) {
        if (to[edge] != fa) {
            dfs(n, to[edge], u);
            siz[u] += siz[to[edge]];
            maxsub[u] = max(maxsub[u], siz[to[edge]]);
        }
    }
    maxsub[u] = max(maxsub[u], n - siz[u]);
}

void solve() {
    int n;
    cin >> n;
    initial(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        addEdge(u, v);
        addEdge(v, u);
    }
    dfs(n, 1, 0);
    for (int i = 1; i <= n; i++) {
        if (maxsub[i] <= n / 2) {
            G[++G[0]] = i;
        }
    }
    for (int i = 1; i <= G[0]; i++) {
        cout << G[i] << ' ';
    }
    cout << '\n';
}

```

3、生成树

Kruskal 最小生成树

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//最小生成树,Kruskal 算法
//根据边权从小到大排序
//若连接当前边不会形成环就连接,否则跳过
//达到 n-1 条边完成
//复杂度取决于边的个数
//点 n,边 m,时间复杂度  $O(m\log n)$ 

//模板例题: https://www.luogu.com.cn/problem/P3366

const int MAXN = 5e3;
const int MAXM = 2e5;

array<int, 3> edge[MAXM + 1];

//并查集
int father[MAXN + 1];
int siz[MAXN + 1];
int road[MAXN + 1];

int find(int u) {
    int cnt = -1;
    while (u != father[u]) {
        road[++cnt] = u;
        u = father[u];
    }
    while (cnt >= 0) {
        father[road[cnt]] = u;
        cnt--;
    }
    return u;
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if (ru == rv) return false;
    if (siz[ru] <= siz[rv]) {
        father[ru] = rv;
        siz[rv] += siz[ru];
    }
```

```

    }
    else {

        father[rv] = ru;
        siz[ru] += siz[rv];
    }
    return true;
}

void initial(int n) {
    //略.....
}

bool cmp(array<int, 3>a, array<int, 3>b) {
    return a[2] < b[2];
}

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//最小生成树,Kruskal 算法
//根据边权从小到大排序
//若连接当前边不会形成环就连接,否则跳过
//达到 n-1 条边完成
//复杂度取决于边的个数
//点 n,边 m,时间复杂度 O(mlogn)

//模板例题: https://www.luogu.com.cn/problem/P3366

const int MAXN = 5e3;
const int MAXM = 2e5;

array<int, 3> edge[MAXM + 1];

//并查集
int father[MAXN + 1];
int siz[MAXN + 1];
int road[MAXN + 1];

int find(int u) {
    int cnt = -1;
    while (u != father[u]) {
        road[++cnt] = u;
        u = father[u];
    }
}

```

```

    while (cnt >= 0) {
        father[road[cnt]] = u;
        cnt--;
    }
    return u;
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if (ru == rv) return false;
    if (siz[ru] <= siz[rv]) {
        father[ru] = rv;
        siz[rv] += siz[ru];
    }
    else {
        father[rv] = ru;
        siz[ru] += siz[rv];
    }
    return true;
}

void initial(int n) {
    //略.....
}

bool cmp(array<int, 3>a, array<int, 3>b) {
    return a[2] < b[2];
}

void solve() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int n, m;
    cin >> n >> m;
    initial(n);
    for (int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edge[i][0] = u;
        edge[i][1] = v;
        edge[i][2] = w;
    }
    sort(edge + 1, edge + m + 1, cmp);
    ll ans = 0;
    ll cnt = 0;
    for (int i = 1; i <= m; i++) {

```

```

        int u = edge[i][0], v = edge[i][1];
        ll w = edge[i][2];
        if (merge(u, v)) {
            ans += w;
            cnt++;
        }
        if (cnt == n - 1) break;
    }
    if (cnt == n - 1) {
        cout << ans << '\n';
    }
    else {
        cout << "orz" << '\n';
    }
}

```

Prim 最小生成树

```

#include<bits/stdc++.h>
using namespace std;

//最小生成树,Prim 算法
//模板例题: https://www.luogu.com.cn/problem/P3366

//Prim 算法, 反向索引堆优化
//时间复杂度, 点 n, 边 m
//优化实现需要手写堆 Heap
//O(n+m)+O((n+m)*logn)
class PrimBetter {
private:
    using ll = long long;

    static const int MAXN = 5e3;
    static const int MAXM = 2e5;

    int head[MAXN + 1];
    int nxt[2 * MAXM + 1];
    int to[2 * MAXM + 1];
    ll w[2 * MAXM + 1];
    int idx = 0;

    bool vis[MAXN + 1];

    void addEdge(int u, int v, ll z) {
        nxt[++idx] = head[u];
        head[u] = idx;
    }

```

```

        to[idx] = v;
        w[idx] = z;
    }

void init(int n) {
    fill(head, head + n + 1, 0);
    fill(vis, vis + n + 1, false);
    fill(f, f + n + 1, -1);
    siz = 0;
}

int f[MAXN + 1]; //堆的反向索引表, 点 i 在堆中的位置
pair<ll, int> heap[MAXN + 1]; //堆:{值, 点}
int siz = 0; //堆的大小

int heapIdx;
ll heapVal;

//上浮
void siftUp(int i) {
    while (i > 0 && heap[i].first < heap[(i - 1) >> 1].first) {
        swap(f[heap[i].second], f[heap[(i - 1) >> 1].second]);
        swap(heap[i], heap[(i - 1) >> 1]);
        i = (i - 1) >> 1;
    }
}

//下沉
void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].first < heap[l].first ? r : l;
        if (heap[best].first >= heap[i].first) {
            return;
        }
        swap(f[heap[best].second], f[heap[i].second]);
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
        r = l + 1;
    }
}

//插入新值
void HeapInsert(ll val, int idx) {
    if (f[idx] != -1) {

```

```

        //已在堆内,执行更新操作
        update(id, val);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].first = val;
    heap[cur].second = id;
    f[id] = cur;
    siftUp(cur);
}

//更新值
void update(int id, ll val) {
    int i = f[id]; //获取该点在堆内的位置
    if (val < heap[i].first) {
        heap[i].first = val;
        siftUp(i);
    }
}

//弹出堆顶
void pop() {
    if (siz == 0) heapIdx = -1;
    swap(f[heap[0].second], f[heap[--siz].second]);
    swap(heap[0], heap[siz]);
    heapIdx = heap[siz].second;
    heapVal = heap[siz].first;
    f[heap[siz].second] = -1; //删除
    siftDown(0);
}

ll Prim(int n) {
    int cnt = 1;
    vis[1] = true;
    ll ans = 0;
    for (int edge = head[1]; edge > 0; edge = nxt[edge]) {
        int son = to[edge];
        ll val = w[edge];
        HeapInsert(val, son);
    }
    while (siz > 0) {
        pop();
        if (vis[heapIdx]) {
            continue;
        }
        ans += heapVal;
    }
}

```

```

        vis[heapIdx] = true; // 标记弹出过, 即在树上
        cnt++;
        for (int edge = head[heapIdx]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            ll val = w[edge];
            if (vis[son]) {
                // 弹出过, 即在树上
                continue;
            }
            HeapInsert(val, son);
        }
    }

    if (cnt != n) return -1;
    return ans;
}

public:

    void solve() {
        // 略.....
    }
};

class PrimNormal {
    // 未优化, 一般时间复杂度
    // 点 n, 边 m,  $O(n+m) + O(m \log m)$ 
private:

    using ll = long long;

    static const int MAXN = 5e3;
    static const int MAXM = 2e5;

    int head[MAXN + 1];
    int nxt[2 * MAXM + 1];
    int to[2 * MAXM + 1];
    ll w[2 * MAXM + 1];
    int idx = 0;

    bool vis[MAXN + 1];

    void addEdge(int u, int v, ll z) {
        nxt[++idx] = head[u];
        head[u] = idx;
        to[idx] = v;
    }

```

```

        w[idx] = z;
    }

    void init(int n) {
        fill(head, head + n + 1, 0);
        fill(vis, vis + n + 1, false);
    }

    ll Prim(int n) {
        int cnt = 1;
        ll ans = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
greater<pair<ll, int>>>q;
        vis[1] = true;
        for (int edge = head[1]; edge > 0; edge = nxt[edge]) {
            int son = to[edge];
            ll val = w[edge];
            q.push({ val, son });
        }
        while (q.size()) {
            auto [val, cur] = q.top();
            q.pop();
            if (!vis[cur]) {
                ans += val;
                cnt++;
                vis[cur] = true;
                for (int edge = head[cur]; edge > 0; edge = nxt[edge]) {
                    int son = to[edge];
                    ll val = w[edge];
                    q.push({ val, son });
                }
            }
        }
        if (cnt < n) ans = -1;
        return ans;
    }

public:
    void solve() {
        //略.....
    }
};

```

拓扑排序

//拓扑排序可以用于环的查询(只能判断环是否存在)

```

//拓扑排序操作完成后
//剩余的度数>0 的点,就是环上点
//核心代码,bfs 时只有入度=0 时才入队

class TopoSort {
private:
    //纯粹的拓朴排序模板代码
    static const int MAXN = 2e5;

    //链式前向星建图
    int idx = 0;
    int head[MAXN + 1];
    int next[2 * MAXN + 1];
    int to[2 * MAXN + 1];
    int indegree[MAXN + 1]; //入度
    int rank[MAXN + 1];
    int cnt = 0;

    void add_edge(int u, int v) {
        next[++idx] = head[u];
        head[u] = idx;
        to[idx] = v;
        indegree[v]++;
    }

    bool TopoSort(int n) {
        //拓扑排序
        //入度删除法,入度为 0 入队,依次删除影响

        //如若需要输出字典序最小的排序
        //把队列更改为小根堆即可
        //priority_queue<int,vector<int>,greater<int>>q;
        queue<int>q;
        for (int i = 1;i <= n;i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        while (q.size()) {
            int cur = q.front(); //q.top()
            q.pop();
            rank[++cnt] = cur;
            for (int edge = head[cur]; edge > 0; edge = next[edge]) {
                int son = to[edge];
            }
        }
    }
};

```

```

        //核心代码,当入度=0时入队
        if (--indegree[son] == 0) {
            q.push(son);
        }
    }

    return cnt == n;
}

void PrintTopo() {
    for (int i = 1; i <= cnt; i++) {
        cout << rank[i] << ' ';
    }
    cout << '\n';
}

void init(int n) {
    fill(head, head + n + 1, 0);
    cnt = idx = 0;
    fill(indegree, indegree + n + 1, 0);
}

public:

    void solve() {
        int n, m;
        cin >> n >> m;
        for (int i = 1; i <= m; i++) {
            int u, v;
            cin >> u >> v;
            add_edge(u, v);
        }
        if (!TopoSort(n)) { //排序失败
            cout << -1 << '\n';
            return;
        }
        PrintTopo();
    }

};

```

六、计算几何

Pick 定理求格点数

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;

// 给定线段(x1, y1), (x2, y2)
// 线段上经过的格点数 = gcd(| x1 - x2 | , | y1 - y2 | ) + 1
// (包括端点)

// 多边形面积: S
// 边上格点数: B
// 内部格点数: I
//  $S = I + B / 2 - 1$ 

// 其中求面积可以用鞋带定理

struct pointll {
    ll x, y;
};
ll crossLL(pointll a, pointll b) {
    return a.x * b.y - a.y * b.x;
}

//  $S = I + B / 2 - 1$ 
// pick 定理获取边界格点数
// 边界格点数 B: 每条边贡献 gcd(|dx|, |dy|) 个点 (只算起点不算终点)
// 注意: 此处用整数二倍运算, 避免精度问题
ll boundaryPoints(const vector<pointll>& p) {
    int n = p.size();
    ll B = 0;
    for (int i = 0; i < n; i++) {
        pointll a = p[i], b = p[(i + 1) % n];
        ll dx = llabs(a.x - b.x);
        ll dy = llabs(a.y - b.y);
        B += std::gcd(dx, dy);
    }
    return B;
}
```

三维几何质心

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point {
    ld x, y, z;
};

point add(point a, point b) {
    return { a.x + b.x, a.y + b.y, a.z + b.z };
}

point mul(point a, ld k) {
    return { a.x * k, a.y * k, a.z * k };
}

point cross(point a, point b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

ld dot(point a, point b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

// 三角形面片
struct Triangle {
    point a, b, c;
};

// 三维多面体质心
// 所有三角形面片，顶点顺序一致（从外部看逆时针）
point polyhedronCentroid(const vector<Triangle>& tris) {
    point C = { 0, 0, 0 };
    ld V = 0;

    for (auto [a, b, c] : tris) {
        ld vol = dot(a, cross(b, c)) / 6.0;    // 有向体积
```

```

        V += vol;

        // 四面体质心 = (0 + a + b + c) / 4
        point ci = mul(add(add(a, b), c), 1.0 / 4.0);
        C = add(C, mul(ci, vol));
    }

    if (fabs(V) < 1e-12) return { 0, 0, 0 };    // 退化
    return mul(C, 1.0 / V);
}

```

三维多边形面积

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 三维:
struct point3 {
    ld x, y, z;
};
point3 cross(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}
point3 add(point3 a, point3 b) {
    return { a.x + b.x, a.y + b.y, a.z + b.z };
}
ld dot(point3 a, point3 b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
ld len(point3 a) {
    return sqrt(dot(a, a));
}
// 三维多边形面积, p 为顶点 (按顺序排列, 且共面)
ld polygonArea3D(vector<point3> p) {
    int n = p.size();
    if (n < 3) return 0;

    point3 N = { 0, 0, 0 };    // 面积法向量累加
    for (int i = 0; i < n; i++) {
        point3 a = p[i];
        point3 b = p[(i + 1) % n];
    }
}

```

```

        N = add(N, cross(a, b));
    }
    return len(N) / 2.0;
}

// 三维空间顶点排序:
// 三维共面点排序 (按逆时针顺序, 基于凸包)
// 前提: 输入点必须共面, 且构成凸多边形 (或求凸包后按凸包顶点返回)
// 输出: 排序好的三维点 (逆时针), 可直接用于 polygonArea3D
vector<point3> sortPoints3D(vector<point3> p) {
    int n = p.size();
    if (n <= 3) return p; // 三角形直接返回

    // 1. 计算多边形所在平面的法向量 (Newell 法)
    point3 N = { 0, 0, 0 };
    for (int i = 0; i < n; i++) {
        point3 a = p[i];
        point3 b = p[(i + 1) % n];
        N = add(N, cross(a, b));
    }

    // 2. 选择投影平面: 忽略法向量中绝对值最大的分量
    int dropAxis;
    if (fabs(N.x) >= fabs(N.y) && fabs(N.x) >= fabs(N.z)) dropAxis = 0;
// 忽略 x
    else if (fabs(N.y) >= fabs(N.z)) dropAxis = 1;
// 忽略 y
    else dropAxis = 2;
// 忽略 z

    // 3. 投影到二维
    vector<pair<ld, ld>> pts2D(n);
    for (int i = 0; i < n; i++) {
        if (dropAxis == 0) pts2D[i] = { p[i].y, p[i].z };
        else if (dropAxis == 1) pts2D[i] = { p[i].x, p[i].z };
        else pts2D[i] = { p[i].x, p[i].y };
    }

    // 4. 在二维上求凸包 (Andrew 算法), 返回点的索引
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(), [&](int i, int j) {
        if (pts2D[i].first != pts2D[j].first) return pts2D[i].first <
pts2D[j].first;

```

```

        return pts2D[i].second < pts2D[j].second;
    });

    vector<int> hullIdx;
    // 下凸包
    for (int i : idx) {
        while (hullIdx.size() >= 2) {
            int m = hullIdx.size();
            auto v1 = make_pair(pts2D[hullIdx[m - 1]].first -
pts2D[hullIdx[m - 2]].first,
                pts2D[hullIdx[m - 1]].second - pts2D[hullIdx[m -
2]].second);
            auto v2 = make_pair(pts2D[i].first - pts2D[hullIdx[m -
2]].first,
                pts2D[i].second - pts2D[hullIdx[m - 2]].second);
            ld cr = v1.first * v2.second - v1.second * v2.first; // 二维叉
乘
            if (cr <= 0) hullIdx.pop_back();
            else break;
        }
        hullIdx.push_back(i);
    }
    // 上凸包
    int lowerSize = hullIdx.size();
    for (int i = n - 2; i >= 0; i--) {
        int idxVal = idx[i];
        while ((int)hullIdx.size() > lowerSize) {
            int m = hullIdx.size();
            auto v1 = make_pair(pts2D[hullIdx[m - 1]].first -
pts2D[hullIdx[m - 2]].first,
                pts2D[hullIdx[m - 1]].second - pts2D[hullIdx[m -
2]].second);
            auto v2 = make_pair(pts2D[idxVal].first - pts2D[hullIdx[m -
2]].first,
                pts2D[idxVal].second - pts2D[hullIdx[m - 2]].second);
            ld cr = v1.first * v2.second - v1.second * v2.first;
            if (cr <= 0) hullIdx.pop_back();
            else break;
        }
        hullIdx.push_back(idxVal);
    }
    hullIdx.pop_back(); // 去掉重复起点

    // 5. 按凸包索引返回三维点
    vector<point3> result;
    for (int i : hullIdx) result.push_back(p[i]);

```

```
    return result;
}
```

二维几何质心

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point { ld x, y; };
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// 返回多边形质心，顶点按边界顺序排列
point polygonCentroid(const vector<point>& p) {
    int n = p.size();
    if (n < 3) return { 0, 0 };    // 退化情况

    ld A = 0, cx = 0, cy = 0;
    for (int i = 0; i < n; i++) {
        point a = p[i];
        point b = p[(i + 1) % n];
        ld cr = cross(a, b);        // 有向面积的两倍
        A += cr;
        cx += (a.x + b.x) * cr;
        cy += (a.y + b.y) * cr;
    }
    A /= 2;                        // 有向面积
    if (fabs(A) < eps) return { 0, 0 }; // 面积为 0，质心不存在

    return { cx / (6 * A), cy / (6 * A) };
}
```

二维多边形面积(鞋带定理)

```
#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;

// 二维
struct point {
    ld x, y;
};
ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
}
```

```

}

// 三角形面积：已知三边（海伦公式）
ld triangleAreaBySides(ld a, ld b, ld c) {
    ld p = (a + b + c) / 2;
    return sqrt(p * (p - a) * (p - b) * (p - c));
}

// 三角形面积：已知顶点坐标
ld triangleAreaByPoints(point A, point B, point C) {
    return fabs(A.x * (B.y - C.y) + B.x * (C.y - A.y) + C.x * (A.y - B.y))
    / 2.0;
}

// 多边形面积：鞋带公式，顶点按顺序(逆时针或顺时针，凸凹均可)
ld polygonArea(const vector<point>& p) {
    int n = p.size();
    if (n < 3) return 0;
    ld area = 0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        // 顺时针：x 后*y 前-x 前*y 后
        // 逆时针：x 前*y 后-x 后*y 前
        area += p[i].x * p[j].y - p[j].x * p[i].y;
    }
    return fabs(area) / 2.0;
}

struct pointll {
    ll x, y;
};

ll crossLL(pointll a, pointll b) {
    return a.x * b.y - a.y * b.x;
}

// S = I + B / 2 - 1
// pick 定理获取边界格点数
// 边界格点数 B：每条边贡献 gcd(|dx|, |dy|) 个点（只算起点不算终点）
// 注意：此处用整数二倍运算，避免精度问题
ll boundaryPoints(const vector<pointll>& p) {
    int n = p.size();
    ll B = 0;
    for (int i = 0; i < n; i++) {
        pointll a = p[i], b = p[(i + 1) % n];
        ll dx = labs(a.x - b.x);

```

```

        ll dy = labs(a.y - b.y);
        B += std::gcd(dx, dy);
    }
    return B;
}

```

凸包(Andrew 算法)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 7、凸包 (Andrew 算法)：凸包就是能框住所有点的最小凸多边形
// 凸包的构建方向从最左侧点逆时针旋转
// 下凸包：从 左->右 扫描，形成凸包的下半部分（从最左边的点出发，经过下面的边界，到最右边的点）
// 上凸包：从 右->左 扫描，形成凸包的上半部分（从最右边的点出发，经过上面的边界，回到最左边的点）
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) > 0, 逆时针左转
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) < 0, 顺时针右转
// cross(st[top] - st[top - 1], p[i] - st[top - 1]) = 0, 三点贡献

struct point { // 二维
    ld x, y;
};
ld dot(point a, point b) {
    // 三维 = x1 * x2 + y1 * y2 + z1 * z2
    return a.x * b.x + a.y * b.y;
}

point cal(point a, point b) {
    return { a.x - b.x,
             a.y - b.y };
}

ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
}

bool cmp(point a, point b) {
    if (a.x == b.x) return a.y < b.y;
    return a.x < b.x;
}

void Andrew(point* st, point* p) {
    // 如果点数 n < 3, 所有点都是凸包
    const int n = 10;
}

```

```

// point st[2 * n]; st 的大小需要开 2 倍
sort(p + 1, p + n + 1, cmp); // x 递增, y 递增排序
// 下凸包
int top = 0;
for (int i = 1; i <= n; i++) {
    while (top >= 2 && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
        top--; // 弹出栈顶
    }
    st[++top] = p[i]; // 当前点入栈
}
// 上凸包: 从 n-1 到 1 反向遍历 (跳过最右边点, 它已在下凸包中)
int tmp = top; // 记录下凸包大小
for (int i = n - 1; i >= 1; i--) {
    while (top > tmp && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
        top--;
    }
    st[++top] = p[i];
}
top--; // 弹出多余 p[1], 维护逆时针顺序, 此时 st 就是凸包顶点
}

```

判断夹角类型

```

#include <bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// 二维给定向量求夹角
struct point { ld x, y; };
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }

// 返回值: 1 = 锐角, 0 = 直角, -1 = 钝角, -2 = 存在零向量无法判断
int angleType(point a, point b) {
    ld la = len(a), lb = len(b);
    if (la < eps || lb < eps) return -2; // 零向量无定义

    ld d = dot(a, b);
    if (d > eps) return 1; // 锐角
    if (d < -eps) return -1; // 钝角
    return 0; // 直角
}

```

```

// 三维
struct point3 { ld x, y, z; };
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }

// 返回值: 1 = 锐角, 0 = 直角, -1 = 钝角, -2 = 存在零向量无法判断
int angleType3D(point3 a, point3 b) {
    ld la = len3(a), lb = len3(b);
    if (la < eps || lb < eps) return -2;

    ld d = dot3(a, b);
    if (d > eps) return 1;
    if (d < -eps) return -1;
    return 0;
}

```

向量夹角

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

//二维给定向量求夹角
struct point { ld x, y; };
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }

// 返回向量 a 和 b 的夹角 (弧度, 范围 [0, π])
ld angleBetween(point a, point b) {
    ld la = len(a), lb = len(b);
    if (la < eps || lb < eps) return 0;    // 零向量无定义, 按需处理

    ld cosv = dot(a, b) / (la * lb);

    // 防止浮点误差导致 cosv 超出 [-1, 1]
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;

    return acos(cosv);
}

// 三维
struct point3 { ld x, y, z; };

```

```

ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }

// 返回向量 a 和 b 的夹角（弧度，范围 [0, π]）
ld angleBetween3D(point3 a, point3 b) {
    ld la = len3(a), lb = len3(b);
    if (la < eps || lb < eps) return 0;    // 零向量无定义

    ld cosv = dot3(a, b) / (la * lb);

    // 防止浮点误差导致 cosv 超出 [-1, 1]
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;

    return acos(cosv);
}

```

圆相关(二维)

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
using ll = long long;
const ld eps = 1e-12;

struct point { ld x, y; };
struct Circle {
    point O;
    ld R;
};

// 三点确定一个圆
// 返回圆，若三点共线则返回一个特殊值（例如半径 -1）
Circle circleFrom3Points(point A, point B, point C) {
    ld ax = A.x, ay = A.y;
    ld bx = B.x, by = B.y;
    ld cx = C.x, cy = C.y;

    ld d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by));
    if (fabs(d) < 1e-12) return { {0,0}, -1 };    // 三点共线，返回半径 -1 表示失败

    ld ux = ((ax * ax + ay * ay) * (by - cy) +
              (bx * bx + by * by) * (cy - ay) +
              (cx * cx + cy * cy) * (ay - by)) / d;

```

```

    ld uy = ((ax * ax + ay * ay) * (cx - bx) +
              (bx * bx + by * by) * (ax - cx) +
              (cx * cx + cy * cy) * (bx - ax)) / d;

    ld R = hypot(ax - ux, ay - uy);
    return { {ux, uy}, R };
}

// 过点 P 和圆 G 的切点

// 需要用到的其他计算函数
point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }
ld dist(point a, point b) { return len(sub(a, b)); }

// 向量逆时针旋转 theta(弧度)
// theta = 角度 * (PI / 180.0)
// PT = acos(-1.0)
point rotate(point v, ld theta) {
    ld c = cos(theta), s = sin(theta);
    return { v.x * c - v.y * s, v.x * s + v.y * c };
}

// 求过点 P 到圆 (C, r) 的切点
// 返回值: 切点个数 (0/1/2), 切点存储在 t 数组
int tangentPoints(point P, point C, ld r, point t[2]) {
    ld d = dist(P, C);

    if (d < r - eps) return 0;           // 点在圆内, 无切线
    if (fabs(d - r) < eps) {             // 点在圆上, 切点就是自己
        t[0] = P;
        return 1;
    }

    point v = sub(P, C);                 // C -> P
    v = mul(v, 1.0 / d);                 // 单位化

    ld theta = acos(r / d);              // 旋转角
    point v1 = rotate(v, theta);
    point v2 = rotate(v, -theta);

    t[0] = add(C, mul(v1, r));           // 切点 1

```

```

    t[1] = add(C, mul(v2, r));          // 切点 2
    return 2;
}

```

平面法向量

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;

// 二维
struct point { ld x, y; };

// 直线 AB 的法向量（不唯一，任取一个）
point normal2D(point A, point B) {
    point v = {B.x - A.x, B.y - A.y};
    return {-v.y, v.x};    // 或 {v.y, -v.x}
}

// 三维
struct point3 { ld x, y, z; };

point3 cross(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

point3 cal(point3 a, point3 b) {    // a - b
    return {a.x - b.x, a.y - b.y, a.z - b.z};
}

// 平面 ABC 的法向量（A,B,C 不共线）
point3 normal3D(point3 A, point3 B, point3 C) {
    return cross(cal(B, A), cal(C, A));
}

```

弧度制角的类型判断及转换

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

```

```

const ld PI = acos(-1.0);
const ld HALF_PI = PI / 2;
const ld DEG_TO_RAD = PI / 180.0;
const ld RAD_TO_DEG = 180.0 / PI;

// 角度制转弧度制
ld degToRad(ld deg) {
    return deg * DEG_TO_RAD;
}

// 弧度制转角度制
ld radToDeg(ld rad) {
    return rad * RAD_TO_DEG;
}

// 根据弧度判断角类型
// 返回 0=零角, 1=锐角, 2=直角, 3=钝角, 4=平角, -1=超出[0,π]
int angleTypeByRad(ld rad) {
    rad = fabs(rad);
    if (rad < eps) return 0;
    if (fabs(rad - HALF_PI) < eps) return 2;
    if (rad < HALF_PI) return 1;
    if (fabs(rad - PI) < eps) return 4;
    if (rad < PI) return 3;
    return -1;
}

// 根据角度制直接判断角类型 (可选)
int angleTypeByDeg(ld deg) {
    return angleTypeByRad(degToRad(deg));
}

```

点乘和叉乘的计算及应用

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 点乘: 就是向量 a 在向量 b 上的 有符号投影长度 * b 的长度
// 性质: 同向最大, 垂直为 0, 反向取反
// 功能:

// 1、判断角度类型(= 0 垂直, > 0 锐角, < 0 钝角)

// 2、判断方向关系

```

```

// 3、A!=B, 给定第三点 P, 判断关系
// 过 A 做线(二维)或平面(三维), 依次为边界, 判断 P 在哪
// dot(B - A, P - A) = 0, P 在过 A 垂直 AB 的线或平面上
// dot(B - A, P - A) > 0, P 在 AB 方向的半平面上或半空间内
// dot(B - A, P - A) < 0, P 在 BA 方向的半平面上或半空间内

// 4、求投影点: 给定点 P, 求 P 在 AB 上的投影点, t 是比例系数
// ld t = dot(P - A, B - A) / dot(B - A, B - A)
// 投影点 = A + t * (B - A)
// t=0 投影点是 A, t=1 投影点是 B, t<0 在 BA 方向上 A 的延长线上, t>1 在 AB 方向上 B
// 的延长线上

// 5、求向量 a 长度: sqrt(dot(a, a))

// 6、求两向量 a, b 的夹角: (C 是夹角)
// cosC = dot(a, b) / (| a | * | b | )
// ld C = acos(dot(a, b) / (| a | * | b | ))

// 7、三维判断面的朝向: dot(面的法向量, 视线方向) < 0, 正面朝向摄像机(即朝向你的
// 眼睛)
// 视线方向: 从眼睛指向面

struct point { ld x, y; };
// 三维 = x1 * x2 + y1 * y2 + z1 * z2
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }

// 叉乘:
// 二维: 结果是一个数, 等于 向量 a 和 向量 b 围成的平行四边形面积(带正负号)
//      正负号表示 b 在 a 的逆时针(+) / 顺时针(-)方向, 0 表示共线
// 三维: 结果是一个向量, 垂直于 向量 a 和 向量 b 所在平面, 长度等于平行四边形面积
//      方向用右手定则: 四指从 向量 a 转向 向量 b, 拇指方向就是结果方向
// 性质: a×b = -b×a; 平行/共线为 0

// 功能(二维叉乘):

// 1、判断顺逆时针方向
//      cross(a, b) > 0, b 在 a 的逆时针方向
//      cross(a, b) < 0, b 在 a 的顺时针方向
//      cross(a, b) == 0, a 和 b 共线

// 2、判断点在线段哪一侧
//      cross(B - A, P - A) > 0, P 在 AB 左侧(逆时针方向)
//      cross(B - A, P - A) < 0, P 在 AB 右侧(顺时针方向)

```

```

//    cross(B - A, P - A) == 0, P 在直线 AB 上

// 3、判断两线段是否相交(跨立实验)
//    线段 AB 和 CD 相交的条件:
//    cross(B - A, C - A) 和 cross(B - A, D - A) 异号 且
//    cross(D - C, A - C) 和 cross(D - C, B - C) 异号
//    (注意边界情况需额外处理, 如端点在线段上, 会误判不相交)

// 4、求三角形面积(三角形 ABC)
// S = | cross(B - A, C - A) / 2 |

// 5、求多边形面积(鞋带公式)
//    area = 0;
//    枚举多边形端点
//    for (int i = 0; i < n; i++)
//        area += cross(p[i], p[(i+1)%n]);
//    area = fabs(area) / 2;

// 6、极角排序(按角度排序)
//    极角: 一个向量和正 x 轴之间的夹角, 逆时针方向算
//    比较两个向量 v1, v2 的角度:
//    cross(v1, v2) > 0, v2 > v1
//    cross(v1, v2) < 0, v1 > v2
//    cross(v1, v2) = 0, 共线, 方向未知
//    dot(v1, v2) > 0 同向相等, < 0 反向 180°

// 7、凸包(Andrew 算法): 凸包就是能框住所有点的最小凸多边形
//    凸包的构建方向从最左侧点逆时针旋转
//    下凸包: 从 左->右 扫描, 形成凸包的下半部分(从最左边的点出发, 经过下面的边界, 到最右边的点)
//    上凸包: 从 右->左 扫描, 形成凸包的上半部分(从最右边的点出发, 经过上面的边界, 回到最左边的点)
//    cross(st[top] - st[top - 1], p[i] - st[top - 1]) > 0, 逆时针左转
//    cross(st[top] - st[top - 1], p[i] - st[top - 1]) < 0, 顺时针右转
//    cross(st[top] - st[top - 1], p[i] - st[top - 1]) = 0, 三点贡献

point cal(point a, point b) {
    return { a.x - b.x,
            a.y - b.y };
}

ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
}

```

```

bool cmp(point a, point b) {}
void Andrew() {
    // 如果点数 n < 3, 所有点都是凸包
    const int n = 10;
    point st[2 * n];
    point p[n];
    sort(p + 1, p + n + 1, cmp); // x 递增, y 递增排序
    // 下凸包
    int top = 0;
    for (int i = 1; i <= n; i++) {
        while (top >= 2 && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--; // 弹出栈顶
        }
        st[++top] = p[i]; // 当前点入栈
    }
    // 上凸包: 从 n-1 到 1 反向遍历 (跳过最右边点, 它已在下凸包中)
    int tmp = top; // 记录下凸包大小
    for (int i = n - 1; i >= 1; i--) {
        while (top > tmp && cross(cal(st[top], st[top - 1]), cal(p[i],
st[top - 1])) <= 0) {
            top--;
        }
        st[++top] = p[i];
    }
    top--; // 弹出多余 p[1], 维护逆时针顺序, 此时 st 就是凸包顶点
}

// 功能 (三维叉乘):

// 1、求平面法向量
//   n = cross(B - A, C - A), 垂直于三角形 ABC 所在平面
//   方向由右手定则确定

// 2、判断点在平面哪一侧
//   dot(cross(B - A, C - A), P - A) > 0, P 在平面一侧 (法向量指向侧)
//   dot(cross(B - A, C - A), P - A) < 0, P 在平面另一侧
//   dot(cross(B - A, C - A), P - A) == 0, P 在平面上

// 3、求三维三角形面积
//   S = |cross(B - A, C - A)| / 2

// 4、结合点乘判断面朝向
//   法向量 n = cross(B - A, C - A), 视线方向 eye -> 面
//   dot(n, eyeDir) < 0 正面朝向摄像机 (见点乘第 7 条)

```

```

struct pointt {
    ld x, y, z;
};
pointt Cross(pointt a, pointt b) {
    return { a.y * b.z - a.z * b.y,
            a.z * b.x - a.x * b.z,
            a.x * b.y - a.y * b.x };
}

```

点线和点面关系判断

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// 二维点线判断

struct point { ld x, y; };
point cal(point a, point b) { return { a.x - b.x, a.y - b.y }; }
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// 返回 >0 表示左侧, <0 表示右侧, 0 在线上
int sideOfLine(point P, point A, point B) {
    ld d = cross(cal(B, A), cal(P, A));
    if (d > eps) return 1;      // 左侧
    if (d < -eps) return -1;    // 右侧
    return 0;                  // 线上
}

// 三维点面判断

struct point3 { ld x, y, z; };
point3 cal3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 cross3(point3 a, point3 b) {
    return { a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x };
}
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

// 返回 >0 表示法向量同侧, <0 表示法向量异侧, 0 在平面上
int sideOfPlane(point3 P, point3 A, point3 B, point3 C) {

```

```

    point3 n = cross3(cal3(B, A), cal3(C, A)); // 平面法向量
    ld d = dot3(n, cal3(P, A));
    if (d > eps) return 1; // 法向量同侧
    if (d < -eps) return -1; // 法向量异侧
    return 0; // 平面上
}

```

点线映射和点面映射

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

//二维，点线映射
struct point { ld x, y; };
point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }

// 点 P 在直线 AB 上的投影点 (A != B)
point projectToLine2D(point P, point A, point B) {
    point v = sub(B, A);
    point w = sub(P, A);
    ld t = dot(w, v) / dot(v, v);
    return add(A, mul(v, t));
}

//三维，点线映射
struct point3 { ld x, y, z; };
point3 add3(point3 a, point3 b) { return { a.x + b.x, a.y + b.y, a.z + b.z }; }
point3 sub3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 mul3(point3 a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

// 点 P 在直线 AB 上的投影点 (A != B)
point3 projectToLine3D(point3 P, point3 A, point3 B) {
    point3 v = sub3(B, A);
    point3 w = sub3(P, A);
    ld t = dot3(w, v) / dot3(v, v);
    return add3(A, mul3(v, t));
}

```

```

//三维，点面映射
point3 cross3(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// 点 P 在平面 ABC 上的投影点 (A, B, C 不共线)
point3 projectToPlane3D(point3 P, point3 A, point3 B, point3 C) {
    point3 n = cross3(sub3(B, A), sub3(C, A));    // 法向量
    point3 w = sub3(P, A);
    ld s = dot3(w, n) / dot3(n, n);
    point3 offset = mul3(n, s);
    return sub3(P, offset);    // P - n * s
}

```

点线距离

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;

// 二维
struct point { ld x, y; };
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
// 点 P 到直线 AB 的距离 (A, B 是直线上两点, A != B)
ld pointLineDist(point P, point A, point B) {
    point v = { B.x - A.x, B.y - A.y };    // 方向向量
    point w = { P.x - A.x, P.y - A.y };    // AP 向量
    return fabs(cross(w, v)) / sqrt(dot(v, v));
}

// 三维
struct point3 { ld x, y, z; };
point3 sub(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 cross(point3 a, point3 b) {    // 三维叉乘
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,

```

```

        a.x * b.y - a.y * b.x
    };
}
ld dot(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len(point3 a) { return sqrt(dot(a, a)); }
// 点 P 到三维直线 AB 的距离, A != B
ld pointLineDist3D(point3 P, point3 A, point3 B) {
    point3 v = sub(B, A);    // 方向向量 B - A
    point3 w = sub(P, A);    // P - A
    point3 c = cross(w, v); // 平行四边形面积向量
    return len(c) / len(v);
}

```

线与线的夹角

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// ----- 二维 -----
struct point {
    ld x, y;
};

point add(point a, point b) { return { a.x + b.x, a.y + b.y }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y }; }
point mul(point a, ld k) { return { a.x * k, a.y * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld len(point a) { return sqrt(dot(a, a)); }
ld cross(point a, point b) { return a.x * b.y - a.y * b.x; }

// 二维两直线夹角（方向向量 v1, v2）

ld angleBetweenLines2D(point v1, point v2) {
    ld la = len(v1), lb = len(v2);
    if (la < eps || lb < eps) return 0;    // 零向量无定义
    ld cosv = dot(v1, v2) / (la * lb);
    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;
    return acos(cosv);
}

// 二维两直线交点

// 直线 1 由 A1,B1 确定, 直线 2 由 A2,B2 确定

```

```

// 返回 0: 正常一个交点 (通过 res 返回)
// 返回 1: 平行无交点
// 返回 2: 共线无穷交点
int intersectLines2D(point A1, point B1, point A2, point B2, point& res) {
    point v1 = sub(B1, A1);
    point v2 = sub(B2, A2);
    ld d = cross(v1, v2);

    if (fabs(d) < eps) {
        // 平行或共线
        if (fabs(cross(sub(A2, A1), v1)) < eps) return 2; // 共线
        return 1; // 平行无交点
    }

    point w = sub(A2, A1);
    ld t = cross(w, v2) / d;
    res = add(A1, mul(v1, t));
    return 0;
}

// ----- 三维 -----
struct point3 {
    ld x, y, z;
};

point3 add3(point3 a, point3 b) { return { a.x + b.x, a.y + b.y, a.z + b.z }; }
point3 sub3(point3 a, point3 b) { return { a.x - b.x, a.y - b.y, a.z - b.z }; }
point3 mul3(point3 a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot3(point3 a, point3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len3(point3 a) { return sqrt(dot3(a, a)); }
point3 cross3(point3 a, point3 b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// 三维两直线夹角 (方向向量 v1, v2)

ld angleBetweenLines3D(point3 v1, point3 v2) {
    ld la = len3(v1), lb = len3(v2);
    if (la < eps || lb < eps) return 0;
    ld cosv = dot3(v1, v2) / (la * lb);
}

```

```

    if (cosv > 1) cosv = 1;
    if (cosv < -1) cosv = -1;
    return acos(cosv);
}

// 三维两直线最短距离(公垂线段)及最近点对

// 直线 1 由 A1,B1 确定, 直线 2 由 A2,B2 确定
// 返回最短距离, 最近点通过 p1,p2 返回
ld distanceLines3D(point3 A1, point3 B1, point3 A2, point3 B2, point3& p1,
point3& p2) {
    point3 v1 = sub3(B1, A1);
    point3 v2 = sub3(B2, A2);
    point3 w = sub3(A2, A1);

    point3 n = cross3(v1, v2);
    ld nlen = len3(n);

    if (nlen > eps) {
        // 异面直线: 公垂线法
        ld s = dot3(cross3(w, v2), n) / (nlen * nlen);
        ld t = dot3(cross3(w, v1), n) / (nlen * nlen);

        p1 = add3(A1, mul3(v1, s));
        p2 = add3(A2, mul3(v2, t));
        return len3(sub3(p1, p2));
    }
    else {
        // 两直线平行: 取 A2 到直线 1 的投影
        p1 = A2;
        point3 v = v1;
        point3 ap = sub3(A2, A1);
        ld t = dot3(ap, v) / dot3(v, v);
        p2 = add3(A1, mul3(v, t));
        return len3(sub3(p1, p2));
    }
}

```

线面夹角和映射

```

#include<bits/stdc++.h>
using namespace std;
using ld = long double;
const ld eps = 1e-12;

// ----- 三维基础 -----
struct point { ld x, y, z; };

```

```

point add(point a, point b) { return { a.x + b.x, a.y + b.y, a.z +
b.z }; }
point sub(point a, point b) { return { a.x - b.x, a.y - b.y, a.z -
b.z }; }
point mul(point a, ld k) { return { a.x * k, a.y * k, a.z * k }; }
ld dot(point a, point b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
ld len(point a) { return sqrt(dot(a, a)); }
point cross(point a, point b) {
    return {
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x
    };
}

// ----- 线面夹角 -----
// 直线方向向量 d, 平面法向量 n
// 返回线面夹角 (弧度), 范围 [0,  $\pi/2$ ]
ld angleLinePlane(point d, point n) {
    ld ldv = len(d), lnv = len(n);
    if (ldv < eps || lnv < eps) return 0;    // 零向量无定义

    ld sinv = fabs(dot(d, n)) / (ldv * lnv);
    if (sinv > 1) sinv = 1;
    return asin(sinv);
}

// ----- 平面法向量 (由三点确定) -----
point normalOfPlane(point A, point B, point C) {
    return cross(sub(B, A), sub(C, A));
}

// ----- 点到平面投影 -----
// 点 P 到平面 (过点 A, 法向量 n) 的投影点
point projectPointToPlane(point P, point A, point n) {
    point w = sub(P, A);
    ld s = dot(w, n) / dot(n, n);
    return sub(P, mul(n, s));
}

// ----- 向量到平面投影 -----
// 将向量 v 投影到法向量为 n 的平面上, 得到平面内的分量
point projectVectorToPlane(point v, point n) {
    ld s = dot(v, n) / dot(n, n);
    return sub(v, mul(n, s));
}

```

```

// ----- 直线到平面投影 -----
// 直线 L: 过点 P0, 方向向量 d
// 平面  $\pi$ : 过点 A, 法向量 n
// 投影直线 L' 过点 P0_proj, 方向向量 d_proj
// 注意: 若 d_proj 接近零向量, 说明原直线垂直于平面, 投影退化为一个点
void projectLineToPlane(point P0, point d, point A, point n,
    point& P0_proj, point& d_proj) {
    P0_proj = projectPointToPlane(P0, A, n);
    d_proj = projectVectorToPlane(d, n);
}

```

七、数据结构

1、前缀树

普通前缀树

```

#include<bits/stdc++.h>

//前缀树(字典树),根据前缀字符建树
//根据前缀信息查询
//时间复杂度低,空间复杂度略高
//模板例题:
https://www.nowcoder.com/practice/7f8a8553ddb4eaab749ec988726702b

using std::string;
using std::fill;

const int MAXN = 1e6;
int tree[MAXN + 1][26]; //'a'~'z', 0~25
int pass[MAXN + 1]; //多少个字符串经过该点
int end[MAXN + 1]; //多少个字符串以该点结尾
int cnt = 1; //节点编号, 申请和撤销空间

void build() {
    cnt = 1; //1 作为头节点
}

void insert(string word) {
    int cur = 1; //cur 初始化 1, 头节点
    pass[cur]++;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            tree[cur][path] = ++cnt;

```

```

        }
        cur = tree[cur][path];
        pass[cur]++;
    }
    end[cur]++;
}

int search(string word) {
    int cur = 1;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return end[cur];
}

int countPrefix(string word) {
    int cur = 1;
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (tree[cur][path] == 0) {
            return 0;
        }
        cur = tree[cur][path];
    }
    return pass[cur];
}

void erase(string word) {
    if (search(word) < 1) return;
    int cur = 1;
    //沿途 pass--
    //终点 end--
    for (int i = 0; i < word.size(); i++) {
        int path = word[i] - 'a';
        if (--pass[tree[cur][path]] == 0) {
            //全部删除完毕,清除节点编号
            tree[cur][path] = 0;
            return;
        }
        cur = tree[cur][path];
    }
    end[cur]--;
}

```

```

}

void clear() {
    for (int i = 1; i <= cnt; i++) {
        fill(tree[i], tree[i] + 26, 0);
        pass[i] = end[i] = 0;
    }
}

```

可持久化前缀树

```

#include<iostream>
using namespace std;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    return 0;
}

```

2、手写堆

手写反向索引堆优化

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

//通过建立映射数组 f[val]=i
//获取值 val 在对内的映射位置 i
//如下模板代码维护小根堆，且仅在插入值不存在或更小时执行操作
//若需要普遍性更新，需要在变大时执行 siftDown 操作
const int MAXN = 2e5;
int f[MAXN + 1]; //堆的反向索引表, 点 i 在堆中的位置
pair<ll, int> heap[MAXN + 1]; //堆:{值, 点}
int siz = 0; //堆的大小

int heapIdx;
ll heapVal;

//上浮
void siftUp(int i) {
    while (i > 0 && heap[i].first < heap[(i - 1) >> 1].first) {
        swap(f[heap[i].second], f[heap[(i - 1) >> 1].second]);
        swap(heap[i], heap[(i - 1) >> 1]);
        i = (i - 1) >> 1;
    }
}

```

```

    }
}

//下沉
void siftDown(int i) {
    int l = 2 * i + 1, r = l + 1;
    while (l < siz) {
        int best = r < siz && heap[r].first < heap[l].first ? r : l;
        if (heap[best].first >= heap[i].first) {
            return;
        }
        swap(f[heap[best].second], f[heap[i].second]);
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
        r = l + 1;
    }
}

//插入新值, 该函数同时实现插入、更新、忽略的判断
void HeapInsert(ll val, int idx) {
    if (f[idx] != -1) {
        //已在堆内, 执行更新操作
        update(idx, val);
        return;
    }
    int cur = ++siz - 1;
    heap[cur].first = val;
    heap[cur].second = idx;
    f[idx] = cur;
    siftUp(cur);
}

//更新值
void update(int idx, ll val) {
    int i = f[idx]; //获取该点在堆内的位置
    if (val < heap[i].first) {
        heap[i].first = val;
        siftUp(i);
    }
}

//弹出堆顶
void pop() {
    if (siz == 0) heapIdx = -1;
    swap(f[heap[0].second], f[heap[--siz].second]);
}

```

```

        swap(heap[0], heap[siz]);
        heapIdx = heap[siz].second;
        heapVal = heap[siz].first;
        f[heap[siz].second] = -1;//删除
        siftDown(0);
    }

```

手写堆

```

#include<bits/stdc++.h>

using namespace std;
using ll = long long;
const int MAXN = 2e5;

// 手写堆可以通过反向索引优化
// 实现非头部数据的修改删除操作
// 详见 PrimMst 算法的优化结合

// 自底向顶建堆的优点在于离线操作建队的复杂度从  $O(n\log n)$  ->  $O(n)$ 
// 动态插入的操作依旧是  $O(\log n)$ 

// 堆的结构基于二叉树,如下
//      0
//    1 2
//  3 4 5 6

class MaxHeap_nlogn {
public:
    //自顶向底建堆
    //复杂度  $O(n\log n)$ 
    int heap[MAXN + 1];
    int siz = 0;//堆的大小

    //大根堆,动态插入
    void heapInsert(int val) {
        int cur = ++siz - 1;//顶点下标=0,所以 siz-1 即当前节点
        heap[cur] = val;
        //当前节点 cur,父节点  $(cur-1)>>1$ 
        while (cur > 0 && heap[cur] > heap[(cur - 1) >> 1]) {
            swap(heap[cur], heap[(cur - 1) >> 1]);
            cur = (cur - 1) >> 1;
        }
    }

    //siftUp 和 siftDown 用于更复杂的堆功能实现

```

```

void siftDown(int i) { //小值下传
    int l = 2 * i + 1;
    while (l < siz) {
        //大根堆,和左右孩子数值最大的交换,且必须大于当前节点值
        int best = l + 1 < siz && heap[l + 1] > heap[l] ? l + 1 : l;
        if (heap[best] <= heap[i]) return;
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
    }
}

void siftUp(int i) { //大值上传
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
        swap(heap[i], heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

int pop() { //出队
    if (siz == 0) return -1;
    //头部 0 和末尾交换位置,丢弃末尾
    swap(heap[0], heap[--siz]);
    siftDown(0);
    //siz 现在就是原先的丢弃位置
    return heap[siz];
}

int top() { //获取头部
    if (siz == 0) return -1;
    return heap[0];
}

};

class MinHeap_n {
public:
    //小根堆
    //自底向顶建堆
    //复杂度 O(n)
    int heap[MAXN + 1];
    int siz = 0;

    //读入数据

```

```

void getArr(vector<int>& nums) {
    for (int i = 0; i < nums.size(); i++) {
        heap[i] = nums[i];
    }
    siz = nums.size();
}

void siftDown(int i) {
    int l = 2 * i + 1;
    while (l < siz) {
        int best = l + 1 < siz && heap[l + 1] < heap[l] ? l + 1 : l;
        if (heap[best] >= heap[i]) return;
        swap(heap[i], heap[best]);
        i = best;
        l = 2 * i + 1;
    }
}

//建堆
void heapInsert() {
    for (int i = (siz - 2) / 2; i >= 0; i--) {
        siftDown(i);
    }
}

int pop() {
    if (siz == 0) return -1;
    //头部 0 和末尾交换位置,丢弃末尾
    swap(heap[0], heap[--siz]);
    siftDown(0);
    //siz 现在就是原先的丢弃位置
    return heap[siz];
}

};

```

3、左偏树

K 短路问题

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ld = long double;

const int MAXN = 5e3;
const int MAXM = 2e5;

```

```

const int MAXT = 2e6;

int headGo[MAXM + 1];
int nxtGo[MAXM + 1];
int toGo[MAXM + 1];
int idxGo = 0;

int headBack[MAXM + 1];
int nxtBack[MAXM + 1];
int toBack[MAXM + 1];
int idxBack = 0;

int path[MAXN + 1];

ld weight[MAXM + 1];

bool vis[MAXN + 1];
ld dist[MAXN + 1];

struct {
    int left;
    int right;
    int dist;
    ld addVal;
    int to;
} node[MAXT + 1];

int cnt = 0;
int top[MAXM + 1];

int create(ld addVal, int to) {
    cnt++;
    node[cnt].addVal = addVal;
    node[cnt].to = to;
    node[cnt].left = node[cnt].right = node[cnt].dist = 0;
    return cnt;
}

int clone(int i) {
    cnt++;
    node[cnt] = node[i];
    return cnt;
}

void addEdgeGo(int u, int v, ld w) {
    nxtGo[++idxGo] = headGo[u];
    headGo[u] = idxGo;

```

```

        toGo[idxGo] = v;
        weight[idxGo] = w;
    }

void addEdgeBack(int u, int v, ld w) {
    nxtBack[++idxBack] = headBack[u];
    headBack[u] = idxBack;
    toBack[idxBack] = v;
    weight[idxBack] = w;
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    if (node[i].addVal > node[j].addVal) {
        swap(i, j);
    }
    int p = clone(i);
    node[p].right = merge(node[p].right, j);
    if (node[node[p].left].dist < node[node[p].right].dist) {
        swap(node[p].left, node[p].right);
    }
    node[p].dist = node[node[p].right].dist + 1;
    return p;
}

int pop(int i) {
    if (node[i].left == 0 && node[i].right == 0) {
        return 0;
    }
    if (node[i].left == 0 || node[i].right == 0) {
        return clone(node[i].left + node[i].right);
    }
    return merge(node[i].left, node[i].right);
}

// 反图上跑最短路算法
void Djk(int n) {
    priority_queue<pair<ld, int>, vector<pair<ld, int>>, greater<pair<ld,
int>>>q;
    fill(vis, vis + n + 1, false);
    fill(dist, dist + n + 1, 1e18);
    dist[n] = 0;
    q.push({ 0, n });
    while (q.size()) {
        auto [w, u] = q.top();

```

```

        q.pop();
        if (!vis[u]) {
            vis[u] = true;
            for (int edge = headBack[u]; edge > 0; edge = nxtBack[edge]) {
                int v = toBack[edge];
                if (dist[v] > dist[u] + weight[edge]) {
                    dist[v] = dist[u] + weight[edge];
                    path[v] = edge;
                    q.push({ dist[v], v });
                }
            }
        }
    }
}

void mergeRoad(int n) {
    priority_queue<pair<ld, int>, vector<pair<ld, int>>, greater<pair<ld,
int>>>>q;
    for (int i = 1; i <= n; i++) {
        q.push({ dist[i], i });
    }
    while (q.size()) {
        auto [w, u] = q.top();
        q.pop();
        for (int edge = headGo[u]; edge > 0; edge = nxtGo[edge]) {
            int v = toGo[edge];
            if (edge != path[u]) {
                // 非树边
                if (top[u] == 0) {
                    top[u] = create(dist[v] + weight[edge] - dist[u], v);
                }
                else {
                    top[u] = merge(top[u], create(dist[v] + weight[edge] -
dist[u], v));
                }
            }
        }
        if (path[u] != 0) {
            // 存在父亲
            int v = toGo[path[u]];
            top[u] = merge(top[u], top[v]);
        }
    }
}

```

```

struct heapNode {
    ld addVal;
    int to;
    int heapRoot;
};

int compute(int n, ld money) {
    auto cmp = [](heapNode a, heapNode b) { return a.addVal > b.addVal; };
    priority_queue<heapNode, vector<heapNode>, decltype(cmp)>q(cmp);

    int ans = 0;
    if (money >= dist[1]) {
        ans++;
        money -= dist[1];
    }
    else {
        return ans;
    }

    q.push({ node[top[1]].addVal,node[top[1]].to,top[1] });
    while (q.size()) {
        auto [addVal, to, heapRoot] = q.top();
        q.pop();

        if (dist[1] + addVal <= money) {
            money -= (dist[1] + addVal);
            ans++;
        }
        else {
            return ans;
        }

        // 替换影响, 方式: 直接选择左右孩子, 不要弹出堆顶, 否则 MLE
        // 左偏树满足: 每个节点的孩子的 key 值都小于父亲
        int ls = node[heapRoot].left;
        int rs = node[heapRoot].right;
        if (ls != 0) {
            ld newAddVal = addVal - node[heapRoot].addVal +
node[ls].addVal;
            q.push({ newAddVal,node[ls].to,ls });
        }

        if (rs != 0) {
            ld newAddVal = addVal - node[heapRoot].addVal +
node[rs].addVal;
            q.push({ newAddVal,node[rs].to,rs });
        }
    }
}

```

```

    }

    // 弹出堆顶，拷贝，会导致!!!MLE!!!
    // int newHead = pop(heapRoot);
    // if (newHead != 0) {
    //     ld newAddVal = addVal - node[heapRoot].addVal +
node[newHead].addVal;
    //     q.push({ newAddVal,node[newHead].to,newHead });
    // }

    // 新增影响

    int newHead = top[to];
    if (newHead != 0) {
        ld newAddVal = addVal + node[newHead].addVal;
        q.push({ newAddVal,node[newHead].to,newHead });
    }

}
return ans;
}

void solve() {

    // https://www.luogu.com.cn/problem/P2483

    // 核心: djik 算法生成的最短路树
    // 对于非树边，一定是连接两个树上节点的边
    // 对于边(u, v)，若选择这条边额外耗费的代价是 dist[v] + weight(u, v) -
dist[u]
    // 使用可持久化左偏树，逐点逐级递推

    int n, m;
    ld money;
    cnt = idxGo = idxBack = 0;
    cin >> n >> m >> money;
    node[0].dist = -1;
    fill(top, top + n + 1, 0);
    fill(path, path + n + 1, 0);
    fill(headGo, headGo + n + 1, 0);
    fill(headBack, headBack + n + 1, 0);
    for (int i = 1; i <= m; i++) {
        int u, v;
        ld w;
        cin >> u >> v >> w;
        // 抵达终点即截止，因此不应有任何边的连接

```

```

        // 避免额外计算
        if (u != n) {
            addEdgeGo(u, v, w);
            addEdgeBack(v, u, w);
        }
    }
    Djk(n);
    mergeRoad(n);
    cout << compute(n, money) << '\n';
}

```

可持久化左偏树

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;
const int MAXT = 4e5;
int cnt = 0;
int version[MAXN + 1];
int verSize[MAXN + 1];
struct leftTree {
    ll key;
    int left;
    int right;
    int dist;
    int size;
}node[MAXT + 1];

void cleanNode(int i) {
    node[i].key = 0;
    node[i].left = node[i].right = node[i].dist = 0;
    node[i].size = 1;
}

void clear(int n) {
    for (int i = 0; i <= cnt; i++) {
        cleanNode(i);
    }
    fill(version, version + n + 1, 0);
    fill(verSize, verSize + n + 1, 0);
    node[0].dist = -1;
    node[0].size = 0;
    cnt = 0;
}

```

```

int clone(int i) {
    int j = ++cnt;
    node[j] = node[i];
    return j;
}

int create(ll val) {
    int i = ++cnt;
    node[i].key = val;
    node[i].left = node[i].right = node[i].dist = 0;
    node[i].size = 1;
    return i;
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    if (node[i].key > node[j].key) {
        swap(i, j);
    }
    int p = clone(i);
    node[p].right = merge(node[p].right, j);
    if (node[node[p].left].dist < node[node[p].right].dist) {
        swap(node[p].left, node[p].right);
    }
    node[p].dist = node[node[p].right].dist + 1;
    node[p].size = node[node[p].left].size + node[node[p].right].size + 1;
    return p;
}

int pop(int i) {
    if (node[i].left == 0 && node[i].right == 0) {
        return 0;
    }
    if (node[i].left == 0 || node[i].right == 0) {
        return clone(node[i].left + node[i].right);
    }
    return merge(node[i].left, node[i].right);
}

// 基于 x 版本插入 v
void treeAdd(int x, ll v, int i) {
    if (version[x] == 0) {
        version[i] = create(v);
        verSize[i] = 1;
    }
}

```

```

    }
    else {
        version[i] = merge(version[x], create(v));
        verSize[i] = verSize[x] + 1;
    }
}

// 基于 x 和 y 版本进行合并
void treeMerge(int x, int y, int i) {
    int v1 = version[x];
    int v2 = version[y];
    if (v1 == 0 && v2 == 0) {
        version[i] = 0;
        verSize[i] = 0;
        return;
    }
    if (v1 == 0 || v2 == 0) {
        version[i] = clone(v1 + v2);
        verSize[i] = verSize[x] + verSize[y];
        return;
    }
    version[i] = merge(v1, v2);
    verSize[i] = verSize[x] + verSize[y];
}

// 基于 x 版本弹出队头
void treePop(int x, int i) {
    if (version[x] != 0) {
        version[i] = pop(version[x]);
        verSize[i] = verSize[x] - 1;
    }
    else {
        version[i] = 0;
        verSize[i] = 0;
    }
}

```

左偏树

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e5;

// 左偏树：一种可并堆结构
// 堆的合并时间复杂度  $O(\log n)$ 

```

```

// 左偏树依赖 dist 值维护平衡性
// dist[i]: 从 i 出发到达空节点经过的最少节点数
// 左偏树的每个节点满足左儿子的 dist >= 右儿子的 dist
// 节点 dist = 右儿子 dist + 1

struct tree {
    ll val;
    int left, right;
    int dist;
    int father;// 辅助进行 remove 删除操作
}node[MAXN + 1];

int fa[MAXN + 1];

void prepare(int n) {
    for (int i = 0; i <= n; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
        fa[i] = i;
    }
    node[0].dist = -1;
}

int find(int u) {
    return u == fa[u] ? u : fa[u] = find(fa[u]);
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    // 维护小根堆
    // 让 i 成为值小的, 若值相等成为下标小的
    if (node[i].val > node[j].val || (node[i].val == node[j].val && i >
j)) {
        swap(i, j);
    }
    node[i].right = merge(node[i].right, j);
    node[node[i].right].father = i;
    if (node[node[i].left].dist < node[node[i].right].dist) {
        swap(node[i].left, node[i].right);
    }
    node[i].dist = node[node[i].right].dist + 1;
    fa[node[i].left] = i;
    fa[node[i].right] = i;
    return i;
}

```

```

// 返回新树的头节点
int pop(int i) {
    fa[node[i].left] = node[i].left;
    fa[node[i].right] = node[i].right;
    int newHead = merge(node[i].left, node[i].right);
    fa[i] = newHead;
    node[i].left = node[i].right = node[i].dist = node[i].father = 0;
    return newHead;
}

int remove(int i) {
    int head = find(i);
    int f = node[i].father;
    int s = pop(i);
    node[s].father = f;

    if (head != i) {
        fa[s] = head;
        if (node[f].left == i) {
            node[f].left = s;
        }
        else {
            node[f].right = s;
        }
        for (int d = node[s].dist; node[f].dist > d + 1; f = node[f].father,
d++) {
            node[f].dist = d + 1;
            if (node[node[f].left].dist < node[node[f].right].dist) {
                swap(node[f].left, node[f].right);
            }
        }
    }
    // 若 i 就是 head, 则 = 0, fa[s] = 0
    // 若 i 不是 head, 则 s 存在, 更新
    return fa[s];
}

// 把 i 调整为 to
void update(int i, ll to) {
    int h = remove(i);
    node[i].val = to;
    fa[i] = i;
    node[i].dist = 0;
    fa[h] = fa[i] = merge(h, i);
}

```

4、并查集

带权并查集

```
#include<bits/stdc++.h>
using namespace std;

using ll = long long;
// https://www.luogu.com.cn/problem/P8779

const int MAXN = 2e5;
int fa[MAXN + 1];
ll dist[MAXN + 1];

void init(int n) {
    for (int i = 0; i <= n; i++) {
        fa[i] = i;
        dist[i] = 0;
    }
}

int find(int u) {
    if (u != fa[u]) {
        int tmp = fa[u];
        fa[u] = find(tmp);
        dist[u] += dist[tmp];
    }
    return fa[u];
}

void merge(int u, int v, ll d) {
    int ru = find(u);
    int rv = find(v);
    if (ru != rv) {
        fa[ru] = rv;
        // u -> ru
        // v -> rv
        // 转变成: ru -> u -> v -> rv
        dist[ru] = d + dist[v] - dist[u];
    }
}

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    init(n + 1);
    for (int i = 1; i <= m; i++) {
```

```

        int l, r;ll d;
        cin >> l >> r >> d;
        r = r + 1;
        merge(l, r, d);
    }
    while (q--){
        int l, r;
        cin >> l >> r;
        r = r + 1;
        if (find(l) != find(r)) {
            cout << "UNKNOWN" << '\n';
        }
        else {
            cout << dist[l] - dist[r] << '\n';
        }
    }
}

```

并查集

```

#include<iostream>
using namespace std;
const int MAXN = 2e5;

//并查集
int father[MAXN + 1];
int siz[MAXN + 1];
int road[MAXN + 1];//辅助实现路径优化

//初始化
void initial(int m) {
    for (int i = 1;i <= m;i++) {
        father[i] = i;
        siz[i] = 1;
    }
}

//查找根节点
int find(int u) {
    //迭代+扁平化初始
    //即有递归的路径优化,又避免递归过深
    int cnt = -1;
    while (father[u] != u) {
        road[++cnt] = u;
        u = father[u];
    }
    //路径优化
}

```

```

    while (cnt >= 0) father[road[cnt--]] = u;
    return u;
}

//按秩合并,避免退化
void merge(int u, int v) {
    int ru = find(u);
    int rv = find(v);
    if (ru == rv) return;
    if (siz[ru] >= siz[rv]) {
        siz[ru] += siz[rv];
        father[rv] = ru;
    }
    else {
        siz[rv] += siz[ru];
        father[ru] = rv;
    }
}
}

```

5、有序表专题

AVL 平衡树

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// AVL: 平衡搜索二叉树
// 每个节点的左右子树高度之差 <= 1

// 有序表:
// 可实现: 增, 删, 改, 查
// 单次时间复杂度  $O(\log n)$ 
// 每个 key(键)映射唯一 value(值)
// 根据 key 的序组织, key 不能重复(唯一)

// https://www.luogu.com.cn/problem/P3369

const int MAXN = 2e5;
const ll INF = 1e18;
int head = 0;
int cnt = 0;
// 可以使用 use 表维护可用内存编号, 避免内存浪费
// canUse[0]表示可用节点数量
// int canUse[MAXN + 1];
struct {

```

```

    ll key;// 键
    int height;// 树高
    int left;// 左儿子
    int right;// 右儿子
    int cnt;// key 词频
    int siz;// 子树大小
}node[MAXN + 1];

void clear() {
    for (int i = 1;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0 };
    }
    cnt = 0;
    head = 0;
}

// 汇总信息
void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].siz = node[l].siz + node[r].siz + node[i].cnt;
    node[i].height = max(node[l].height, node[r].height) + 1;
}

// 左旋, 父节点向左下沉
// 右孩子的左儿子成为父亲的右孩子
// 父亲成为右孩子的左儿子
int leftRotate(int i) {
    int r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;
    up(i);
    up(r);
    return r;
}

// 右旋
int rightRotate(int i) {
    int l = node[i].left;
    node[i].left = node[l].right;
    node[l].right = i;
    up(i);
    up(l);
    return l;
}

```

```

// 检查子树是否违规并调整
int maintain(int i) {
    int l = node[i].left, r = node[i].right;
    int lh = node[l].height;
    int rh = node[r].height;
    if (lh - rh > 1) {
        // LL
        if (node[node[l].left].height >= node[node[l].right].height) {
            i = rightRotate(i);
        }
        else {
            // LR
            node[i].left = leftRotate(l);
            i = rightRotate(i);
        }
    }
    else if (rh - lh > 1) {
        // RR
        if (node[node[r].right].height >= node[node[r].left].height) {
            i = leftRotate(i);
        }
        else {
            // RL
            node[i].right = rightRotate(r);
            i = leftRotate(i);
        }
    }
    return i;
}

// 查询 < val 的 key 个数
int small(int i, int val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].siz + node[i].cnt + small(node[i].right,
val);
    }
}

```

```

// 查询排名
int rank(int val) {
    return small(head, val) + 1;
}

// 查询子树 i 上第 k 小的数
ll index(int i, int k) {
    int l = node[i].left;
    if (node[l].siz >= k) {
        return index(l, k);
    }
    else if (node[l].siz + node[i].cnt < k) {
        return index(node[i].right, k - node[l].siz - node[i].cnt);
    }
    return node[i].key;
}

ll index(int k) {
    return index(head, k);
}

// 查找子树 i 上 val 的前驱(< val 的最大值)
ll pre(int i, int val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll pre(int val) {
    return pre(head, val);
}

ll suf(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return suf(node[i].right, val);
    }
}

```

```

    else {
        return min(node[i].key, suf(node[i].left, val));
    }
}

// 向子树 i 添加值 val
int add(int i, int val) {
    if (i == 0) {
        node[++cnt].key = val;
        node[cnt].cnt = node[cnt].siz = node[cnt].height = 1;
        return cnt;
    }
    if (node[i].key == val) {
        node[i].cnt++;
    }
    else if (node[i].key > val) {
        node[i].left = add(node[i].left, val);
    }
    else {
        node[i].right = add(node[i].right, val);
    }
    up(i);
    return maintain(i);
}

// 向整颗树添加值 val, 返回新的头
void add(int val) {
    head = add(head, val);
}

// 删除子树 i 上的 low 节点
int removeLow(int i, int low) {
    if (i == low) {
        return node[i].right;
    }
    else {
        node[i].left = removeLow(node[i].left, low);
        up(i);
        return maintain(i);
    }
}

```

```

// 删除子树 i 上的 val
int remove(int i, int val) {
    if (node[i].key < val) {
        node[i].right = remove(node[i].right, val);
    }
    else if (node[i].key > val) {
        node[i].left = remove(node[i].left, val);
    }
    else {
        if (node[i].cnt > 1) {
            node[i].cnt--;
        }
        else {
            if (node[i].left == 0 && node[i].right == 0) {
                return 0;
            }
            else if (node[i].left == 0 && node[i].right != 0) {
                i = node[i].right;
            }
            else if (node[i].left != 0 && node[i].right == 0) {
                i = node[i].left;
            }
            else {
                int low = node[i].right;
                while (node[low].left != 0) {
                    low = node[low].left;
                }
                node[i].right = removeLow(node[i].right, low);
                node[low].left = node[i].left;
                node[low].right = node[i].right;
                i = low;
            }
        }
    }
    up(i);
    return maintain(i);
}

// 单次删除整棵树上的 key
void remove(int val) {
    if (::rank(val) != ::rank(val + 1)) {
        head = remove(head, val);
    }
}

```

FHQ 可持久化

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 5e5;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

int version[MAXN + 1];
int cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size;
}node[MAXN * 50 + 1];

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0 };
    }
    cnt = 0;
}

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int create(int val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].priority = dist(gen);
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

void split(int l, int r, int i, ll val) {
    if (i == 0) {
```

```

        node[l].right = node[r].left = 0;
    }
    else {
        i = clone(i);
        if (node[i].key <= val) {
            node[l].right = i;
            split(i, r, node[i].right, val);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, val);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        l = clone(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        r = clone(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

int remove(int v, ll val) {
    // 基于 v 版本, 删除一个 val
    int head = version[v];
    split(0, 0, head, val);
    //    <=val                >val
    int l = node[0].right, r = node[0].left;
    node[0].left = node[0].right = 0;
    split(0, 0, l, val - 1);
    //    <val                =val
    int lm = node[0].right, rm = node[0].left;
    node[0].left = node[0].right = 0;
    int ans = merge(merge(lm, merge(node[rm].left, node[rm].right)), r);
    node[0].left = node[0].right = 0;
}

```

```

    return ans;
}

int add(int v, ll val) {
    int head = version[v];
    split(0, 0, head, val);
    //   <=val               >val
    int l = node[0].right, r = node[0].left;
    node[0].left = node[0].right = 0;
    int ans = merge(merge(l, create(val)), r);
    return ans;
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + 1 + small(node[i].right, val);
    }
}

int rank(int v, ll val) {
    return small(version[v], val) + 1;
}

ll index(int i, int rk) {
    if (i == 0) {
        return 0;
    }
    if (node[node[i].left].size >= rk) {
        return index(node[i].left, rk);
    }
    else if (node[node[i].left].size + 1 < rk) {
        return index(node[i].right, rk - node[node[i].left].size - 1);
    }
    return node[i].key;
}

ll pre(int i, ll val) {
    if (i == 0) {
        return INT_MIN + 1;
    }
    if (node[i].key >= val) {

```

```

        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll post(int i, ll val) {
    if (i == 0) {
        return INT_MAX;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

void solve() {
    int n;
    cin >> n;
    clear();
    for (int i = 1; i <= n; i++) {
        int v, op;
        ll val;
        cin >> v >> op >> val;
        if (op == 1) {
            version[i] = add(v, val);
        }
        else if (op == 2) {
            version[i] = remove(v, val);
        }
        else {
            version[i] = version[v];
            if (op == 3) {
                cout << ::rank(i, val) << '\n';
            }
            else if (op == 4) {
                cout << index(version[i], val) << '\n';
            }
            else if (op == 5) {
                cout << pre(version[i], val) << '\n';
            }
            else {
                cout << post(version[i], val) << '\n';
            }
        }
    }
}

```

```

    }
}
}

```

Splay 树

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 2e5;
const ll INF = 1e18;

// splay 树中提根操作在每个增，删，改，查操作都有必要执行
// 因为提根操作可以缩短树的高度

// 以下方法内不含提根的只有 find 方法
// 因为 remove 依赖连续的 find 辅助，一旦提根改变树结构，remove 操作非法

int head = 0, cnt = 0;
struct {
    ll key;
    int left;
    int right;
    int father;
    int size;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
    }
    head = cnt = 0;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

int side(int i) {
    return node[node[i].father].left == i ? 1 : 2;
}

```

```

}

// 左旋右旋的同时，维护争取的父子关系
// 其中节点 0 不设任何父亲
int leftRotate(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;

    node[r].father = node[i].father;
    node[i].father = r;
    if (node[i].right != 0) {
        // 0 号节点不设父亲
        node[node[i].right].father = i;
    }
    return r;
}

int rightRotate(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].left = node[l].right;
    node[l].right = i;

    node[l].father = node[i].father;
    node[i].father = l;
    if (node[i].left != 0) {
        node[node[i].left].father = i;
    }
    return l;
}

// 让 i 上升一步
void rotate(int i) {
    // father grandfather
    int fa = node[i].father, gfa = node[fa].father;
    // 不存在父亲，根节点不执行操作
    if (fa != 0) {
        int s1 = side(i), s2 = side(fa);
        if (s1 == 1) {
            rightRotate(fa);
        }
        else {
            leftRotate(fa);
        }
        // 存在爷爷，更新

```

```

        if (gfa != 0) {
            if (s2 == 1) {
                node[gfa].left = i;
            }
            else {
                node[gfa].right = i;
            }
        }
        else { // 不存在爷爷, 此时 i 成为根
            head = i;
        }
        up(fa);
        up(i);
    }
}

// 让 i 成为 goal 的儿子
void splay(int i, int goal) {
    int fa = node[i].father, gfa = node[fa].father;
    while (fa != goal) {
        // 若爷爷是目标点, i 上升一步即可
        if (gfa != goal) {
            // 一字型
            if (side(i) == side(fa)) {
                rotate(fa);
            }
            else { // 之字型
                rotate(i);
            }
        }
        rotate(i);
        fa = node[i].father;
        gfa = node[fa].father;
    }
    if (goal == 0) {
        // i 成为根节点
        head = i;
    }
}

// 查询排名 k 的节点
int find(int k) {
    int i = head;
    while (i != 0) {
        // 满足
        if (node[node[i].left].size + 1 == k) {

```

```

        return i;
    }
    else if (node[node[i].left].size >= k) {
        // 位于左侧
        i = node[i].left;
    }
    else { // 位于右侧
        k -= (node[node[i].left].size + 1);
        i = node[i].right;
    }
}
return 0; // 不存在
}

int small(ll val) {
    int i = head, last = head;
    int ans = 0;
    while (i != 0) {
        last = i;
        if (node[i].key >= val) {
            i = node[i].left;
        }
        else {
            ans += node[node[i].left].size + 1;
            i = node[i].right;
        }
    }
    splay(last, 0);
    return ans;
}

int rank(ll val) {
    return small(val) + 1;
}

void add(ll val) {
    int i = create(val);
    if (head == 0) {
        head = i;
    }
    else {
        int fa = 0, u = head, side = 0;
        while (u != 0) {
            // 左 <= 中 <= 右
            if (node[u].key <= val) {
                side = 2;
            }
        }
    }
}

```

```

        fa = u;
        u = node[u].right;
    }
    else {
        side = 1;
        fa = u;
        u = node[u].left;
    }
}
if (side == 1) {
    node[fa].left = i;
}
else {
    node[fa].right = i;
}
node[i].father = fa;
splay(i, 0);
}
}

void remove(ll val) {
    int kth = ::rank(val);
    if (kth != ::rank(val + 1)) {
        int i = find(kth);
        splay(i, 0);
        if (node[i].left == 0) {
            head = node[i].right;
        }
        else if (node[i].right == 0) {
            head = node[i].left;
        }
        else {
            // i 排名 kth, kth+1 必定在 i 的右子树
            // 把 j 提到 i 的右儿子, 此时 j 必然不存在左儿子
            // (因为 kth 是中序遍历的顺序, 所以 j 作为 kth+1 名必然不存在左儿子)
            // 把 i 的左儿子给 j, 让 j 成为头即可
            int j = find(kth + 1);
            splay(j, i);
            node[j].left = node[i].left;
            node[node[j].left].father = j;
            up(j);
            head = j;
        }
        node[head].father = 0;
    }
}
}

```

```

ll index(int x) {
    int i = find(x);
    splay(i, 0);
    return node[i].key;
}

ll pre(ll val) {
    int i = head, last = head;
    ll ans = -INF;
    while (i != 0) {
        last = i;
        if (node[i].key >= val) {
            i = node[i].left;
        }
        else {
            ans = max(ans, node[i].key);
            i = node[i].right;
        }
    }
    splay(last, 0);
    return ans;
}

ll post(ll val) {
    int i = head, last = head;
    ll ans = INF;
    while (i != 0) {
        last = i;
        if (node[i].key <= val) {
            i = node[i].right;
        }
        else {
            ans = min(ans, node[i].key);
            i = node[i].left;
        }
    }
    splay(last, 0);
    return ans;
}

```

TreapFHQ(经典非词频压缩)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

// FHQTreap
// 核心：根据 key 值执行分裂，根据优先级执行合并操作

// merge(l,r)：合并后树的中序遍历顺序就是先 l 树,再 r 树
// 因此,FHQ 树可以巧妙地维护中序遍历的特定顺序
// 同时又因为优先级的设定，使得树高也得到维护

// 如若只在乎特定的顺序，merge 和 split 的结合更为灵活，此时所谓的排名 rank 就是中序遍历顺序
// 形象地说，如果把中序遍历结果放进一维数组，此时的 rank 就是下标，即维护的就是下标

// 如若在乎 key 值的大小关系，那么 merge 操作对于 split 分裂后的左右子树，顺序不能改变，rank 不仅表示下标，还表示第 k 小
// 否则 rank 失去了关于元素大小的比较功能，只能代表遍历顺序，即只维护下标

const int MAXN = 2e5;
const ll INF = 1e18;
int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size;// 节点总数
}node[MAXN + 1];

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0 };
    }
    head = cnt = 0;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

// 若当前 curv <= key，则左侧子树不需要改变，向右子树递归
// 反之，右侧子树不需要改变，向左子树递归

```

```

// key 值分裂
void splitByKey(int l, int r, int i, ll key) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[i].key <= key) {
            node[l].right = i;
            splitByKey(i, r, node[i].right, key);
        }
        else {
            node[r].left = i;
            splitByKey(l, i, node[i].left, key);
        }
        up(i);
    }
}

// 经典非词频压缩, 可正常使用
void splitByRank(int l, int r, int i, ll rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            splitByRank(i, r, node[i].right, rk - node[node[i].left].size
- 1);
        }
        else {
            node[r].left = i;
            splitByRank(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
}

```

```

    else {
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

void add(ll val) {
    splitByKey(0, 0, head, val);
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].priority = dist(gen);
    head = merge(merge(node[0].right, cnt), node[0].left);
}

void remove(ll val) {
    splitByKey(0, 0, head, val);
    int lm = node[0].right; // <=val
    int r = node[0].left; // >val
    splitByKey(0, 0, lm, val - 1);
    int l = node[0].right; // <=val-1
    int m = node[0].left; // =val
    // =val 的去掉一个(头), val-1 部分和>val 部分全部保留
    head = merge(merge(l, merge(node[m].left, node[m].right)), r);
}

int small(ll val) {
    splitByKey(0, 0, head, val - 1);
    int ans = node[node[0].right].size;
    head = merge(node[0].right, node[0].left);
    return ans;
}

int rank(ll val) {
    return small(val) + 1;
}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + 1 < x) {
        return index(node[i].right, x - node[node[i].left].size - 1);
    }
    return node[i].key;
}

```

```

ll index(int x) {
    return index(head, x);
}

ll pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

ll pre(ll val) {
    return pre(head, val);
}

ll post(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

ll post(ll val) {
    return post(head, val);
}

```

TreapFHQ(词频压缩)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// FHQTreap
// 核心：根据 key 值执行分裂，根据优先级执行合并操作

const int MAXN = 2e5;
const ll INF = 1e18;

```

```

int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left;
    int right;
    int size; // 总元素个数(词频和)
    int count;
} node[MAXN + 1];

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int> dist(1, INT_MAX);

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0, 0 };
    }
    head = cnt = 0;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size +
node[i].count;
}

// 若当前 curv <= key, 则左侧子树不需要改变, 向右子树递归
// 反之, 右侧子树不需要改变, 向左子树递归

// key 值分裂
void splitByKey(int l, int r, int i, ll key) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[i].key <= key) {
            node[l].right = i;
            splitByKey(i, r, node[i].right, key);
        }
        else {
            node[r].left = i;
            splitByKey(l, i, node[i].left, key);
        }
        up(i);
    }
}

```

```

// 排名分裂
// 该排名分裂，只能解决每个位置的 count = 1 的情况
// 若存在重复计数，需要定制
void splitByRank(int l, int r, int i, ll rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            splitByRank(i, r, node[i].right, rk - node[node[i].left].size
- 1);
        }
        else {
            node[r].left = i;
            splitByRank(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

int find(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key == val) {
        return i;
    }
    else if (node[i].key < val) {

```

```

        return find(node[i].right, val);
    }
    else {
        return find(node[i].left, val);
    }
}

void changeCount(int i, ll val, int change) {
    if (node[i].key == val) {
        node[i].count += change;
    }
    else if (node[i].key > val) {
        changeCount(node[i].left, val, change);
    }
    else {
        changeCount(node[i].right, val, change);
    }
    up(i);
}

void add(ll val) {
    if (find(head, val) != 0) {
        changeCount(head, val, 1);
    }
    else {
        splitByKey(0, 0, head, val);
        node[++cnt].key = val;
        node[cnt].count = 1;
        node[cnt].size = 1;
        node[cnt].priority = dist(gen);
        head = merge(merge(node[0].right, cnt), node[0].left);
    }
}

void remove(ll val) {
    int i = find(head, val);
    if (i != 0) {
        if (node[i].count > 1) {
            changeCount(head, val, -1);
        }
        else {
            splitByKey(0, 0, head, val);
            int lm = node[0].right;
            int r = node[0].left;
            splitByKey(0, 0, lm, val - 1);
            int l = node[0].right;
            head = merge(l, r);
        }
    }
}

```

```

    }
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + node[i].count < x) {
        return index(node[i].right, x - node[node[i].left].size -
node[i].count);
    }
    return node[i].key;
}

ll index(int x) {
    return index(head, x);
}

ll pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(node[i].key, pre(node[i].right, val));
    }
}

```

```

ll pre(ll val) {
    return pre(head, val);
}

ll post(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return post(node[i].right, val);
    }
    else {
        return min(node[i].key, post(node[i].left, val));
    }
}

ll post(ll val) {
    return post(head, val);
}

```

Treap 旋转树

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

// Treap 树:
// 每个节点存储{key, priority}
// 根据键值 key 维护搜索二叉树顺序(左<中<右)
// 随后根据优先级 priority 维护堆结构(大/小根堆)
// priority 用随机函数 random 生成, 保证了 Treap 的平衡性

const int MAXN = 2e5;
const ll INF = 1e18;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(0, INT_MAX);

int head = 0, cnt = 0;
struct {
    ll key;
    int priority;
    int left, right;
    int size;
}

```

```

    int count;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0, 0 };
    }
    cnt = head = 0;
}

void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].size = node[l].size + node[r].size + node[i].count;
}

int rightRotate(int i) {
    int l = node[i].left;
    node[i].left = node[l].right;
    node[l].right = i;
    up(i);
    up(l);
    return l;
}

int leftRotate(int i) {
    int r = node[i].right;
    node[i].right = node[r].left;
    node[r].left = i;
    up(i);
    up(r);
    return r;
}

int add(int i, ll val) {
    if (i == 0) {
        node[++cnt].key = val;
        node[cnt].count = node[cnt].size = 1;
        node[cnt].priority = dist(gen);
        return cnt;
    }
    if (node[i].key == val) {
        node[i].count++;
    }
    else if (node[i].key > val) {
        node[i].left = add(node[i].left, val);
    }
    else {

```

```

        node[i].right = add(node[i].right, val);
    }
    up(i);
    if (node[i].left != 0 && node[node[i].left].priority >
node[i].priority) {
        return rightRotate(i);
    }
    if (node[i].right != 0 && node[node[i].right].priority >
node[i].priority) {
        return leftRotate(i);
    }
    return i;
}

void add(ll val) {
    head = add(head, val);
}

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

ll index(int i, int x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
    else if (node[node[i].left].size + node[i].count < x) {
        return index(node[i].right, x - node[node[i].left].size -
node[i].count);
    }
    return node[i].key;
}

```

```

11 index(int x) {
    return index(head, x);
}

11 pre(int i, ll val) {
    if (i == 0) {
        return -INF;
    }
    if (node[i].key >= val) {
        return pre(node[i].left, val);
    }
    else {
        return max(pre(node[i].right, val), node[i].key);
    }
}

11 pre(ll val) {
    return pre(head, val);
}

11 suf(int i, ll val) {
    if (i == 0) {
        return INF;
    }
    if (node[i].key <= val) {
        return suf(node[i].right, val);
    }
    else {
        return min(suf(node[i].left, val), node[i].key);
    }
}

11 suf(ll val) {
    return suf(head, val);
}

int remove(int i, ll val) {
    if (node[i].key < val) {
        node[i].right = remove(node[i].right, val);
    }
    else if (node[i].key > val) {
        node[i].left = remove(node[i].left, val);
    }
    else {
        if (node[i].count > 1) {
            node[i].count--;
        }
    }
}

```

```

        else {
            if (node[i].left == 0 && node[i].right == 0) {
                return 0;
            }
            else if (node[i].left == 0 && node[i].right != 0) {
                return node[i].right;
            }
            else if (node[i].right == 0 && node[i].left != 0) {
                return node[i].left;
            }
            else {
                if (node[node[i].left].priority >=
node[node[i].right].priority) {
                    i = rightRotate(i);
                    node[i].right = remove(node[i].right, val);
                }
                else {
                    i = leftRotate(i);
                    node[i].left = remove(node[i].left, val);
                }
            }
        }
    }
    up(i);
    return i;
}

void remove(ll val) {
    if (::rank(val) != ::rank(val + 1)) {
        head = remove(head, val);
    }
}
}

```

可持久化文艺平衡树

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

const int MAXN = 2e5;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

int cnt = 0;

```

```

int version[MAXN + 1];
struct {
    ll key;
    ll sum;
    int left;
    int right;
    int size;
    int priority;
    bool isRev;
}node[MAXN * 50 + 1];

void clear() {
    for (int i = 0;i <= cnt;i++) {
        node[i] = { 0,0,0,0,0,0,0 };
    }
    cnt = 0;
}

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].sum = val;
    node[cnt].priority = dist(gen);
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
    node[i].sum = node[node[i].left].sum + node[node[i].right].sum +
node[i].key;
}

void down(int i) {
    if (node[i].isRev) {
        // 若存在, 则拷贝
        if (node[i].left != 0) {
            node[i].left = clone(node[i].left);
            node[node[i].left].isRev ^= 1;
        }
        if (node[i].right != 0) {
            node[i].right = clone(node[i].right);
            node[node[i].right].isRev ^= 1;
        }
    }
}

```

```

    }
    // 对拷贝后的进行懒更新
    swap(node[i].left, node[i].right);
    node[i].isRev = false;
}
}

void split(int l, int r, int i, int rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        i = clone(i);
        down(i);
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            split(i, r, node[i].right, rk - node[node[i].left].size - 1);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, rk);
        }
        up(i);
    }
}

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        l = clone(l);
        down(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        r = clone(r);
        down(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}
}

```

```

// 基于版本 v, 对区间[jobl, jobr]进行翻转
int reverse(int v, int jobl, int jobr) {
    // 新版本的头
    int head = version[v];

    split(0, 0, head, jobr);
    //    <=jobr                >jobr
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    split(0, 0, l, jobl - 1);
    //    <jobl                [jobl, jobr]
    int lm = node[0].right, rm = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    int ans;
    node[rm].isRev ^= 1;
    lm = merge(lm, rm);
    ans = merge(lm, r);

    return ans;
}

// 基于版本 v, 删除第 x 个元素
int remove(int v, int x) {
    int head = version[v];

    split(0, 0, head, x);
    //    <=x                >x
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    split(0, 0, l, x - 1);
    //    <x                =x
    int lm = node[0].right, rm = node[0].left;
    node[0] = { 0,0,0,0,0,0,0 };

    lm = merge(lm, merge(node[rm].left, node[rm].right));
    int ans = merge(lm, r);

    return ans;
}

// 基于版本 v, 在第 x 个元素后, 插入 val
int insert(int v, int x, ll val) {
    int head = version[v];

```

```

    split(0, 0, head, x);
    //    <=x                >x
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    l = merge(l, create(val));
    int ans = merge(l, r);

    return ans;
}

// 基于版本 v, 查询[l,r]的和
int query(int v, int jobl, int jobr, ll& sum) {
    int head = version[v];

    split(0, 0, head, jobr);
    // <=jobr                >jobr
    int l = node[0].right, r = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    split(0, 0, l, jobl - 1);
    //    <jobl                [jobl,jobr]
    int lm = node[0].right, rm = node[0].left;
    node[0] = { 0,0,0,0,0,0,0,0 };

    sum = node[rm].sum;
    lm = merge(lm, rm);
    int ans = merge(lm, r);
    return ans;
}

void solve() {
    int n;
    cin >> n;
    ll lastAns = 0;
    clear();
    for (int i = 1; i <= n; i++) {
        int v, op;
        cin >> v >> op;
        if (op == 1) {
            ll p, ll x;
            cin >> p >> x;
            p ^= lastAns, x ^= lastAns;
            version[i] = insert(v, p, x);
        }
        else if (op == 2) {

```

```

        ll p;
        cin >> p;
        p ^= lastAns;
        version[i] = remove(v, p);
    }
    else if (op == 3) {
        ll l, r;
        cin >> l >> r;
        l ^= lastAns, r ^= lastAns;
        version[i] = reverse(v, l, r);
    }
    else {
        ll l, r;
        cin >> l >> r;
        l ^= lastAns, r ^= lastAns;
        ll sum;
        version[i] = query(v, l, r, sum);
        cout << sum << '\n';
        lastAns = sum;
    }
}
}

```

左偏树

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e5;

// 左偏树：一种可并堆结构
// 堆的合并时间复杂度  $O(\log n)$ 
// 左偏树依赖 dist 值维护平衡性
// dist[i]：从 i 出发到达空节点经过的最少节点数
// 左偏树的每个节点满足左儿子的 dist  $\geq$  右儿子的 dist
// 节点 dist = 右儿子 dist + 1

struct tree {
    ll val;
    int left, right;
    int dist;
    int father;
} node[MAXN + 1];

int fa[MAXN + 1];

```

```

void prepare(int n) {
    for (int i = 0; i <= n; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
        fa[i] = i;
    }
    node[0].dist = -1;
}

int find(int u) {
    return u == fa[u] ? u : fa[u] = find(fa[u]);
}

int merge(int i, int j) {
    if (i == 0 || j == 0) {
        return i + j;
    }
    // 维护小根堆
    // 让 i 成为值小的, 若值相等成为下标小的
    if (node[i].val > node[j].val || (node[i].val == node[j].val && i >
j)) {
        swap(i, j);
    }
    node[i].right = merge(node[i].right, j);
    node[node[i].right].father = i;
    if (node[node[i].left].dist < node[node[i].right].dist) {
        swap(node[i].left, node[i].right);
    }
    node[i].dist = node[node[i].right].dist + 1;
    fa[node[i].left] = i;
    fa[node[i].right] = i;
    return i;
}

// 返回新树的头节点
int pop(int i) {
    fa[node[i].left] = node[i].left;
    fa[node[i].right] = node[i].right;
    fa[i] = merge(node[i].left, node[i].right);
    node[i].left = node[i].right = node[i].dist = node[i].father = 0;
    return fa[i];
}

int remove(int i) {
    int head = find(i);
    int f = node[i].father;
    int s = pop(i);

```

```

    node[s].father = f;
    if (head != i) {
        fa[s] = head;
        if (node[f].left == i) {
            node[f].left = s;
        }
        else {
            node[f].right = s;
        }
        for (int d = node[s].dist; node[f].dist > d + 1; f = node[f].father,
d++) {
            node[f].dist = d + 1;
            if (node[node[f].left].dist < node[node[f].right].dist) {
                swap(node[f].left, node[f].right);
            }
        }
    }
    return fa[s];
}

// 把 i 调整为 to
void update(int i, ll to) {
    int h = remove(i);
    node[i].val = to;
    fa[i] = i;
    node[i].dist = 0;
    fa[h] = fa[i] = merge(h, i);
}

```

文艺平衡树(FHQ 懒标记)

```

#include<bits/stdc++.h>
using namespace std;

const int MAXN = 1e5;
int head = 0, cnt = 0;

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, INT_MAX);

struct {
    int key;
    int priority;
    int left;
    int right;
    int size;
}

```

```

    bool isRev;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0, 0 };
    }
}

int create(int val) {
    node[++cnt].key = val;
    node[cnt].size = 1;
    node[cnt].isRev = false;
    node[cnt].priority = dist(gen);
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size + 1;
}

void down(int i) {
    if (node[i].isRev) {
        swap(node[i].left, node[i].right);
        node[node[i].left].isRev ^= 1;
        node[node[i].right].isRev ^= 1;
        node[i].isRev = false;
    }
}

void split(int l, int r, int i, int rk) {
    if (i == 0) {
        node[l].right = node[r].left = 0;
    }
    else {
        down(i);
        if (node[node[i].left].size + 1 <= rk) {
            node[l].right = i;
            split(i, r, node[i].right, rk - node[node[i].left].size - 1);
        }
        else {
            node[r].left = i;
            split(l, i, node[i].left, rk);
        }
        up(i);
    }
}

```

```

int merge(int l, int r) {
    if (l == 0 || r == 0) {
        return l + r;
    }
    if (node[l].priority >= node[r].priority) {
        down(l);
        node[l].right = merge(node[l].right, r);
        up(l);
        return l;
    }
    else {
        down(r);
        node[r].left = merge(l, node[r].left);
        up(r);
        return r;
    }
}

void printTree(int i) {
    if (i != 0) {
        down(i);
        printTree(node[i].left);
        cout << node[i].key << ' ';
        printTree(node[i].right);
    }
}

void reverse(int jobl, int jobr) {
    split(0, 0, head, jobr);
    //    <=jobr                >jobr
    int l = node[0].right, r = node[0].left;
    split(0, 0, l, jobl - 1);
    //    <jobl                jobl<=&&<=jobr
    int lm = node[0].right, rm = node[0].left;
    node[rm].isRev ^= 1;
    l = merge(lm, rm);
    head = merge(l, r);
}

void solve() {
    int n, m;
    cin >> n >> m;
    clear();
    for (int i = 1; i <= n; i++) {
        head = merge(head, create(i));
    }
}

```

```

    while (m--) {
        int l, r;
        cin >> l >> r;
        reverse(l, r);
    }
    printTree(head);
}

```

替罪羊树(词频压缩)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

// 替罪羊树:
// 平衡因子 Alpha, 最优取 0.7
// 当左或右子树节点数 > n * Alpha 时
// 进行整颗子树的重构
// 若存在多个违规子树, 选择最高的节点子树

const int MAXN = 2e5;
const double Alpha = 0.7; // 平衡因子
const ll INF = 1e18;
int head = 0, cnt = 0;
int collect[MAXN + 1]; // 中序遍历收集节点
// 最高的不平衡点 top 的父亲 top 和父亲的关系
int top, father, side;
struct {
    ll key; // 键值
    int count; // 当前 key, 词频
    int left; // 左儿子
    int right; // 右儿子
    int diff; // 维护子树上不同 key 的个数(即节点数), different
    int size; // 收集的元素个数(词频和)
} node[MAXN + 1];

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].count = 1;
    node[cnt].left = node[cnt].right = 0;
    node[cnt].diff = 1;
    node[cnt].size = 1;
    return cnt;
}

void up(int i) {

```

```

    int l = node[i].left, r = node[i].right;
    node[i].size = node[l].size + node[r].size + node[i].count;
    // 由于替罪羊树的懒删除机制, 此处 diff 需要判断节点 i 是否存在词频
    node[i].diff = node[l].diff + node[r].diff + (node[i].count > 0 ? 1 :
0);
}

// 中序遍历
// 左 -> 中 -> 右
void inorder(int i) {
    if (i != 0) {
        inorder(node[i].left);
        if (node[i].count > 0) {
            collect[++collect[0]] = i;
        }
        inorder(node[i].right);
    }
}

int build(int l, int r) {
    if (l > r) {
        return 0;
    }
    int mid = (l + r) >> 1;
    int h = collect[mid];
    node[h].left = build(l, mid - 1);
    node[h].right = build(mid + 1, r);
    up(h);
    return h;
}

// 二分重构
void rebuild() {
    if (top != 0) {
        collect[0] = 0;
        inorder(top);
        if (collect[0] > 0) {
            if (father == 0) {
                head = build(1, collect[0]);
            }
            else if (side == 1) {
                node[father].left = build(1, collect[0]);
            }
            else {
                node[father].right = build(1, collect[0]);
            }
        }
    }
}

```

```

    }
}

bool isBalance(int i) {
    return Alpha * node[i].diff >= max(node[node[i].left].diff,
node[node[i].right].diff);
}

// 当前节点 父亲 父子关系 key 值
void add(int i, int fa, int s, ll val) {
    if (i == 0) {
        if (fa == 0) {
            head = create(val);
        }
        else if (s == 1) {
            node[fa].left = create(val);
        }
        else {
            node[fa].right = create(val);
        }
    }
    else {
        if (node[i].key == val) {
            node[i].count++;
        }
        else if (node[i].key > val) {
            add(node[i].left, i, 1, val);
        }
        else {
            add(node[i].right, i, 2, val);
        }
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = fa;
        side = s;
    }
}

void add(ll val) {
    top = father = side = 0;
    add(head, 0, 0, val);
    rebuild();
}

```

```

int small(int i, ll val) {
    if (i == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return small(node[i].left, val);
    }
    else {
        return node[node[i].left].size + node[i].count +
small(node[i].right, val);
    }
}

int rank(ll val) {
    return small(head, val) + 1;
}

void remove(int i, int fa, int s, ll val) {
    if (node[i].key == val) {
        node[i].count--;
    }
    else if (node[i].key > val) {
        remove(node[i].left, i, 1, val);
    }
    else {
        remove(node[i].right, i, 2, val);
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = fa;
        side = s;
    }
}

void remove(ll val) {
    if (::rank(val) != ::rank(val + 1)) {
        top = father = side = 0;
        remove(head, 0, 0, val);
        rebuild();
    }
}

ll index(int i, ll x) {
    if (node[node[i].left].size >= x) {
        return index(node[i].left, x);
    }
}

```

```

        else if (node[node[i].left].size + node[i].count < x) {
            return index(node[i].right, x - node[node[i].left].size -
node[i].count);
        }
        return node[i].key;
    }

ll index(ll x) {
    return index(head, x);
}

ll pre(ll val) {
    int kth = ::rank(val);
    if (kth == 1) {
        return -INF;
    }
    return index(kth - 1);
}

ll suf(ll val) {
    int kth = ::rank(val + 1);
    if (kth == node[head].size + 1) {
        return INF;
    }
    return index(kth);
}

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0,0,0,0,0,0 };
    }
    cnt = 0;
    head = 0;
}

```

替罪羊树(非词频压缩)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
using ld = long double;
const int MAXN = 2e6;
const ld Alpha = 0.7;
const ll INF = 1e18;

int top, father, side;

```

```

int collects[MAXN + 1];
int ci = 0;

int head = 0, cnt = 0;
struct tree {
    ll key;
    int aliveSiz;
    int left;
    int right;
    bool alive;
}node[MAXN + 1];

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i] = { 0, 0, 0, 0, 0 };
    }
    head = cnt = 0;
}

int create(ll val) {
    node[++cnt].key = val;
    node[cnt].aliveSiz = 1;
    node[cnt].alive = true;
    node[cnt].left = node[cnt].right = 0;
    return cnt;
}

void up(int i) {
    node[i].aliveSiz = node[node[i].left].aliveSiz +
node[node[i].right].aliveSiz + (node[i].alive ? 1 : 0);
}

int build(int l, int r) {
    if (l > r) {
        return 0;
    }
    int mid = (l + r) >> 1;
    int p = collects[mid];
    node[p].left = build(l, mid - 1);
    node[p].right = build(mid + 1, r);
    up(p);
    return p;
}

void inorder(int i) {
    if (i != 0 && node[i].aliveSiz > 0) {

```

```

        inorder(node[i].left);
        if (node[i].alive) {
            collects[++ci] = i;
        }
        inorder(node[i].right);
    }
}

void rebuild() {
    if (top != 0) {
        ci = 0;
        inorder(top);
        int u = build(1, ci);
        if (side == 1) {
            node[father].left = u;
        }
        else if (side == 2) {
            node[father].right = u;
        }
        else {
            head = u;
        }
        top = father = side = 0;
    }
}

bool isBalance(int i) {
    return node[i].aliveSiz * Alpha >= max(node[node[i].left].aliveSiz,
node[node[i].right].aliveSiz);
}

int add(int i, int f, int s, ll val) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return create(val);
    }
    if (node[i].key >= val) {
        node[i].left = add(node[i].left, i, 1, val);
    }
    else {
        node[i].right = add(node[i].right, i, 2, val);
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = f;
        side = s;
    }
}

```

```

    return i;
}

void add(ll val) {
    top = father = side = 0;
    head = add(head, 0, 0, val);
    rebuild();
}

int samll(int i, ll val) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return 0;
    }
    if (node[i].key >= val) {
        return samll(node[i].left, val);
    }
    else {
        return node[node[i].left].aliveSiz + (node[i].alive ? 1 : 0) +
samll(node[i].right, val);
    }
}

int rank(ll val) {
    return samll(head, val) + 1;
}

ll index(int i, int rank) {
    int cur = node[i].alive ? 1 : 0;
    if (node[node[i].left].aliveSiz >= rank) {
        return index(node[i].left, rank);
    }
    else if (node[node[i].left].aliveSiz + cur < rank) {
        return index(node[i].right, rank - node[node[i].left].aliveSiz -
cur);
    }
    return node[i].key;
}

ll index(int rank) {
    return index(head, rank);
}

int remove(int i, int f, int s, int rank) {
    if (i == 0 || node[i].aliveSiz == 0) {
        return 0;
    }
}

```

```

    int lsiz = node[node[i].left].aliveSiz;
    int cur = node[i].alive ? 1 : 0;
    if (rank <= lsiz) {
        node[i].left = remove(node[i].left, i, 1, rank);
    }
    else if (node[i].alive && rank == lsiz + cur) {
        node[i].alive = false;
    }
    else {
        node[i].right = remove(node[i].right, i, 2, rank - lsiz - cur);
    }
    up(i);
    if (!isBalance(i)) {
        top = i;
        father = f;
        side = s;
    }
    return i;
}

void remove(ll val) {
    int k1 = ::rank(val);
    int k2 = ::rank(val + 1);
    if (k1 != k2) {
        top = father = side = 0;
        head = remove(head, 0, 0, k1);
        rebuild();
    }
}

ll post(ll val) {
    int kth = ::rank(val + 1);
    if (kth == node[head].aliveSiz + 1) {
        return INF;
    }
    else {
        return index(kth);
    }
}

ll pre(ll val) {
    int kth = ::rank(val);
    if (kth == 1) {
        return -INF;
    }
    else {

```

```

        return index(kth - 1);
    }
}

```

笛卡尔树

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ill = __int128_t;

// 笛卡尔树:
// 笛卡尔树的核心在于:
// 1、根据每个元素 i 的特征值 a[i] 天然维护该特征值的单调顺序
// 2、中序遍历的结果就是元素 i 的插入顺序

// 特征:
// 每个节点存储 value
// 节点编号就是 key
// 节点编号按照搜索二叉树 左<根<右 组织
// value 满足每颗子树的头节点是最大/最小值
// 整棵树维护广义上的(大/小根)堆结构

// 建树过程依赖单调栈:
// 按照 key 的值升序遍历, 若压入栈, 则作为压住的元素(即旧的栈顶)的右儿子
// 若弹出节点, 记录最后弹出的节点编号, 作为新入栈节点的左儿子

const int MAXN = 1e7;
ll a[MAXN + 1];
int head = 0;
struct {
    ll value;
    int left, right;
} node[MAXN + 1];

void link(int i, int fa, int side) {
    // i 连接到 fa 的 side 侧
    if (side == 1) {
        node[fa].left = i;
    }
    else {
        node[fa].right = i;
    }
}

void clear(int n) {

```

```

    head = 0;
    for (int i = 1; i <= n; i++) {
        node[i] = { 0, 0, 0 };
    }
}

void build(int n) {
    clear(n);
    stack<pair<int, ll>>sta;
    for (int i = 1; i <= n; i++) {
        node[i].value = a[i];
        if (!sta.size()) {
            head = i;
            sta.push({ i, a[i] });
        }
        else {
            if (sta.top().second < a[i]) {
                link(i, sta.top().first, 2);
            }
            else {
                int last = 0; // 最后出栈的节点
                while (sta.size() && sta.top().second > a[i]) {
                    last = sta.top().first;
                    sta.pop();
                }
                if (sta.size()) {
                    link(i, sta.top().first, 2);
                }
                else {
                    head = i;
                }
                link(last, i, 1);
            }
            sta.push({ i, a[i] });
        }
    }
}

```

跳表

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 2e5;
const ll INF = 1e18;

```

```

// 跳表:
// 核心是维护每个节点的层级指针所覆盖的区间元素个数
// 使用随机函数 random
// 模拟二分查找, 期望结构类似于平衡树
// 实现 logn 的时间复杂度

// https://www.luogu.com.cn/problem/P3369

random_device rd;
mt19937 gen(rd());
uniform_int_distribution<int>dist(1, 100);
int cnt = 0;
// 最大层限制 log2(n)足够
int UpLimit = 20;
struct {
    ll key;// 键值
    int count;// 词频
    int level;// 层数
    int next[21];// 指针(指向下一个)
    int len[21];// 每个指针跨越的元素个数
}node[MAXN + 1];

void build() {
    cnt = 1;
    node[cnt].key = -INF;
    node[cnt].level = UpLimit;
}

void clear() {
    for (int i = 0; i <= cnt; i++) {
        node[i].key = node[i].count = node[i].level = 0;
        fill(node[i].next, node[i].next + UpLimit + 1, 0);
        fill(node[i].len, node[i].len + UpLimit + 1, 0);
    }
    cnt = 0;
}

// 随机层数
int randomLevel() {
    int ans = 1;
    while (ans <= UpLimit && dist(gen) < 50) {
        ans++;
    }
    return min(ans, UpLimit);
}

```

```

// 从 i 号节点 h 层出发，找到 val 所在的空间编号
int find(int i, int h, ll val) {
    // 水平跳跃 -> 竖直跳跃 -> 抵达 1 层返回
    int* next = node[i].next;// 指针数组
    while (next[h] != 0 && node[next[h]].key < val) { // 该指针指向有效空间，
        且该空间的 key 值 < val
        i = next[h];
        next = node[i].next;
    }
    if (h == 1) {
        if (next[h] != 0 && node[next[h]].key == val) {
            return next[h];
        }
        else { // 不存在 val
            return 0;
        }
    }
    // 前往下一层
    return find(i, h - 1, val);
}

void addCount(int i, int h, ll val) {
    // 增加词频
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        // 当前位置是 < val 的位置，下一个是 val
        node[node[i].next[h]].count++;
    }
    else {
        addCount(i, h - 1, val);
    }
    node[i].len[h]++; // 指针跨越元素个数 + 1
}

// 实际增加节点
// 返回值：底层指针跨越的元素个数
int addNode(int i, int h, int idx) {
    int rightCnt = 0;
    // 路径累计元素个数
    while (node[i].next[h] != 0 && node[node[i].next[h]].key <
node[idx].key) {
        rightCnt += node[i].len[h]; // 指针计数
    }
}

```

```

        i = node[i].next[h];
    }
    if (h == 1) {
        node[idx].next[h] = node[i].next[h];
        node[i].next[h] = idx;
        node[idx].len[h] = node[i].len[h];
        node[i].len[h] = node[idx].count;
        return rightCnt;
    }
    else {
        // (i ~ idx) 之间 < node[idx].val 的有多少个
        int downCnt = addNode(i, h - 1, idx);
        if (h > node[idx].level) {
            // 该层无法设置节点
            node[i].len[h]++;
        }
        else {
            node[idx].next[h] = node[i].next[h];
            node[i].next[h] = idx;
            node[idx].len[h] = node[i].len[h] + 1 - downCnt -
node[idx].count;
            node[i].len[h] = downCnt + node[idx].count;
        }
        return rightCnt + downCnt;
    }
}

void add(ll val) {
    if (find(1, UpLimit, val) != 0) {
        addCount(1, UpLimit, val);
    }
    else {
        node[++cnt].key = val;
        node[cnt].count = 1;
        node[cnt].level = randomLevel();
        addNode(1, UpLimit, cnt);
    }
}

void removeCount(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        node[node[i].next[h]].count--;
    }
}

```

```

    else {
        removeCount(i, h - 1, val);
    }
    node[i].len[h]--;
}

void removeNode(int i, int h, int idx) {
    if (h < 1) {
        return;
    }
    while (node[i].next[h] != 0 && node[node[i].next[h]].key <
node[idx].key) {
        i = node[i].next[h];
    }
    if (h > node[idx].level) {
        node[i].len[h]--;
    }
    else {
        node[i].next[h] = node[idx].next[h];
        node[i].len[h] += node[idx].len[h] - 1;
    }
    removeNode(i, h - 1, idx);
}

void remove(ll val) {
    int idx = find(1, UpLimit, val);
    if (idx != 0) {
        if (node[idx].count > 1) {
            removeCount(1, UpLimit, val);
        }
        else {
            removeNode(1, UpLimit, idx);
        }
    }
}

int small(int i, int h, ll val) {
    int rightCnt = 0;
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        rightCnt += node[i].len[h];
        i = node[i].next[h];
    }
    if (h == 1) {
        return rightCnt;
    }
    else {
        return rightCnt + small(i, h - 1, val);
    }
}

```

```

    }
}

int rank(ll val) { // 查询排名
    return small(1, UpLimit, val) + 1;
}

ll index(int i, int h, int x) {
    int c = 0;
    while (node[i].next[h] != 0 && c + node[i].len[h] < x) {
        c += node[i].len[h];
        i = node[i].next[h];
    }
    if (h == 1) {
        return node[node[i].next[h]].key;
    }
    else {
        return index(i, h - 1, x - c);
    }
}

ll index(int x) {
    return index(1, UpLimit, x);
}

ll pre(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        return i == 1 ? -INF : node[i].key;
    }
    else {
        return pre(i, h - 1, val);
    }
}

ll pre(ll val) {
    return pre(1, UpLimit, val);
}

ll suf(int i, int h, ll val) {
    while (node[i].next[h] != 0 && node[node[i].next[h]].key < val) {
        i = node[i].next[h];
    }
    if (h == 1) {
        if (node[i].next[h] == 0) {

```

```

        return INF;
    }
    if (node[node[i].next[h]].key > val) {
        return node[node[i].next[h]].key;
    }
    i = node[i].next[h];
    if (node[i].next[h] == 0) {
        return INF;
    }
    else {
        return node[node[i].next[h]].key;
    }
}
else {
    return suf(i, h - 1, val);
}
}

ll suf(ll val) {
    return suf(1, UpLimit, val);
}

```

6、树状数组

单点增加, 范围查询(二维数组)

```

#include<bits/stdc++.h>
using namespace std;
//树状数组, 树状数组的下标必须以 1 为基底
//功能: 主要用于维护可差分信息, 如累加和、乘积...

// 二维依赖矩阵求和
class SingleAndRange {
    //单点增加+范围查询
    static const int MAXN = 2000;
    static int tree[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }
    //(1,1)->(x,y)范围增加
    void add(int x, int y, int v, int n, int m) {
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree[i][j] += v;
            }
        }
    }
}

```

```

    }
    //(1,1)->(x,y)范围求和
    int sum(int x, int y) {
        int ans = 0;
        for (int i = x; i > 0; i -= lowbit(i)) {
            for (int j = y; j > 0; j -= lowbit(j)) {
                ans += tree[i][j];
            }
        }
        return ans;
    }
    //范围求和(1,r)->(x,y)
    int queryRange(int l, int r, int x, int y) {
        return sum(x, y) - sum(x, r - 1) - sum(l - 1, y) + sum(l - 1, r -
1);
    }
};

```

单点增加, 范围查询

```

#include<bits/stdc++.h>
using namespace std;
//树状数组, 树状数组的下标必须以 1 为基底
//功能: 主要用于维护可差分信息, 如累加和、乘积...

class Single_AND_Range {
private:
    static const int MAXN = 2e5;
    static int tree[MAXN + 1];
    //获取最低位 1
    int lowbit(int x) {
        return x & -x;
    }
    //单点增加
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n 是右边界
            tree[i] += val;
            i += lowbit(i);
        }
    }

    //普通求和[1...i]
    int sum(int i) {
        int ans = 0;
        while (i > 0) {
            ans += tree[i];

```

```

        i -= lowbit(i);
    }
    return ans;
}
//区间查询
int queryRange(int l, int r) {
    return sum(r) - sum(l - 1);
}
};

```

树状数组

```

#include<bits/stdc++.h>
using namespace std;
//树状数组,树状数组的下标必须以1为基底
//功能:主要用于维护可差分信息,如累加和、乘积...
//1、一维数组:单点/范围增加,单点/范围查询
//2、二维数组:单点/范围增加,范围查询
//一维树状数组
//一维数组:单点增加+范围查询

class Single_AND_Range {
private:
    static const int MAXN = 2e5;
    static int tree[MAXN + 1];
    //获取最低位1
    int lowbit(int x) {
        return x & -x;
    }
    //单点增加
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n是右边界
            tree[i] += val;
            i += lowbit(i);
        }
    }

    //普通求和[1...i]
    int sum(int i) {
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }
}

```

```

//区间查询
int queryRange(int l, int r) {
    return sum(r) - sum(l - 1);
}
};

//一维树状数组: 范围增加+单点查询
class Range_AND_Single {
    static const int MAXN = 2e5;
    static int treeDiff[MAXN + 1]; //范围增加依赖于 一维差分数组

    int lowbit(int x) {
        return x & -x;
    }
    //单点增加, 基于差分数组
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n 是右边界
            treeDiff[i] += val;
            i += lowbit(i);
        }
    }
    //范围增加
    void addRange(int n, int l, int r, int val) {
        addSingle(n, l, val);
        addSingle(n, r + 1, -val);
    }
    //单点查询
    int querySingle(int i) {
        int ans = 0;
        while (i > 0) {
            ans += treeDiff[i];
            i -= lowbit(i);
        }
        return ans;
    }
};

//一维树状数组: 范围增加+范围查询
class Range_AND_Range {
    static const int MAXN = 2e5;
    static int tree1[MAXN + 1]; //一维差分数组
    static int tree2[MAXN + 1]; //tree2[i]=(i-1)*d[i], 辅助数组
    int lowbit(int x) {
        return x & -x;
    }
}

```

```

//单点增加
void add(int n, int* tree, int i, int val) {
    while (i <= n) {
        tree[i] += val;
        i += lowbit(i);
    }
}

//树 tree 区间和[1...i]
int sum(int* tree, int i) {
    //1...i
    int ans = 0;
    while (i > 0) {
        ans += tree[i];
        i -= lowbit(i);
    }
    return ans;
}

//tree 区间修改
void addRange(int n, int l, int r, int v) {
    add(n, tree1, l, v);
    add(n, tree1, r + 1, -v);
    add(n, tree2, l, (l - 1) * v);
    add(n, tree2, r + 1, -r * v);
}

//求原数组区间和
int queryRange(int l, int r) {
    return sum(tree1, r) * r - sum(tree2, r) - sum(tree1, l - 1) * (l
- 1) + sum(tree2, l - 1);
}

};

class TwoSingleAndRange {
    //单点增加+范围查询
    static const int MAXN = 2000;
    static int tree[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }

    //(1,1)->(x,y)范围增加
    void add(int x, int y, int v, int n, int m) {
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree[i][j] += v;
            }
        }
    }
}

```

```

    }
}
//(1,1)->(x,y)范围求和
int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            ans += tree[i][j];
        }
    }
    return ans;
}
//范围求和(l,r)->(x,y)
int queryRange(int l, int r, int x, int y) {
    return sum(x, y) - sum(x, r - 1) - sum(l - 1, y) + sum(l - 1, r -
1);
}
};

class TwoRangeAndRange {
    //范围增加+范围查询
    static const int MAXN = 2000;
    //二维差分数组,tree 维护四个信息
    //diff[i][j]
    static int tree1[MAXN + 1][MAXN + 1];
    //diff[i][j]*i
    static int tree2[MAXN + 1][MAXN + 1];
    //diff[i][j]*j
    static int tree3[MAXN + 1][MAXN + 1];
    //diff[i][j]*i*j
    static int tree4[MAXN + 1][MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }

    //位置(x,y)单点增加
    void addSingle(int x, int y, int v, int n, int m) {
        int v1 = v;
        int v2 = v * x;
        int v3 = v * y;
        int v4 = x * y * v;
        for (int i = x; i <= n; i += lowbit(i)) {
            for (int j = y; j <= m; j += lowbit(j)) {
                tree1[i][j] += v1;

```

```

        tree2[i][j] += v2;
        tree3[i][j] += v3;
        tree4[i][j] += v4;
    }
}
}
//求和(1,1)->(x,y)
int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            //四个信息
            ans += (x + 1) * (y + 1) * tree1[i][j]
                - (y + 1) * tree2[i][j]
                - (x + 1) * tree3[i][j]
                + tree4[i][j];
        }
    }
    return ans;
}
//范围增加
void addRange(int a, int b, int c, int d, int v, int n, int m) {
    addSingle(a, b, v, n, m);
    addSingle(a, d + 1, -v, n, m);
    addSingle(c + 1, b, -v, n, m);
    addSingle(c + 1, d + 1, v, n, m);
}
//范围查询
int queryRange(int a, int b, int c, int d, int n, int m) {
    int ans = 0;
    ans = sum(c, d) - sum(a - 1, d) - sum(c, b - 1) + sum(a - 1, b -
1);
    return ans;
}
};

```

范围增加, 单点查询

```

#include<bits/stdc++.h>
using namespace std;
//树状数组, 树状数组的下标必须以 1 为基底
//功能: 主要用于维护可差分信息, 如累加和、乘积...

//一维树状数组: 范围增加+单点查询
//范围增加依赖于维护一维差分数组

```

```

class Range_AND_Single {
    static const int MAXN = 2e5;
    static int treeDiff[MAXN + 1];

    int lowbit(int x) {
        return x & -x;
    }
    //单点增加,基于差分数组
    void addSingle(int n, int i, int val) {
        while (i <= n) { //n 是右边界
            treeDiff[i] += val;
            i += lowbit(i);
        }
    }
    //范围增加
    void addRange(int n, int l, int r, int val) {
        addSingle(n, l, val);
        addSingle(n, r + 1, -val);
    }
    //单点查询
    int querySingle(int i) {
        int ans = 0;
        while (i > 0) {
            ans += treeDiff[i];
            i -= lowbit(i);
        }
        return ans;
    }
};

```

范围增加, 范围查询(二维数组)

```

#include<bits/stdc++.h>
using namespace std;
//树状数组,树状数组的下标必须以 1 为基底
//功能:主要用于维护可差分信息,如累加和、乘积...

// 二维查询依赖于差分数组维护信息
class RangeAndRange {
    //范围增加+范围查询
    static const int MAXN = 2000;
    //二维差分数组,tree 维护四个信息
    //diff[i][j]
    static int tree1[MAXN + 1][MAXN + 1];
    //diff[i][j]*i

```

```

static int tree2[MAXN + 1][MAXN + 1];
//diff[i][j]*j
static int tree3[MAXN + 1][MAXN + 1];
//diff[i][j]*i*j
static int tree4[MAXN + 1][MAXN + 1];

int lowbit(int x) {
    return x & -x;
}

//位置(x,y)单点增加
void addSingle(int x, int y, int v, int n, int m) {
    int v1 = v;
    int v2 = v * x;
    int v3 = v * y;
    int v4 = x * y * v;
    for (int i = x; i <= n; i += lowbit(i)) {
        for (int j = y; j <= m; j += lowbit(j)) {
            tree1[i][j] += v1;
            tree2[i][j] += v2;
            tree3[i][j] += v3;
            tree4[i][j] += v4;
        }
    }
}

//求和(1,1)->(x,y)
int sum(int x, int y) {
    int ans = 0;
    for (int i = x; i > 0; i -= lowbit(i)) {
        for (int j = y; j > 0; j -= lowbit(j)) {
            //四个信息
            ans += (x + 1) * (y + 1) * tree1[i][j]
                - (y + 1) * tree2[i][j]
                - (x + 1) * tree3[i][j]
                + tree4[i][j];
        }
    }
    return ans;
}

//范围增加
void addRange(int a, int b, int c, int d, int v, int n, int m) {
    addSingle(a, b, v, n, m);
    addSingle(a, d + 1, -v, n, m);
    addSingle(c + 1, b, -v, n, m);
    addSingle(c + 1, d + 1, v, n, m);
}

```

```

//范围查询
int queryRange(int a, int b, int c, int d, int n, int m) {
    int ans = 0;
    ans = sum(c, d) - sum(a - 1, d) - sum(c, b - 1) + sum(a - 1, b -
1);
    return ans;
}

};

```

范围增加, 范围查询

```

#include<bits/stdc++.h>
using namespace std;
//树状数组, 树状数组的下标必须以 1 为基底
//功能: 主要用于维护可差分信息, 如累加和、乘积...

//一维树状数组: 范围增加+范围查询
//依赖两个一维数组维护
class Range_AND_Range {
    static const int MAXN = 2e5;
    static int tree1[MAXN + 1]; //一维差分数组
    static int tree2[MAXN + 1]; //tree2[i]=(i-1)*d[i], 辅助数组
    int lowbit(int x) {
        return x & -x;
    }
    //单点增加
    void add(int n, int* tree, int i, int val) {
        while (i <= n) {
            tree[i] += val;
            i += lowbit(i);
        }
    }
    //树 tree 区间和[1...i]
    int sum(int* tree, int i) {
        //1...i
        int ans = 0;
        while (i > 0) {
            ans += tree[i];
            i -= lowbit(i);
        }
        return ans;
    }
    //tree 区间修改
    void addRange(int n, int l, int r, int v) {

```

```

        add(n, tree1, l, v);
        add(n, tree1, r + 1, -v);
        add(n, tree2, l, (l - 1) * v);
        add(n, tree2, r + 1, -r * v);
    }
    //求原数组区间和
    int queryRange(int l, int r) {
        return sum(tree1, r) * r - sum(tree2, r) - sum(tree1, l - 1) * (l
- 1) + sum(tree2, l - 1);
    }
};

```

7、线段树

可持久化线段树(区间 mex 值)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e5;

// lateLeft 表示
// 所管辖的区间内，所有出现的元素的最晚出现位置的最左位置
// 核心判断：某个节点左儿子的 lateLeft >= L，意味着要查询范围的 mex 值在右侧
// 线段树的节点，维护的是所管辖的值域区间，值出现的位置的情况

int n, m;
int a[MAXN + 5];
int version[MAXN + 5];

int cnt = 0;
struct tree {
    int left;
    int right;
    int lateLeft;
}node[(MAXN << 2 | 1) + MAXN * 60];

int clone(int i) {
    cnt++;
    node[cnt] = node[i];
    return cnt;
}

void up(int i) {
    node[i].lateLeft = min(node[node[i].left].lateLeft,
node[node[i].right].lateLeft);
}

```

```

}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].left = node[rt].right = node[rt].lateLeft = 0;
    if (l != r) {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

int update(int jobi, int jobv, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].lateLeft = jobv;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = update(jobi, jobv, l, mid, node[rt].left);
        }
        else {
            node[rt].right = update(jobi, jobv, mid + 1, r,
node[rt].right);
        }
        up(rt);
    }
    return rt;
}

int query(int L, int l, int r, int i) {
    if (l == r) {
        return l;
    }
    int mid = (l + r) >> 1;
    if (node[node[i].left].lateLeft >= L) {
        return query(L, mid + 1, r, node[i].right);
    }
    else {
        return query(L, l, mid, node[i].left);
    }
}

void prepare() {

```

```

    cnt = 0;
    version[0] = build(1, n);
    for (int v, i = 1; i <= n; i++) {
        // [0, n - 1] a[i] > n
        // 映射[1, n] a[i] + 1 > n + 1
        v = a[i] + 1;
        if (v > n + 1) {
            version[i] = version[i - 1];
        }
        else {
            version[i] = update(v, i, 1, n + 1, version[i - 1]);
        }
    }
}

void solve() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    prepare();
    while (m--) {
        int l, r;
        cin >> l >> r;
        cout << query(l, 1, n + 1, version[r]) - 1 << '\n';
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    solve();
    return 0;
}

```

可持久化线段树(区间第 k 小,单点+区间)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;

int a[MAXN + 1];
int version[MAXN + 1];

```

```

int d[MAXN + 1];
int m = 0;
int cnt = 0;
struct tree {
    int left;
    int right;
    int size;
}node[(MAXN << 2 | 1) + MAXN * 20];

int clone(int i) {
    ++cnt;
    node[cnt] = node[i];
    return cnt;
}

void up(int i) {
    node[i].size = node[node[i].left].size + node[node[i].right].size;
}

int id(int val) {
    int l = 1, r = m;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (d[mid] == val) {
            return mid;
        }
        else if (d[mid] > val) {
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return l;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].size = 0;
    if (l == r) {
        return rt;
    }
    int mid = (l + r) >> 1;
    node[rt].left = build(l, mid);
    node[rt].right = build(mid + 1, r);
    up(rt);
    return rt;
}

```

```

}

int insert(int jobi, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].size++;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = insert(jobi, l, mid, node[rt].left);
        }
        else {
            node[rt].right = insert(jobi, mid + 1, r, node[rt].right);
        }
        up(rt);
    }
    return rt;
}

int query(int jobk, int l, int r, int h1, int h2) {
    if (l == r) {
        return d[l];
    }
    int mid = (l + r) >> 1;
    int lsiz = node[node[h2].left].size - node[node[h1].left].size;
    if (lsiz >= jobk) {
        return query(jobk, l, mid, node[h1].left, node[h2].left);
    }
    else {
        return query(jobk - lsiz, mid + 1, r, node[h1].right,
node[h2].right);
    }
}

void prepare() {
    int c = m;
    m = 0;
    sort(d + 1, d + c + 1);
    d[0] = -1;
    for (int i = 1; i <= c; i++) {
        if (d[i] != d[m]) {
            d[++m] = d[i];
        }
    }
}
}

```

```
// https://www.luogu.com.cn/problem/P3834

void solve() {
    int n, q;
    cin >> n >> q;
    cnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        d[i] = a[i];
    }
    m = n;
    prepare();
    version[0] = build(1, m);
    for (int i = 1; i <= n; i++) {
        // 基于上一个版本, 排名为 id(a[i]) 的位置, 词频 + 1
        version[i] = insert(id(a[i]), 1, m, version[i - 1]);
    }
    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << query(k, 1, m, version[l - 1], version[r]) << '\n';
    }
}
```

可持久化线段树(懒更新,范围操作)

```
#include<bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;

ll a[MAXN + 1];
int version[MAXN + 1];

int cnt = 0;
struct tree {
    int left;
    int right;
    ll sum;
    ll lazyAdd;
} node[(MAXN << 2 | 1) + MAXN * 70];

void up(int i) {
    node[i].sum = node[node[i].left].sum + node[node[i].right].sum;
}
```

```

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].lazyAdd = 0;
    if (l == r) {
        node[rt].sum = a[l];
    }
    else {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

void lazy(int i, int siz, ll jobv) {
    node[i].sum += siz * jobv;
    node[i].lazyAdd += jobv;
}

void down(int i, int lsiz, int rsiz) {
    if (node[i].lazyAdd != 0) {
        node[i].left = clone(node[i].left);
        node[i].right = clone(node[i].right);
        lazy(node[i].left, lsiz, node[i].lazyAdd);
        lazy(node[i].right, rsiz, node[i].lazyAdd);
        node[i].lazyAdd = 0;
    }
}

ll add(int jobl, int jobr, ll jobv, int l, int r, int i) {
    int rt = clone(i);
    if (jobl <= l && r <= jobr) {
        lazy(rt, r - l + 1, jobv);
    }
    else {
        int mid = (l + r) >> 1;
        down(rt, mid - l + 1, r - mid);
        if (jobl <= mid) {
            node[rt].left = add(jobl, jobr, jobv, l, mid, node[rt].left);

```

```

    }
    if (jobr > mid) {
        node[rt].right = add(jobl, jobr, jobv, mid + 1, r,
node[rt].right);
    }
    up(rt);
}
return rt;
}

ll query(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - 1 + 1, r - mid);
        ll ans = 0;
        if (jobl <= mid) {
            ans += query(jobl, jobr, l, mid, node[i].left);
        }
        if (jobr > mid) {
            ans += query(jobl, jobr, mid + 1, r, node[i].right);
        }
        up(i);
        return ans;
    }
}

void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    int time = 0;
    version[0] = build(1, n);
    while (m--) {
        char op;
        cin >> op;
        if (op == 'C') {
            int l, r;
            ll v;
            cin >> l >> r >> v;
            version[time + 1] = add(1, r, v, 1, n, version[time]);
            time++;
        }
    }
}

```

```

        else if (op == 'Q') {
            int l, r;
            cin >> l >> r;
            cout << query(l, r, 1, n, version[time]) << '\n';
        }
        else if (op == 'H') {
            int l, r, ver;
            cin >> l >> r >> ver;
            cout << query(l, r, 1, n, version[ver]) << '\n';
        }
        else {
            cin >> time;
        }
    }
}

```

可持久化线段树(标记永久化)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const int MAXN = 1e5;

int version[MAXN + 1];
ll a[MAXN + 1];

// https://acm.hdu.edu.cn/showproblem.php?pid=4348

struct tree {
    int left;
    int right;
    ll sum;
    ll addTag;
} node[(MAXN << 2 | 1) + MAXN * 20];
int cnt = 0;

int clone(int i) {
    node[++cnt] = node[i];
    return cnt;
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].addTag = 0;
    node[rt].sum = 0;
    node[rt].left = node[rt].right = 0;
    if (l == r) {

```

```

        node[rt].sum = a[l];
    }
    else {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        node[rt].sum = node[node[rt].left].sum + node[node[rt].right].sum;
    }
    return rt;
}

int add(int jobl, int jobr, ll tag, int l, int r, int i) {
    int rt = clone(i);
    ll a = max(l, jobl), b = min(jobr, r);
    node[rt].sum += tag * (b - a + 1);
    // 只有当，节点被完全包裹时，打上标签信息
    if (jobl <= l && r <= jobr) {
        node[rt].addTag += tag;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            node[rt].left = add(jobl, jobr, tag, l, mid, node[rt].left);
        }
        if (jobr > mid) {
            node[rt].right = add(jobl, jobr, tag, mid + 1, r,
node[rt].right);
        }
    }
    return rt;
}

ll query(int jobl, int jobr, ll addHistory, int l, int r, int i) {
    ll a = max(l, jobl), b = min(jobr, r);
    if (jobl <= l && r <= jobr) {
        return node[i].sum + addHistory * (b - a + 1LL);
    }
    else {
        int mid = (l + r) >> 1;
        // 此处的 ans，初始化为 0，在子节点上获取正确的累加和值
        ll ans = 0;
        if (jobl <= mid) {
            ans += query(jobl, jobr, addHistory + node[i].addTag, l, mid,
node[i].left);
        }
        if (jobr > mid) {

```

```

        ans += query(jobl, jobr, addHistory + node[i].addTag, mid + 1,
r, node[i].right);
    }
    return ans;
}
}

void solve() {
    int n, m;
    cin >> n >> m;
    int time = 0;
    cnt = 0;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    version[0] = build(1, n);
    while (m--) {
        char op;
        int l, r, ver;
        ll v;
        cin >> op;
        if (op == 'C') {
            cin >> l >> r >> v;
            version[time + 1] = add(l, r, v, 1, n, version[time]);
            time++;
        }
        else if (op == 'Q') {
            cin >> l >> r;
            cout << query(l, r, 0, 1, n, version[time]) << '\n';
        }
        else if (op == 'H') {
            cin >> l >> r >> ver;
            cout << query(l, r, 0, 1, n, version[ver]) << '\n';
        }
        else {
            cin >> time;
        }
    }
}
}

```

可持久化线段树(浮动区间最大上中位数)

```

#include<bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 2e4;

```

```

const ll INF = 1e18;

// 查询区间[l, r]的上中位数
// 其中 l 属于[a, b], r 属于[c, a]
// a < b < c < d

// 解题策略：把原始数组 a 根据值排序，同时存储下标
// 根据排序后的数组生成每个位置的可持久化线段树
// 排序后的数组 a，对于位置 i, j < i 的 j 位置存储的下标，线段树上标记为 - 1
// 在排序后的数组 a 上二分查找答案

int cnt = 0;
int n, m;
int version[MAXN + 1];
int query[4];

struct Arr {
    int val;
    int pos;
}arr[MAXN + 1];

struct tree {
    int left;
    int right;
    ll pre;
    ll suf;
    ll sum;
}node[(MAXN << 2 | 1) + MAXN * 60];

int clone(int i) {
    int rt = ++cnt;
    node[rt] = node[i];
    return rt;
}

void up(int i) {
    int l = node[i].left, r = node[i].right;
    node[i].sum = node[l].sum + node[r].sum;
    node[i].pre = max(node[l].pre, node[l].sum + node[r].pre);
    node[i].suf = max(node[r].suf, node[r].sum + node[l].suf);
}

int build(int l, int r) {
    int rt = ++cnt;
    node[rt].pre = node[rt].suf = node[rt].sum = r - l + 1;
    node[rt].left = node[rt].right = 0;
}

```

```

    if (l != r) {
        int mid = (l + r) >> 1;
        node[rt].left = build(l, mid);
        node[rt].right = build(mid + 1, r);
        up(rt);
    }
    return rt;
}

// 把 jobi 位置的值更新为-1
int update(int jobi, int l, int r, int i) {
    int rt = clone(i);
    if (l == r) {
        node[rt].pre = node[rt].suf = node[rt].sum = -1;
    }
    else {
        int mid = (l + r) >> 1;
        if (jobi <= mid) {
            node[rt].left = update(jobi, l, mid, node[rt].left);
        }
        else {
            node[rt].right = update(jobi, mid + 1, r, node[rt].right);
        }
        up(rt);
    }
    return rt;
}

// 查询区间[jobl, jobr]以 jobl 开头的最大的前缀和
array<ll, 2> queryMaxPreSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return { node[i].pre, node[i].sum };
    }
    else {
        int mid = (l + r) >> 1;
        if (jobr <= mid) { // 完全被左侧包裹
            return queryMaxPreSum(jobl, jobr, l, mid, node[i].left);
        }
        else if (jobl > mid) { // 完全被右侧包裹
            return queryMaxPreSum(jobl, jobr, mid + 1, r, node[i].right);
        }
        else {
            array<ll, 2> left = queryMaxPreSum(jobl, jobr, l, mid,
node[i].left);
            array<ll, 2> right = queryMaxPreSum(jobl, jobr, mid + 1, r,
node[i].right);

```

```

        ll pre, sum;
        pre = max(left[0], left[1] + right[0]);
        sum = left[1] + right[1];
        return { pre, sum };
    }
}

// 查询区间[jobl, jobr]以 jobr 结尾的最大后缀和
array<ll, 2> queryMaxSufSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return { node[i].suf, node[i].sum };
    }
    else {
        int mid = (l + r) >> 1;
        if (jobl > mid) { // 完全被右侧包裹
            return queryMaxSufSum(jobl, jobr, mid + 1, r, node[i].right);
        }
        else if (jobr <= mid) { // 完全被左侧包裹
            return queryMaxSufSum(jobl, jobr, l, mid, node[i].left);
        }
        else {
            array<ll, 2> left = queryMaxSufSum(jobl, jobr, l, mid,
node[i].left);
            array<ll, 2> right = queryMaxSufSum(jobl, jobr, mid + 1, r,
node[i].right);
            ll suf, sum;
            suf = max(right[0], right[1] + left[0]);
            sum = left[1] + right[1];
            return { suf, sum };
        }
    }
}

// 查询区间和
ll queryRangeSum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        ll ans = 0;
        if (jobl <= mid) {
            ans += queryRangeSum(jobl, jobr, l, mid, node[i].left);
        }
        if (jobr > mid) {

```

```

        ans += queryRangeSum(jobl, jobr, mid + 1, r, node[i].right);
    }
    return ans;
}

}

void prepare() {
    cnt = 0;
    sort(arr + 1, arr + n + 1, [](Arr& a, Arr& b) {return a.val <
b.val;});
    version[1] = build(1, n);
    for (int i = 2; i <= n; i++) {
        version[i] = update(arr[i - 1].pos, 1, n, version[i - 1]);
    }
}

bool check(int a, int b, int c, int d, int pos) {
    ll sum = queryMaxSufSum(a, b, 1, n, version[pos])[0];
    sum += queryMaxPreSum(c, d, 1, n, version[pos])[0];
    sum += queryRangeSum(b + 1, c - 1, 1, n, version[pos]);
    return sum >= 0;
}

ll compute(int a, int b, int c, int d) {
    int l = 1, r = n;
    int ans = 0;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (check(a, b, c, d, mid)) {
            ans = mid;
            l = mid + 1;
        }
        else {
            r = mid - 1;
        }
    }
    return arr[ans].val;
}

// https://www.luogu.com.cn/problem/P2839

void solve() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i].val;
    }
}

```

```

        arr[i].pos = i;
    }
    prepare();
    ll lastAns = 0;
    cin >> m;
    while (m--) {
        cin >> query[0] >> query[1] >> query[2] >> query[3];
        query[0] = (query[0] + lastAns) % n;
        query[1] = (query[1] + lastAns) % n;
        query[2] = (query[2] + lastAns) % n;
        query[3] = (query[3] + lastAns) % n;
        sort(query, query + 4);
        // 下标转化为 1 基底
        lastAns = compute(query[0] + 1, query[1] + 1, query[2] + 1,
query[3] + 1);
        cout << lastAns << '\n';
    }
}

```

线段树区间最值问题

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;
const int MAXN = 1e6;
ll a[MAXN + 1];
ll sum[4 * MAXN + 1];
ll maxval[4 * MAXN + 1];
ll cnt[4 * MAXN + 1];
ll cmaxval[4 * MAXN + 1];

void up(int i) {
    int l = 2 * i, r = l + 1;
    sum[i] = sum[l] + sum[r];
    maxval[i] = max(maxval[l], maxval[r]);
    if (maxval[l] > maxval[r]) {
        cmaxval[i] = max(maxval[r], cmaxval[l]);
        cnt[i] = cnt[l];
    }
    else if (maxval[r] > maxval[l]) {
        cmaxval[i] = max(maxval[l], cmaxval[r]);
        cnt[i] = cnt[r];
    }
    else {
        cmaxval[i] = max(cmaxval[l], cmaxval[r]);
        cnt[i] = cnt[l] + cnt[r];
    }
}

```

```

    }
}
void build(int l, int r, int i) {
    if (l == r) {
        maxval[i] = a[l];
        cmaxval[i] = -INF;
        sum[i] = a[l];
        cnt[i] = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, 2 * i);
        build(mid + 1, r, 2 * i + 1);
        up(i);
    }
}
void lazy(int i, ll jobv) {
    if (jobv < maxval[i]) {
        sum[i] -= (maxval[i] - jobv) * cnt[i];
        maxval[i] = jobv;
    }
}
void down(int i) {
    int l = 2 * i, r = l + 1;
    lazy(l, maxval[i]);
    lazy(r, maxval[i]);
}
void setMin(int jobl, int jobr, ll jobv, int l, int r, int i) {
    if (jobv >= maxval[i]) {
        return;
    }
    if (jobl <= l && r <= jobr && cmaxval[i] < jobv) {
        //cmax<jobv<=max, 懒标记
        lazy(i, jobv);
    }
    else {
        //1.任务没有全包
        //2.jobv <= cmax 不得不下传
        int mid = (l + r) >> 1;
        down(i);
        if (jobl <= mid) {
            setMin(jobl, jobr, jobv, l, mid, 2 * i);
        }
        if (jobr > mid) {
            setMin(jobl, jobr, jobv, mid + 1, r, 2 * i + 1);
        }
    }
}

```

```

        up(i);
    }
}
ll querySum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return sum[i];
    }
    int mid = (l + r) >> 1;
    down(i);
    ll ans = 0;
    if (jobl <= mid) {
        ans += querySum(jobl, jobr, l, mid, 2 * i);
    }
    if (jobr > mid) {
        ans += querySum(jobl, jobr, mid + 1, r, 2 * i + 1);
    }
    up(i);
    return ans;
}

ll queryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return maxval[i];
    }
    int mid = (l + r) >> 1;
    down(i);
    ll ans = -INF;
    if (jobl <= mid) {
        ans = max(ans, queryMax(jobl, jobr, l, mid, 2 * i));
    }
    if (jobr > mid) {
        ans = max(ans, queryMax(jobl, jobr, mid + 1, r, 2 * i + 1));
    }
    up(i);
    return ans;
}

```

线段树历史最值问题

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = 1e18;
const int MAXN = 5e5;
ll a[MAXN + 1];
struct {
    ll sum, max, cnt, cmax;

```

```

    ll maxAdd, otherAdd;
    ll maxHistory, maxAddTop;
    ll otherAddTop;
    // sum          总和
    // max          最大值
    // cnt          最大值个数
    // cmax         次最大值
    // maxHistory   历史最大值
    // 以下属于懒信息标记内容:
    // maxAdd       最大值增幅(当前)
    // otherAdd     其他值增幅(当前)
    // maxAddTop    最大值的历史最大增幅
    // otherAddTop  其他值的历史最大增幅
}node[4 * MAXN + 1];

void up(int i) {
    int l = i << 1;
    int r = i << 1 | 1;
    node[i].sum = node[l].sum + node[r].sum;
    node[i].max = max(node[l].max, node[r].max);
    node[i].maxHistory = max(node[l].maxHistory, node[r].maxHistory);
    if (node[l].max > node[r].max) {
        node[i].cnt = node[l].cnt;
        node[i].cmax = max(node[l].cmax, node[r].max);
    }
    else if (node[l].max < node[r].max) {
        node[i].cnt = node[r].cnt;
        node[i].cmax = max(node[r].cmax, node[l].max);
    }
    else {
        node[i].cnt = node[l].cnt + node[r].cnt;
        node[i].cmax = max(node[l].cmax, node[r].cmax);
    }
}

//传参: 节点, 大小, 最大值增幅, 其他值增幅, 最大值历史最大增幅, 其他值历史最大增幅
void lazy(int i, int n, ll maxAddv, ll otherAddv, ll maxUpv, ll otherUpv)
{
    node[i].maxHistory = max(node[i].maxHistory, node[i].max + maxUpv);
    node[i].maxAddTop = max(node[i].maxAddTop, node[i].maxAdd + maxUpv);
    node[i].otherAddTop = max(node[i].otherAddTop, node[i].otherAdd +
otherUpv);
    node[i].sum += maxAddv * node[i].cnt + otherAddv * (n - node[i].cnt);
    node[i].max += maxAddv;
    node[i].cmax += (node[i].cmax == -INF) ? 0 : otherAddv;
}

```

```

    node[i].maxAdd += maxAddv;
    node[i].otherAdd += otherAddv;
}
void down(int i, int ln, int rn) {
    int l = i << 1;
    int r = i << 1 | 1;
    ll tmp = max(node[l].max, node[r].max); //父节点最大子来自子节点,父节点可能已经发生改变
    if (node[l].max == tmp) {
        lazy(l, ln, node[i].maxAdd, node[i].otherAdd, node[i].maxAddTop, node[i].otherAddTop);
    }
    else {
        lazy(l, ln, node[i].otherAdd, node[i].otherAdd, node[i].otherAddTop, node[i].otherAddTop);
    }
    if (node[r].max == tmp) {
        lazy(r, rn, node[i].maxAdd, node[i].otherAdd, node[i].maxAddTop, node[i].otherAddTop);
    }
    else {
        lazy(r, rn, node[i].otherAdd, node[i].otherAdd, node[i].otherAddTop, node[i].otherAddTop);
    }
    node[i].maxAdd = node[i].otherAdd = node[i].maxAddTop = node[i].otherAddTop = 0;
}
void build(int l, int r, int i) {
    if (l == r) {
        node[i].sum = node[i].max = node[i].maxHistory = a[l];
        node[i].cmax = -INF;
        node[i].cnt = 1;
    }
    else {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1);
        build(mid + 1, r, i << 1 | 1);
        up(i);
    }
    //懒信息
    node[i].maxAdd = node[i].otherAdd = node[i].maxAddTop = node[i].otherAddTop = 0;
}
//区间增加
void add(int jobl, int jobr, ll jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {

```

```

        lazy(i, r - l + 1, jobv, jobv, jobv, jobv);
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
        up(i);
    }
}
//设置区间最大值
void setMin(int jobl, int jobr, ll jobv, int l, int r, int i) {
    if (jobv >= node[i].max) {
        return;
    }
    if (jobl <= l && r <= jobr && node[i].cmax < jobv) {
        lazy(i, r - l + 1, jobv - node[i].max, 0, jobv - node[i].max, 0);
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        if (jobl <= mid) {
            setMin(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            setMin(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
        up(i);
    }
}
//查询区间和
ll querySum(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].sum;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = 0;
        if (jobl <= mid) {
            ans += querySum(jobl, jobr, l, mid, i << 1);
        }
    }
}

```

```

        if (jobr > mid) {
            ans += querySum(jobl, jobr, mid + 1, r, i << 1 | 1);
        }
        return ans;
    }
}

//查询区间最大值
ll queryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].max;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = -INF;
        if (jobl <= mid) {
            ans = max(ans, queryMax(jobl, jobr, l, mid, i << 1));
        }
        if (jobr > mid) {
            ans = max(ans, queryMax(jobl, jobr, mid + 1, r, i << 1 | 1));
        }
        return ans;
    }
}

//查询区间历史最大值
ll queryHistoryMax(int jobl, int jobr, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        return node[i].maxHistory;
    }
    else {
        int mid = (l + r) >> 1;
        down(i, mid - l + 1, r - mid);
        ll ans = -INF;
        if (jobl <= mid) {
            ans = max(ans, queryHistoryMax(jobl, jobr, l, mid, i << 1));
        }
        if (jobr > mid) {
            ans = max(ans, queryHistoryMax(jobl, jobr, mid + 1, r, i << 1
| 1));
        }
        return ans;
    }
}

```

线段树和扫描线(矩阵周长并)

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 5e3;
array<int, 4>infox[2 * MAXN + 5];
array<int, 4>infoy[2 * MAXN + 5];
int fx[2 * MAXN + 5];
int fy[2 * MAXN + 5];
int n, m, cntx, cnty;

struct {
    ll len;
    ll cover;
    int cnt;
}node[8 * MAXN + 5];

void up(int i, int l, int r) {
    if (l == r) {
        node[i].cover = node[i].cnt > 0 ? node[i].len : 0;
    }
    else {
        node[i].cover = node[i].cnt > 0 ? node[i].len : node[i << 1].cover
+ node[i << 1 | 1].cover;
    }
}

void build(int l, int r, int i, int* f) {
    node[i].len = f[r + 1] - f[l];
    node[i].cover = 0;
    node[i].cnt = 0;
    if (l != r) {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1, f);
        build(mid + 1, r, i << 1 | 1, f);
        up(i, l, r);
    }
}

void add(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        node[i].cnt += jobv;
    }
    else {
        int mid = (l + r) >> 1;
```

```

        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
    }
    up(i, l, r);
}

void Disc() {
    sort(fy + 1, fy + m + 1);
    fy[0] = -1;
    int k = m; m = 0;
    for (int i = 1; i <= k; i++) {
        if (fy[i] != fy[m]) {
            fy[++m] = fy[i];
        }
    }
    fy[m + 1] = fy[m];

    sort(fx + 1, fx + n + 1);
    fx[0] = -1;
    k = n; n = 0;
    for (int i = 1; i <= k; i++) {
        if (fx[i] != fx[n]) {
            fx[++n] = fx[i];
        }
    }
    fx[n + 1] = fx[n];
}

int rankk(int* disc, int R, int v) {
    int l = 1, r = R;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (disc[mid] == v) {
            return mid;
        }
        else if (disc[mid] > v) {
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return l;
}

```

```

}

bool cmp(array<int, 4>a, array<int, 4>b) {
    if (a[0] == b[0]) {
        return a[3] > b[3];
    }
    return a[0] < b[0];
}

void solve() {
    int N;
    cin >> N;
    m = n = cntx = cnty = 0;
    for (int i = 1; i <= N; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        x1 += 1e5, x2 += 1e5, y1 += 1e5, y2 += 1e5;
        infoy[++cnty] = { x1, y1, y2, 1 };
        infoy[++cnty] = { x2, y1, y2, -1 };
        fy[++m] = y1;
        fy[++m] = y2;

        infox[++cntx] = { y1, x1, x2, 1 };
        infox[++cntx] = { y2, x1, x2, -1 };
        fx[++n] = x1;
        fx[++n] = x2;

    }
    sort(infox + 1, infox + cntx + 1, cmp);
    sort(infoy + 1, infoy + cnty + 1, cmp);
    Disc();
    ll ans = 0;
    //处理 y
    build(1, m, 1, fy);
    for (int i = 1, pre = 0; i <= cnty; i++) {
        pre = node[1].cover;
        add(rankk(fy, m, infoy[i][1]), rankk(fy, m, infoy[i][2]) - 1,
infoy[i][3], 1, m, 1);
        ans += llabs(node[1].cover - pre);
    }
    //处理 x
    build(1, n, 1, fx);
    for (int i = 1, pre = 0; i <= cntx; i++) {
        pre = node[1].cover;
        add(rankk(fx, n, infox[i][1]), rankk(fx, n, infox[i][2]) - 1,
infox[i][3], 1, n, 1);
        ans += llabs(node[1].cover - pre);
    }
}

```

```

    }
    cout << ans << '\n';
}

```

线段树和扫描线(矩阵面积并)

```

#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const int MAXN = 1e5;
array<int, 4>info[2 * MAXN + 5];
int fy[2 * MAXN + 5];
int m, cnt;

struct {
    ll len;
    ll cover;
    int cnt;
}node[8 * MAXN + 5];

void up(int i, int l, int r) {
    if (l == r) {
        node[i].cover = node[i].cnt > 0 ? node[i].len : 0;
    }
    else {
        node[i].cover = node[i].cnt > 0 ? node[i].len : node[i << 1].cover
+ node[i << 1 | 1].cover;
    }
}

void build(int l, int r, int i) {
    node[i].len = fy[r + 1] - fy[l];
    node[i].cover = 0;
    node[i].cnt = 0;
    if (l != r) {
        int mid = (l + r) >> 1;
        build(l, mid, i << 1);
        build(mid + 1, r, i << 1 | 1);
        up(i, l, r);
    }
}

void add(int jobl, int jobr, int jobv, int l, int r, int i) {
    if (jobl <= l && r <= jobr) {
        node[i].cnt += jobv;
    }
}

```

```

    else {
        int mid = (l + r) >> 1;
        if (jobl <= mid) {
            add(jobl, jobr, jobv, l, mid, i << 1);
        }
        if (jobr > mid) {
            add(jobl, jobr, jobv, mid + 1, r, i << 1 | 1);
        }
    }
    up(i, l, r);
}

void Disc() {
    sort(fy + 1, fy + m + 1);
    fy[0] = -1;
    int k = m; m = 0;
    for (int i = 1; i <= k; i++) {
        if (fy[i] != fy[m]) {
            fy[++m] = fy[i];
        }
    }
    fy[m + 1] = fy[m];
}

int rankk(int* disc, int n, int v) {
    int l = 1, r = n;
    while (l <= r) {
        int mid = (l + r) >> 1;
        if (disc[mid] == v) {
            return mid;
        }
        else if (disc[mid] > v) {
            r = mid - 1;
        }
        else {
            l = mid + 1;
        }
    }
    return l;
}

bool cmp(array<int, 4>a, array<int, 4>b) {
    if (a[0] == b[0]) return a[3] > b[3]; //处理坐标相同问题
    return a[0] < b[0];
}

void solve() {
    int n;

```

```

cin >> n;
m = cnt = 0;
for (int i = 1; i <= n; i++) {
    int x1, y1, x2, y2;
    cin >> x1 >> y1 >> x2 >> y2;
    info[++cnt] = { x1, y1, y2, 1 };
    info[++cnt] = { x2, y1, y2, -1 };
    fy[++m] = y1;
    fy[++m] = y2;
}
sort(info + 1, info + cnt + 1, cmp);
Disc();
ll ans = 0;
build(1, m, 1);
for (int i = 1; i <= cnt; i++) {
    if (i > 1) {
        ans += ll(info[i][0] - info[i - 1][0]) * ll(node[1].cover);
    }
    add(rankk(fy, m, info[i][1]), rankk(fy, m, info[i][2]) - 1,
info[i][3], 1, m, 1);
}
cout << ans << '\n';
}

```